CSE 4303
Data Structures

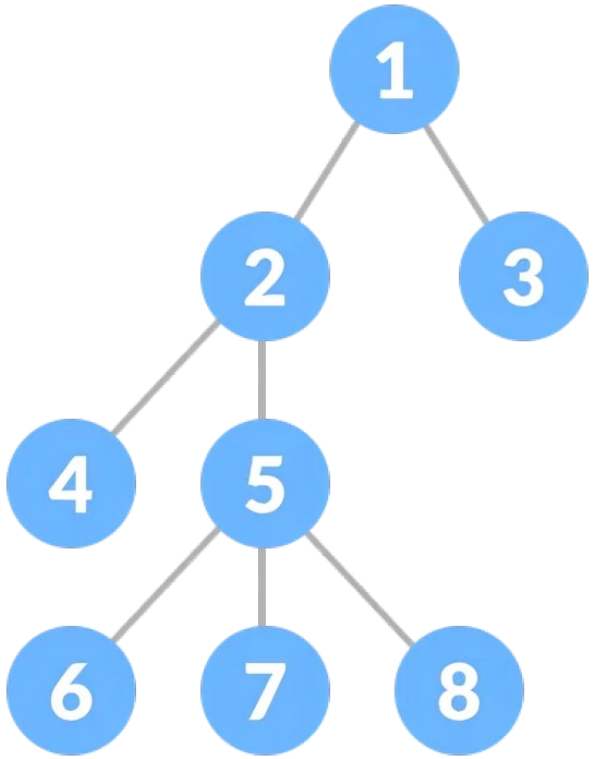
Topic: Trees

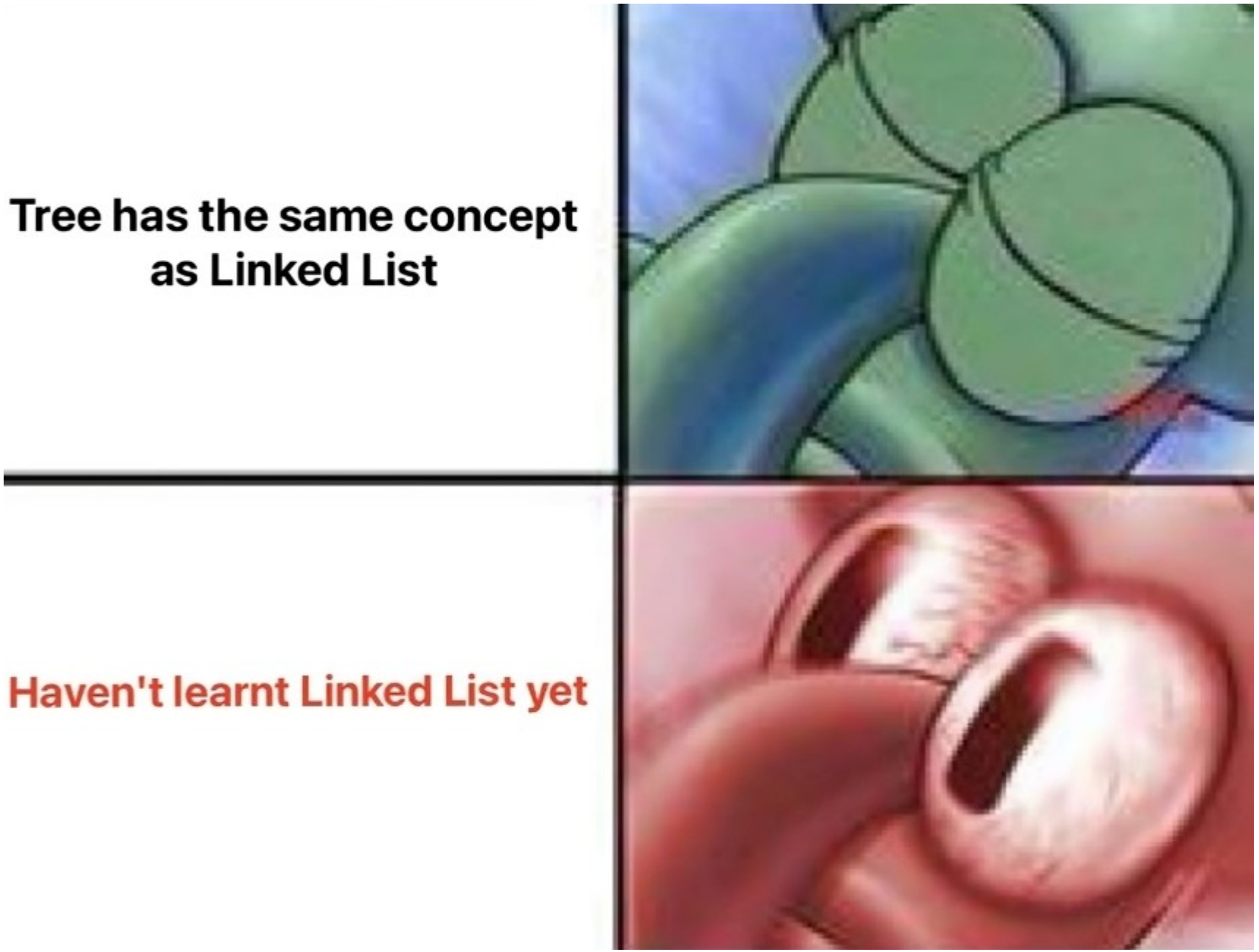
Sabbir Ahmed
Assistant Prof | CSE | IUT
sabbirahmed@iut-dhaka.edu

Arrays, linked lists, stacks, queues, etc., are **linear** data structures that store data sequentially

Arrays, linked lists, stacks, queues, etc., are **linear** data structures that store data sequentially

Tree is a **nonlinear hierarchical** data structure that consists of nodes connected by edges.




Tree has the same concept as Linked List
Haven't learnt Linked List yet

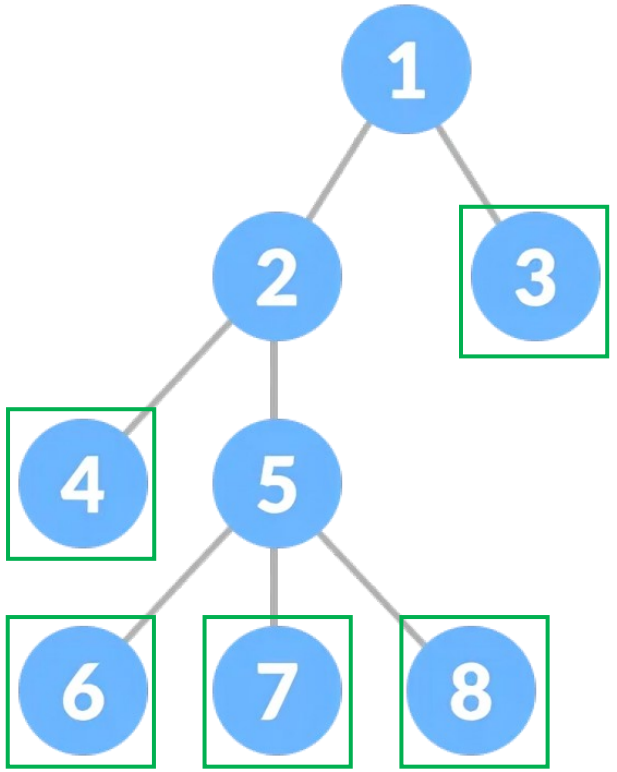
Tree Terminologies

- **Node**: an entity that contains a key or value and pointers to its child nodes.

Tree Terminologies

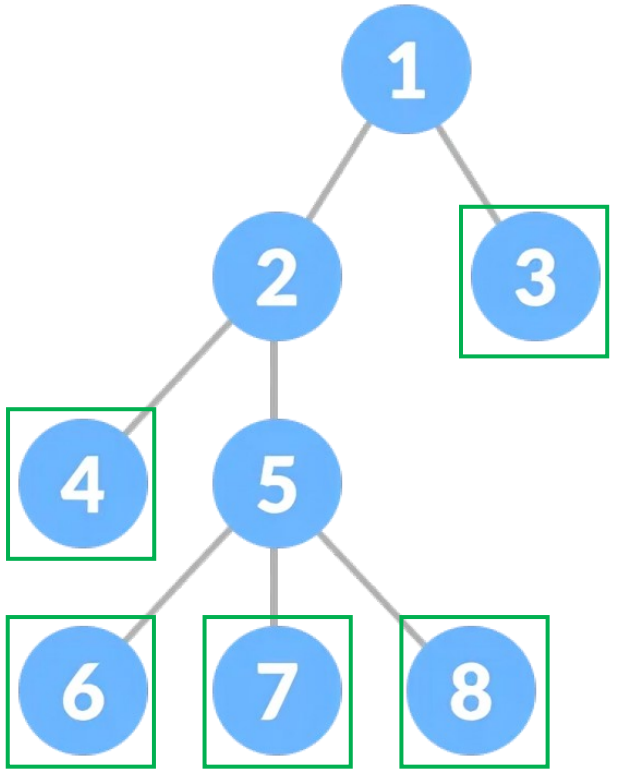
- **Node**: an entity that contains a key or value and pointers to its child nodes.

- The last nodes of each path are called leaf nodes or external nodes that do not contain a link/pointer to child nodes.



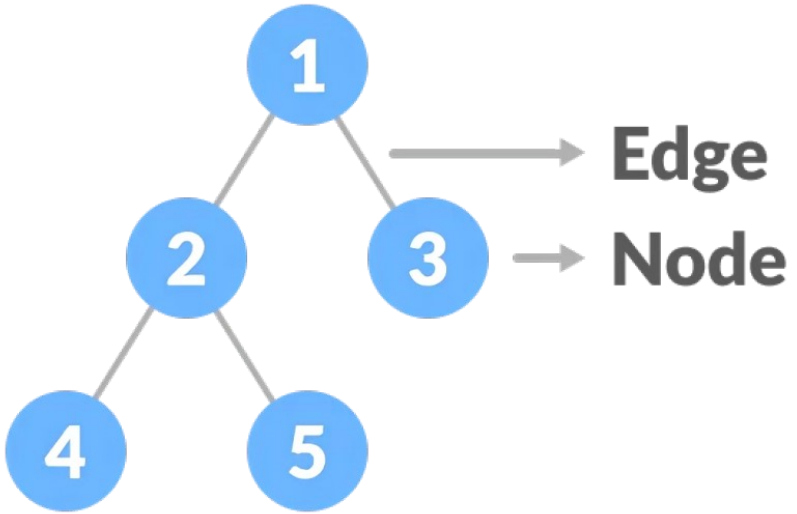
Tree Terminologies

- **Node**: an entity that contains a key or value and pointers to its child nodes.
- The last nodes of each path are called leaf nodes or external nodes that do not contain a link/pointer to child nodes.
- The node having at least a child node is called an **internal node or parent node.**



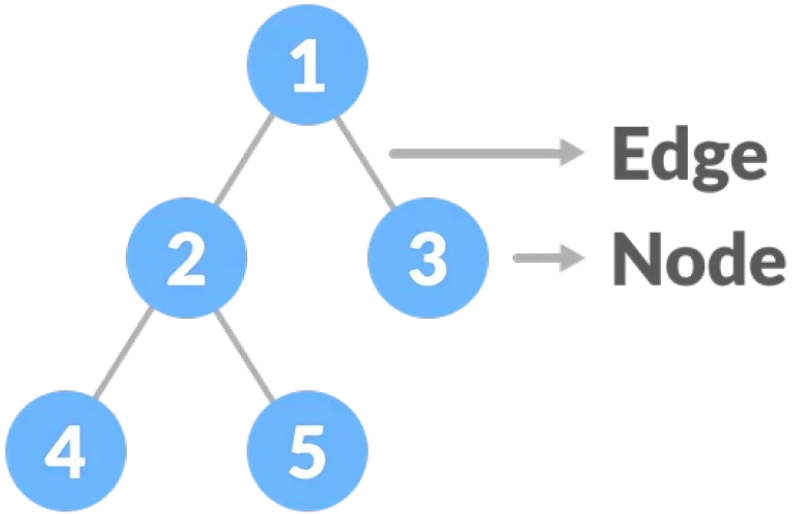
Tree Terminologies

- **Edge:** the link between any two nodes. (unidirectional)



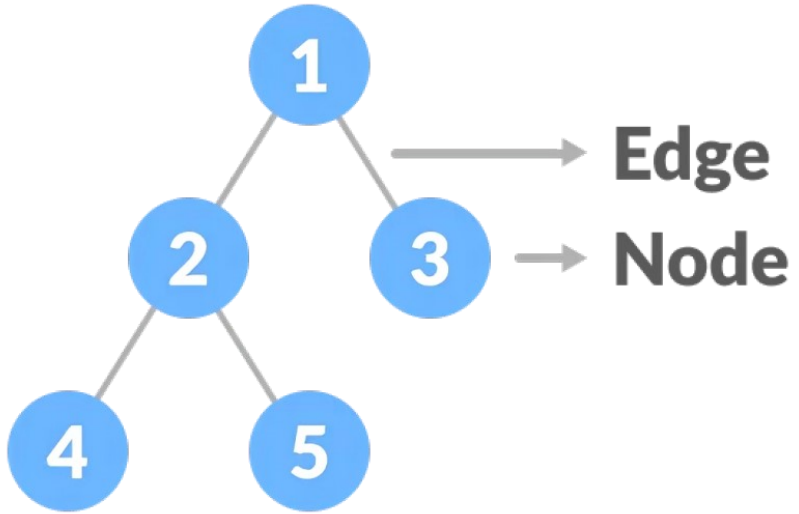
Tree Terminologies

- **Edge:** the link between any two nodes. (unidirectional)
- **Root**: It is the first/topmost node of a tree.



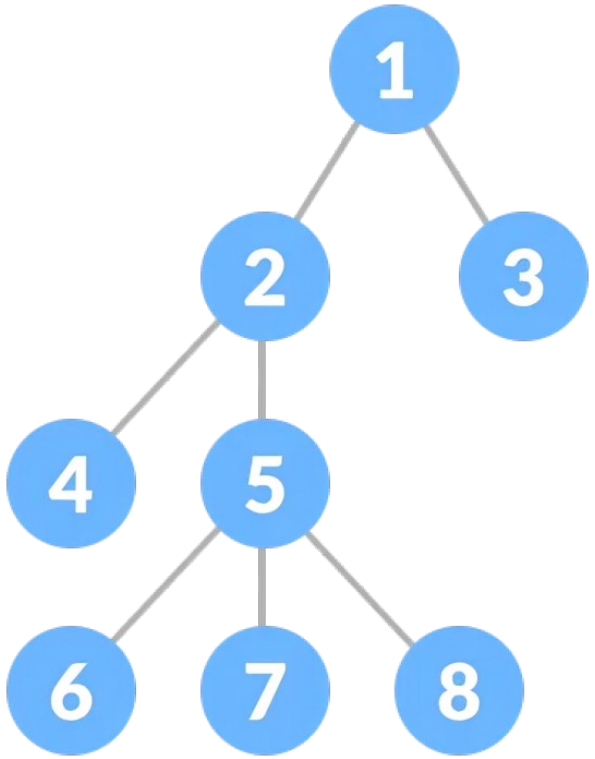
Tree Terminologies

- **Edge:** the link between any two nodes. (unidirectional)
- **Root**: It is the first/topmost node of a tree.
- **Degree of a Node:** the total number of branches/ children of that node.



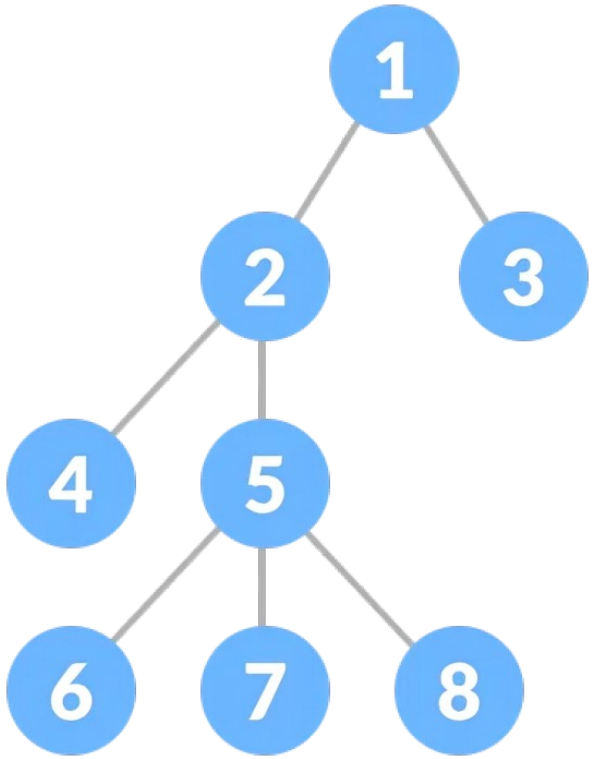
Tree Terminologies

- **Path:** the sequence of nodes and edges from one node to another node



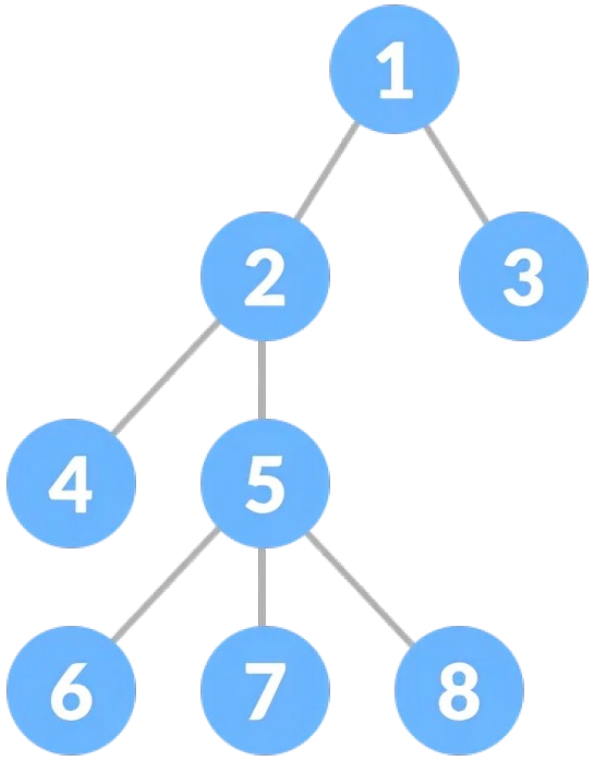
Tree Terminologies

- **Path:** the sequence of nodes and edges from one node to another node
- Path between (1,6): ??



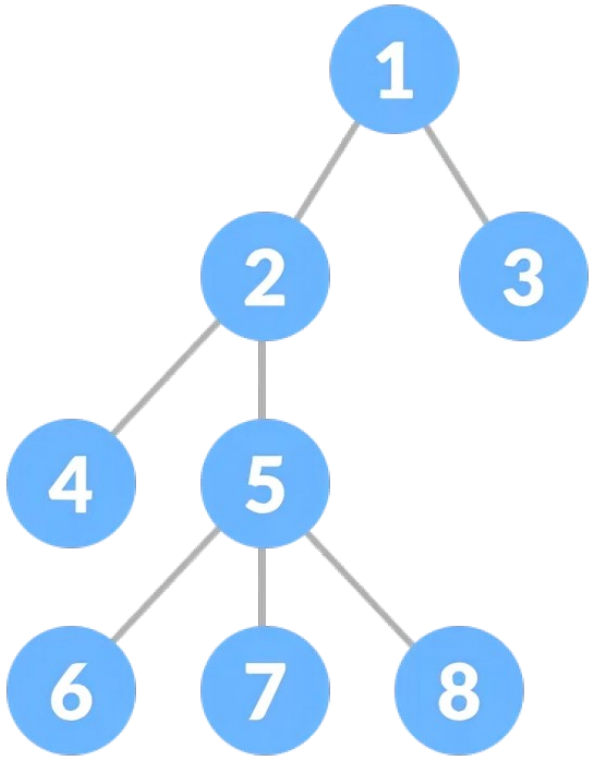
Tree Terminologies

- **Path:** the sequence of nodes and edges from one node to another node
- Path between (1,6): 1-2-5-6
- Path between (2,7): 2-5-7



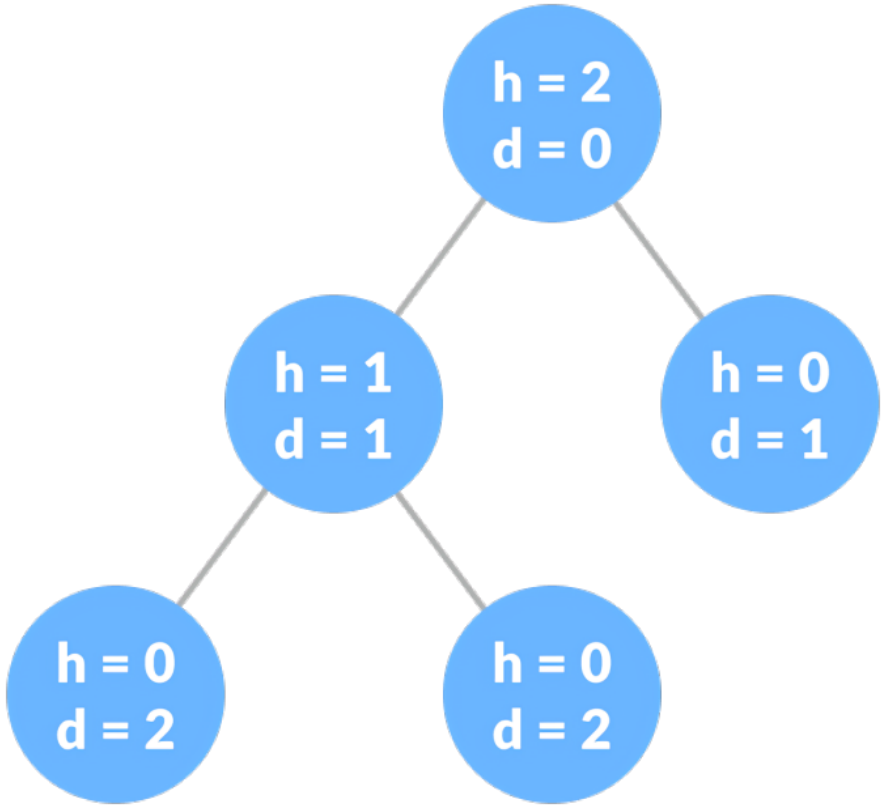
Tree Terminologies

- **Path:** the sequence of nodes and edges from one node to another node
- Path between (1,6): 1-2-5-6
- Path between (2,7): 2-5-7
- Length of a path: total number of nodes in a path.



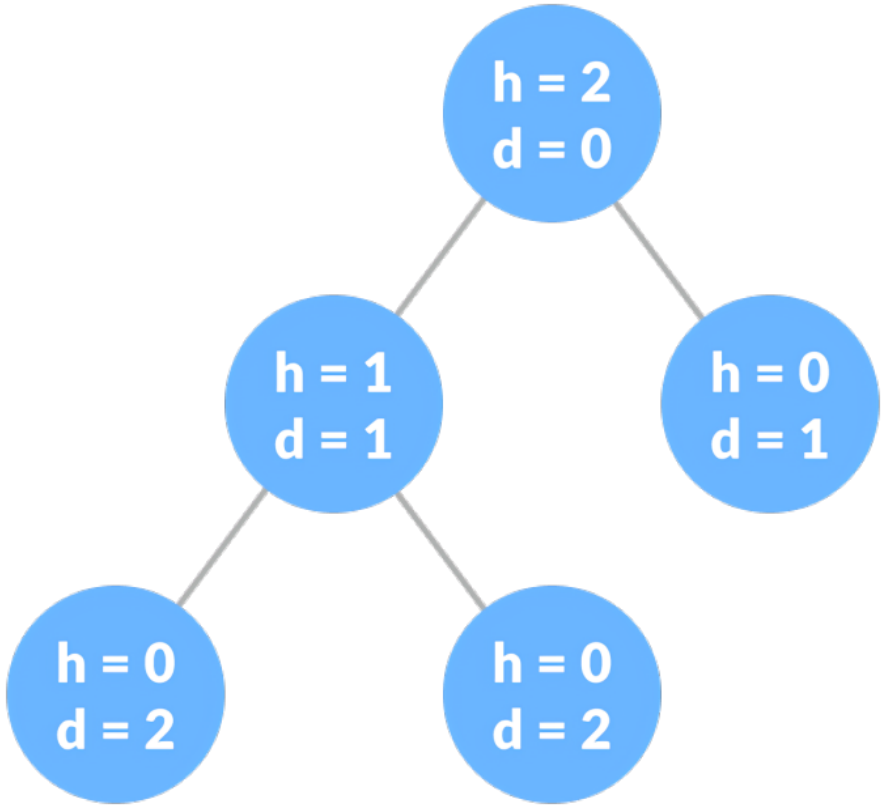
Tree Terminologies

- **Height of a Node:** #of edges from that node to the deepest leaf (the longest path from the node to a leaf node).



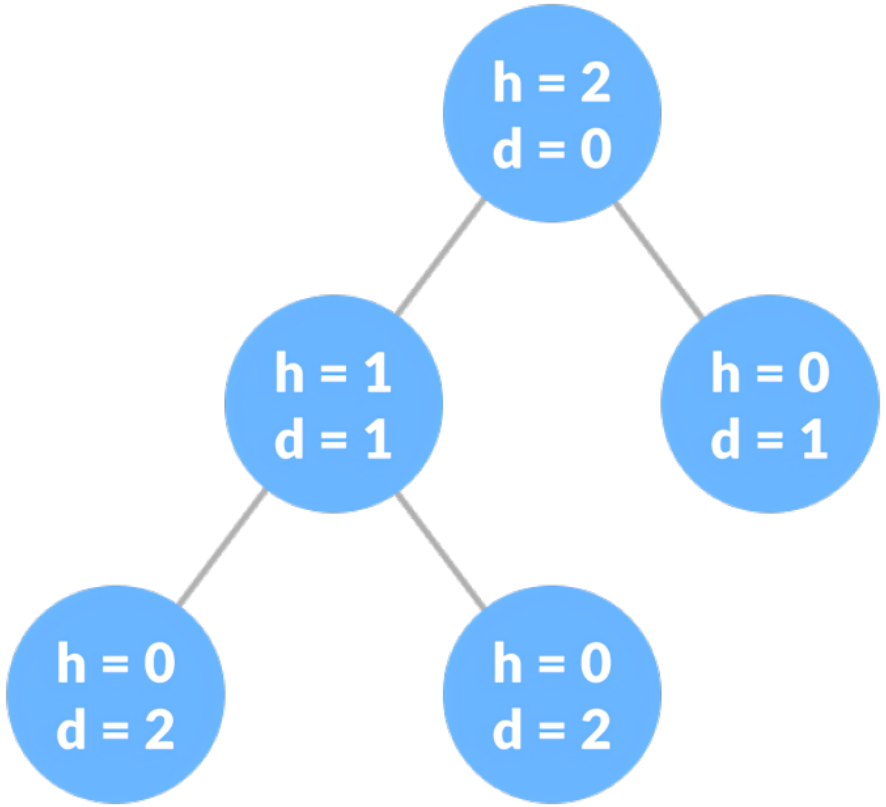
Tree Terminologies

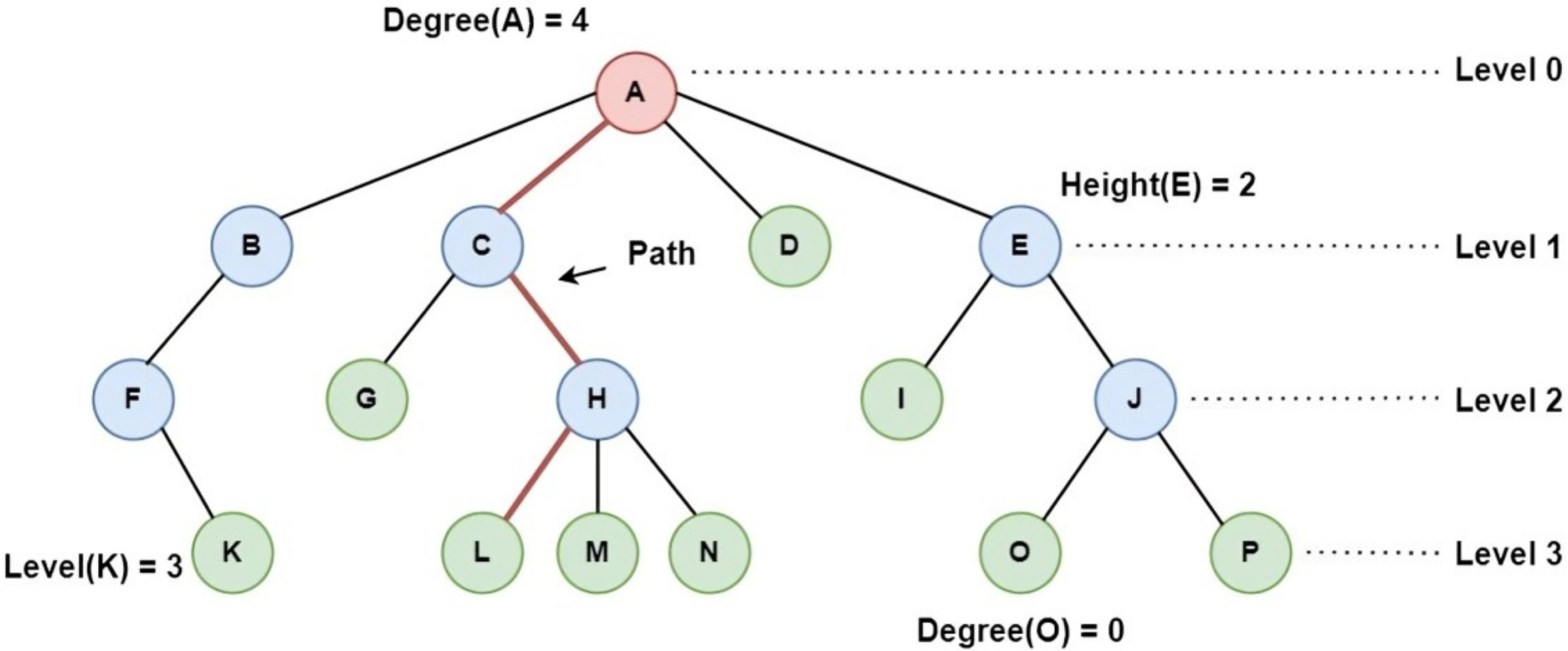
- **Height of a Node:** #of edges from that node to the deepest leaf (the longest path from the node to a leaf node).
- **Height of a Tree**: the height of the root node.



Tree Terminologies

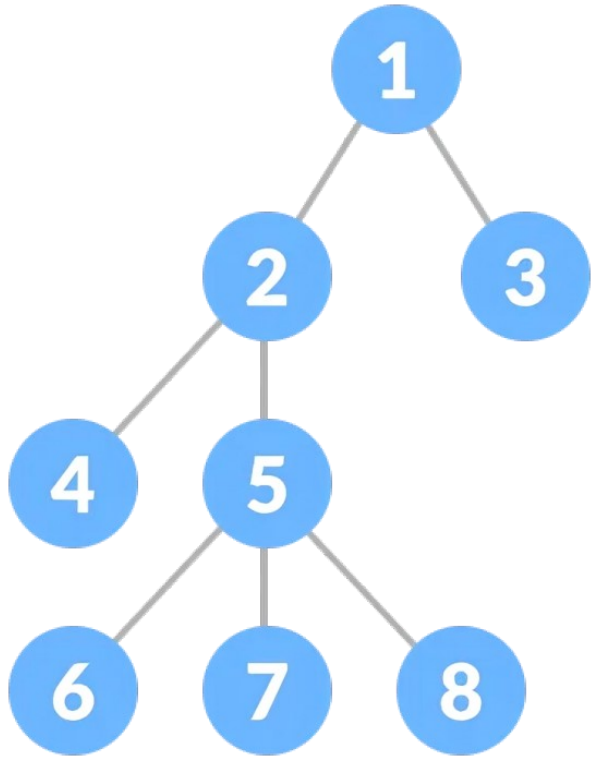
- **Height of a Node:** #of edges from that node to the deepest leaf (the longest path from the node to a leaf node).
- **Height of a Tree**: the height of the root node.
- **Depth of a Node**: #of edges from the root to that node.




Degree(A) = 4
A
Level 0
Height(E) = 2
B
C
Path
D
E
Level 1
F
G
H
I
J
Level 2
Level(K) = 3
K
L
M
N
O
P
Level 3
Degree(O) = 0

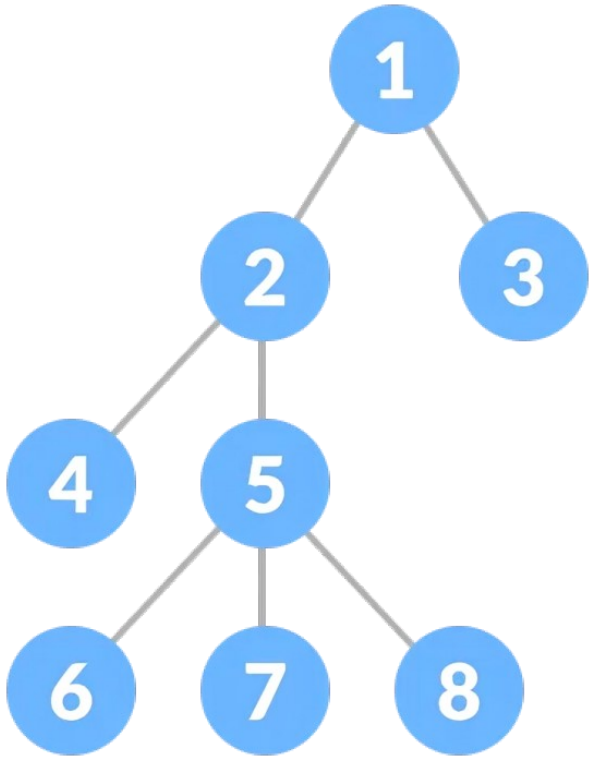
Tree Terminologies

- **Ancestor of a Node**: Any predecessor nodes on the path of the root.
- {1,2} are the ancestor nodes of the node {5}



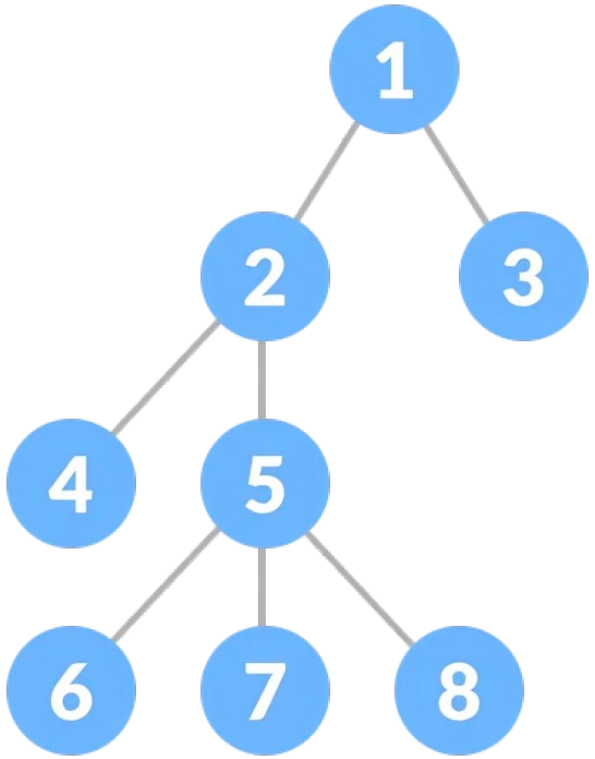
Tree Terminologies

- **Descendant**: A node x is a descendant of another node y if and only if y is an ancestor of x.
  - 5,7,8 are decendants of 2

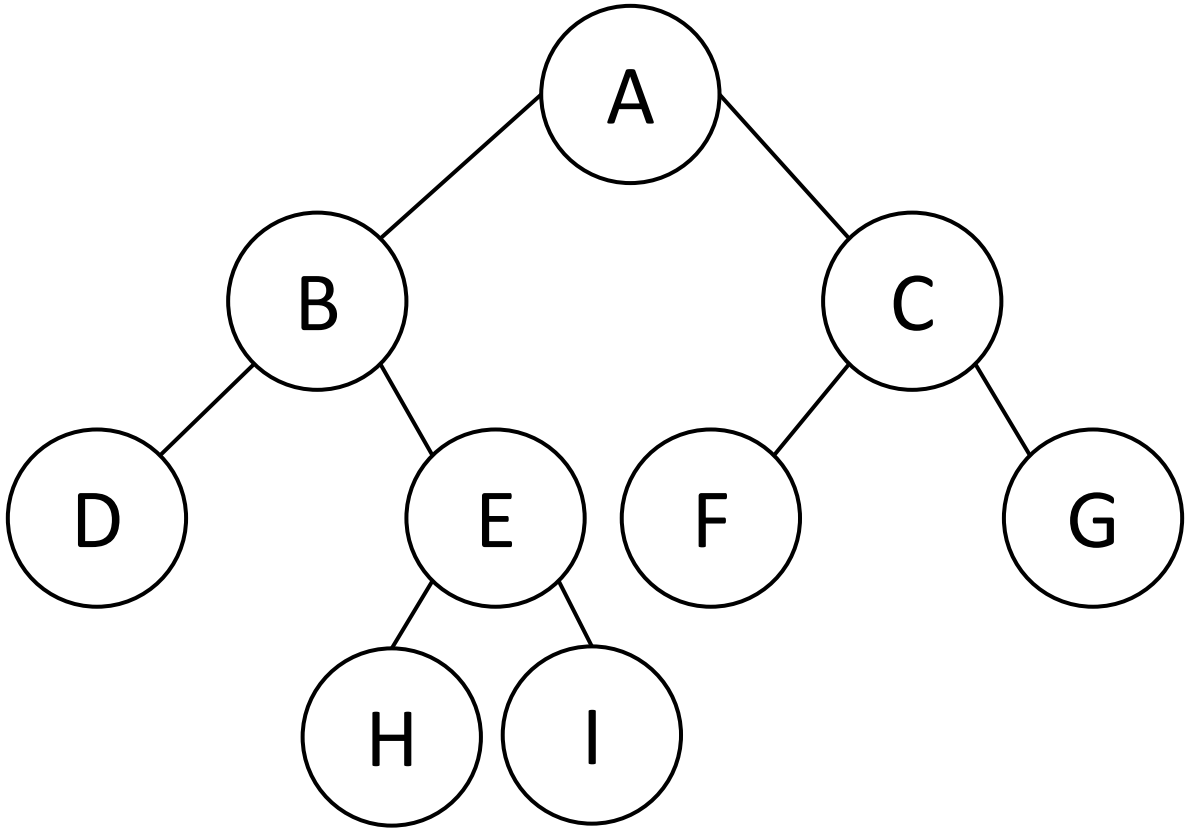


Tree Terminologies

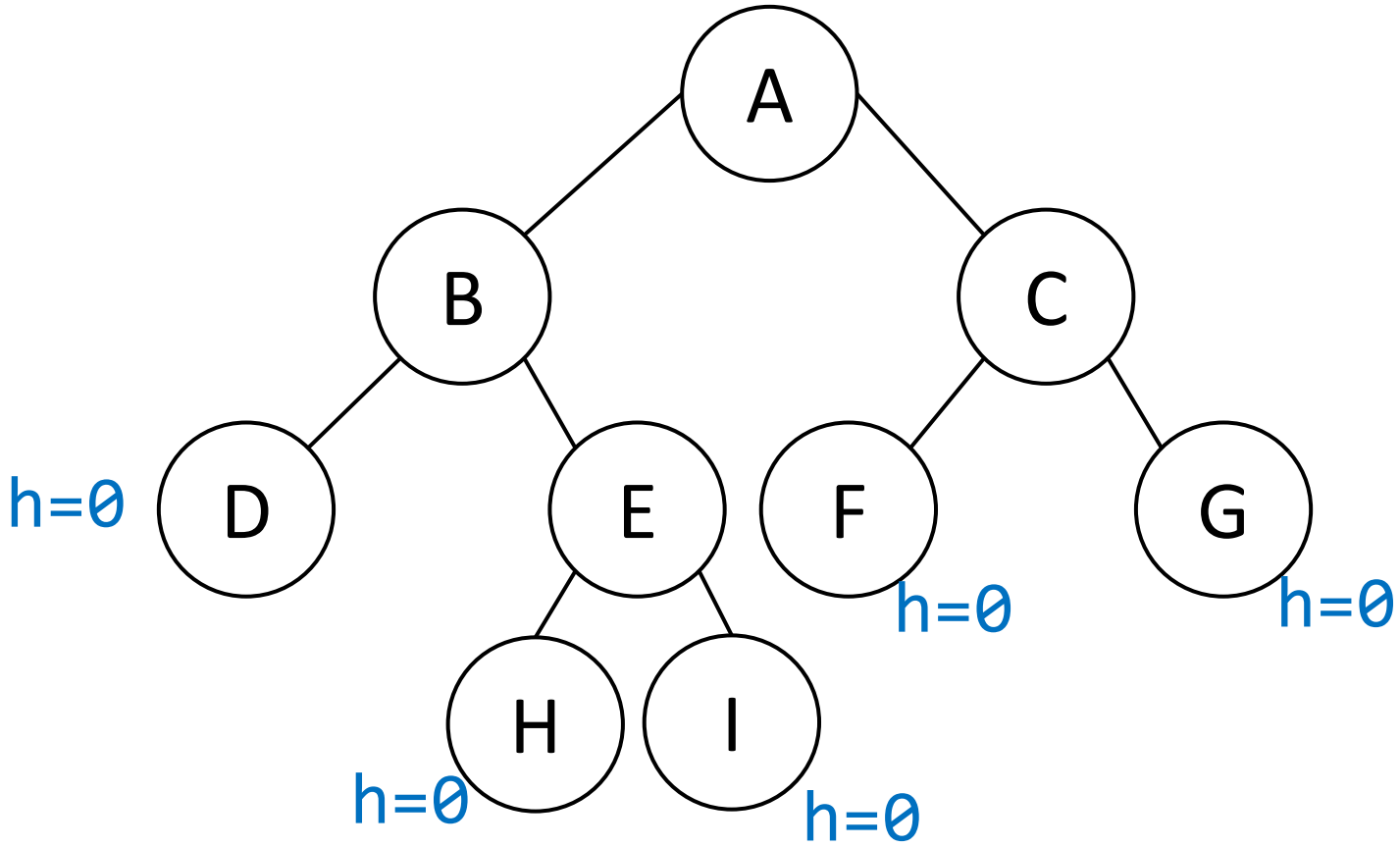
- **Descendant**: A node x is a descendant of another node y if and only if y is an ancestor of x.
  - 5,7,8 are decendants of 2
- **Sibling**: Children of the same parent node are called siblings. {6,7,8} are called siblings.



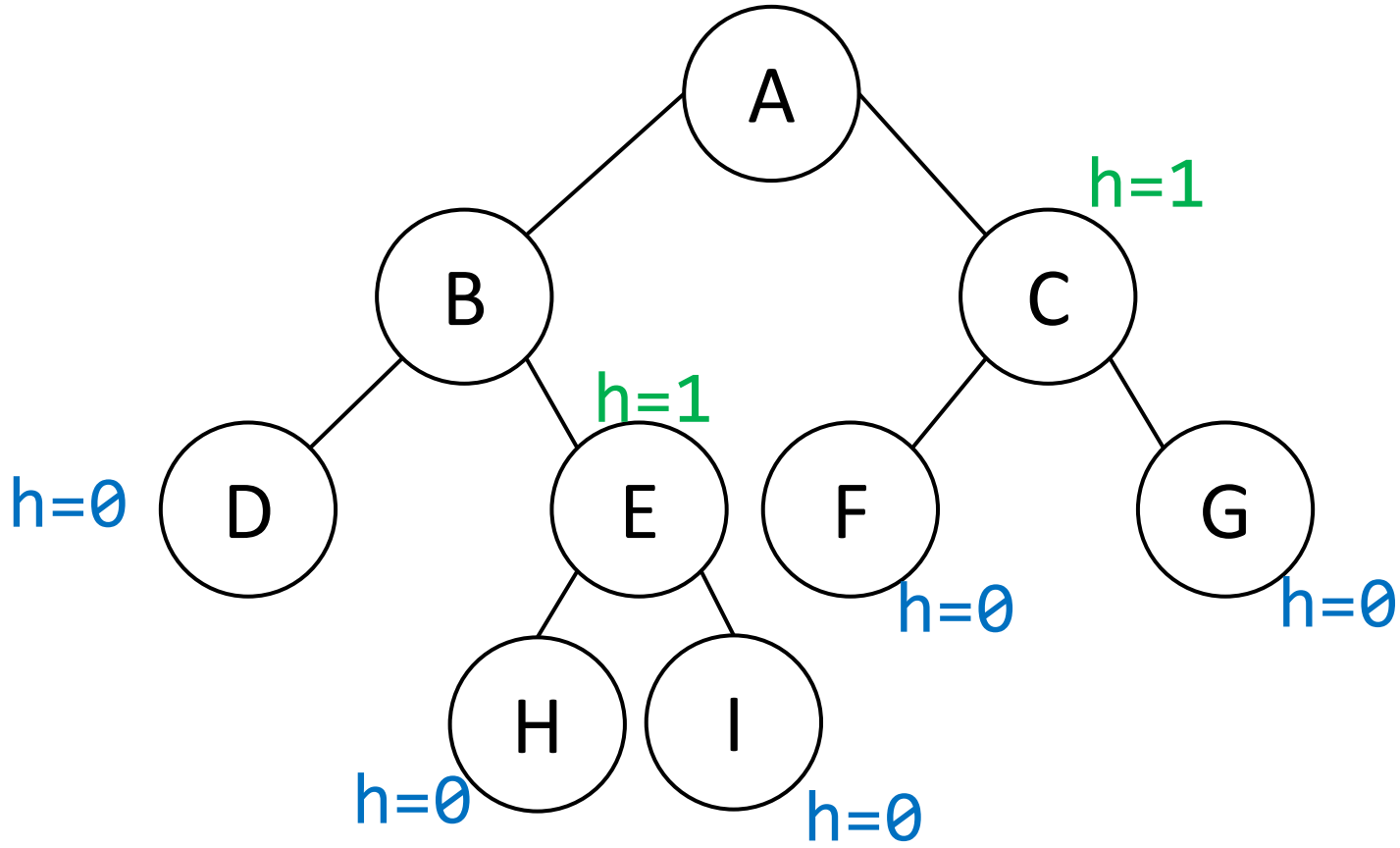
How to find the height of a node?



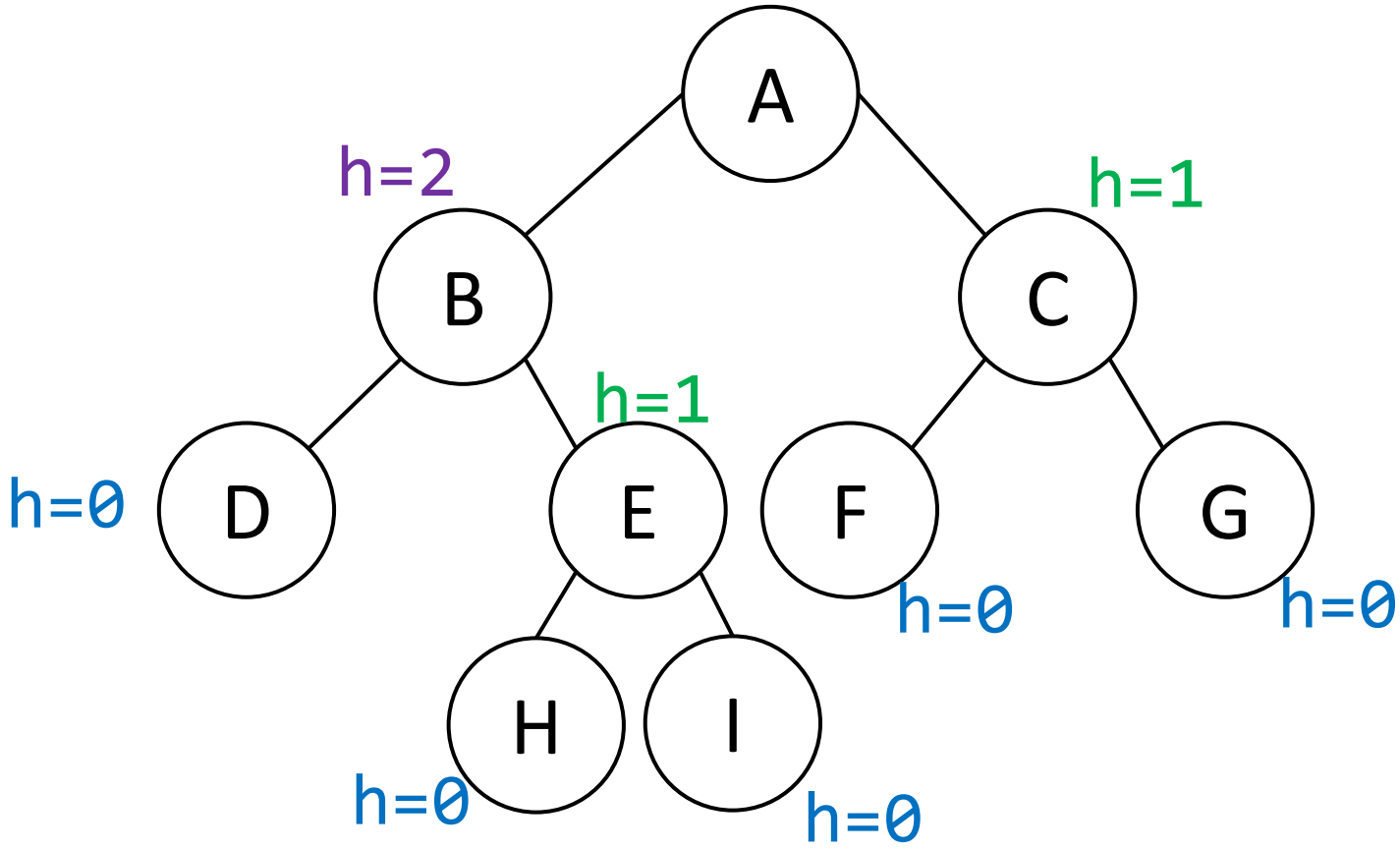
How to find the height of a node?



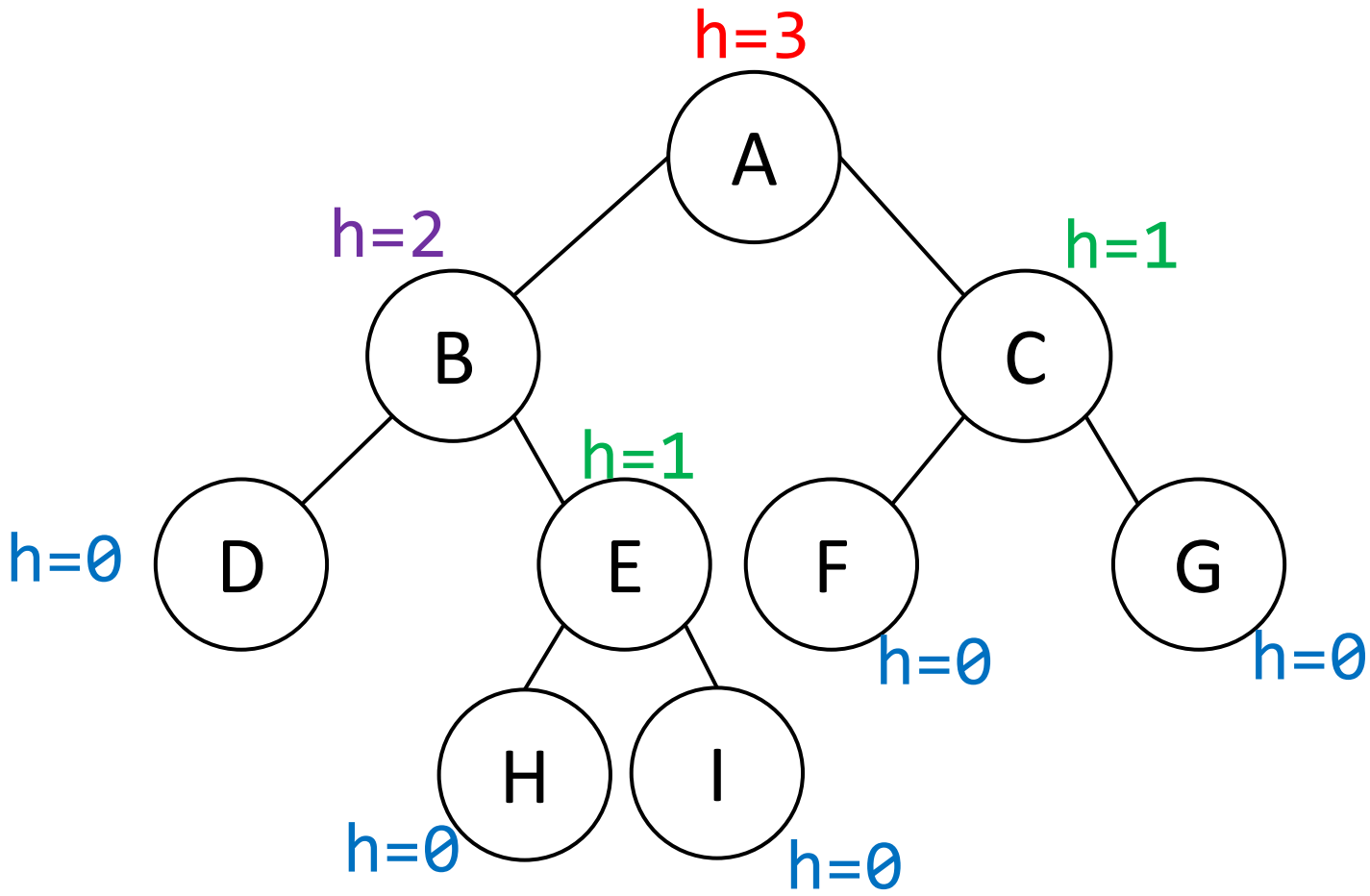
How to find the height of a node?



How to find the height of a node?

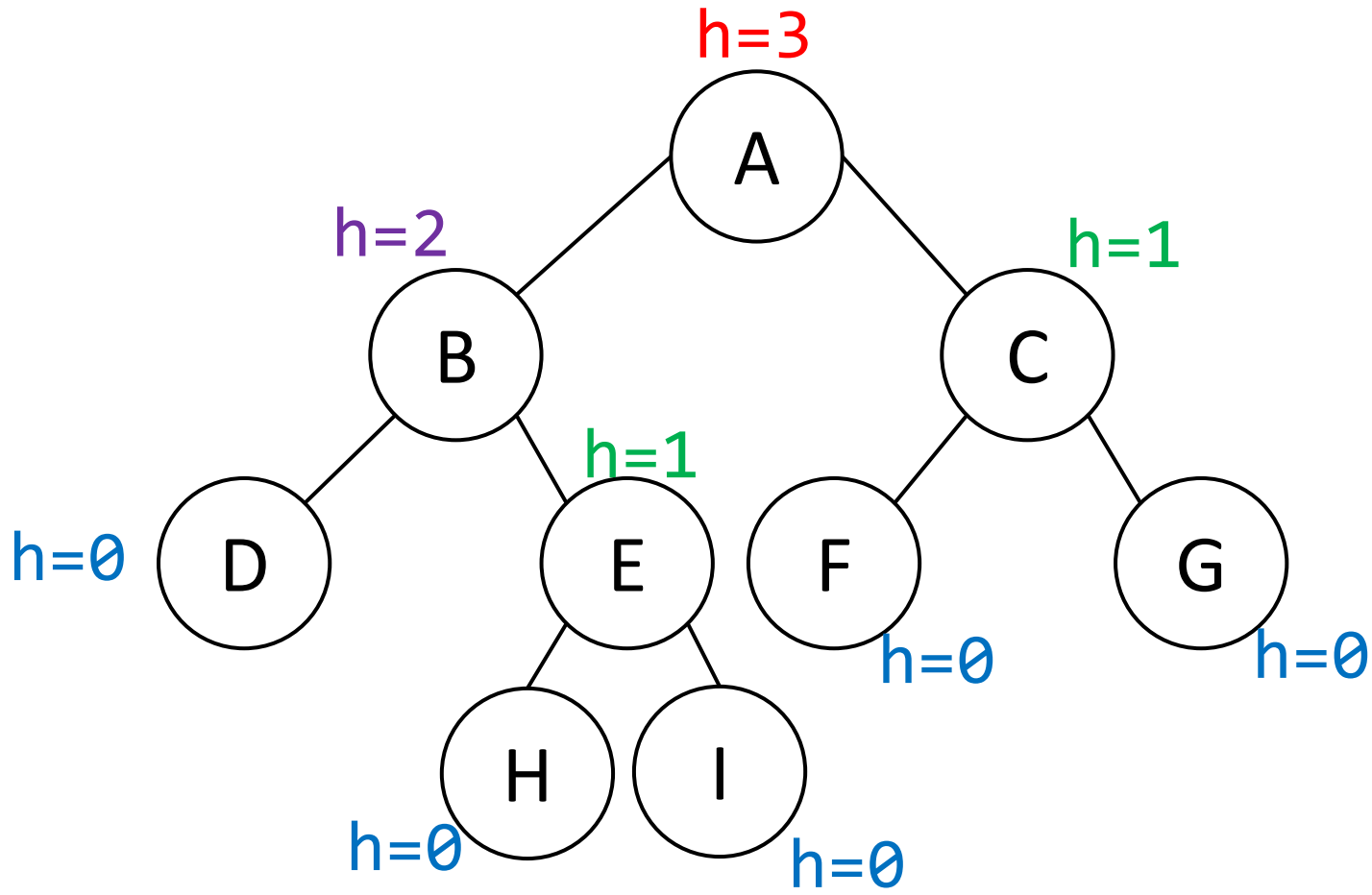


How to find the height of a node?



How to find the height of a node?

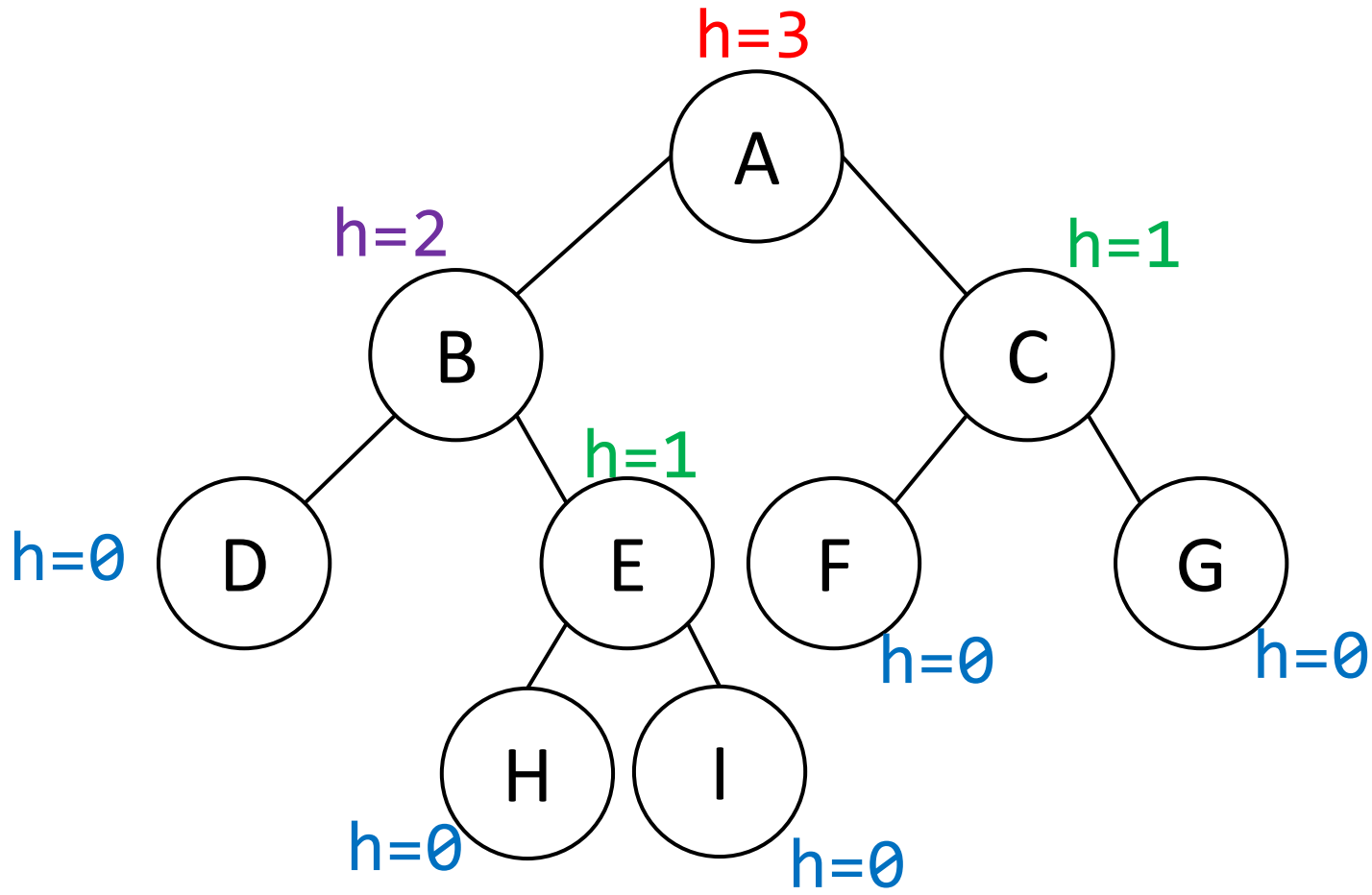
Height(A) depends on height(B) and height(C)



How to find the height of a node?

Height(A) depends on height(B) and height(C)

h(A) = max(h(B),h(C)) + 1

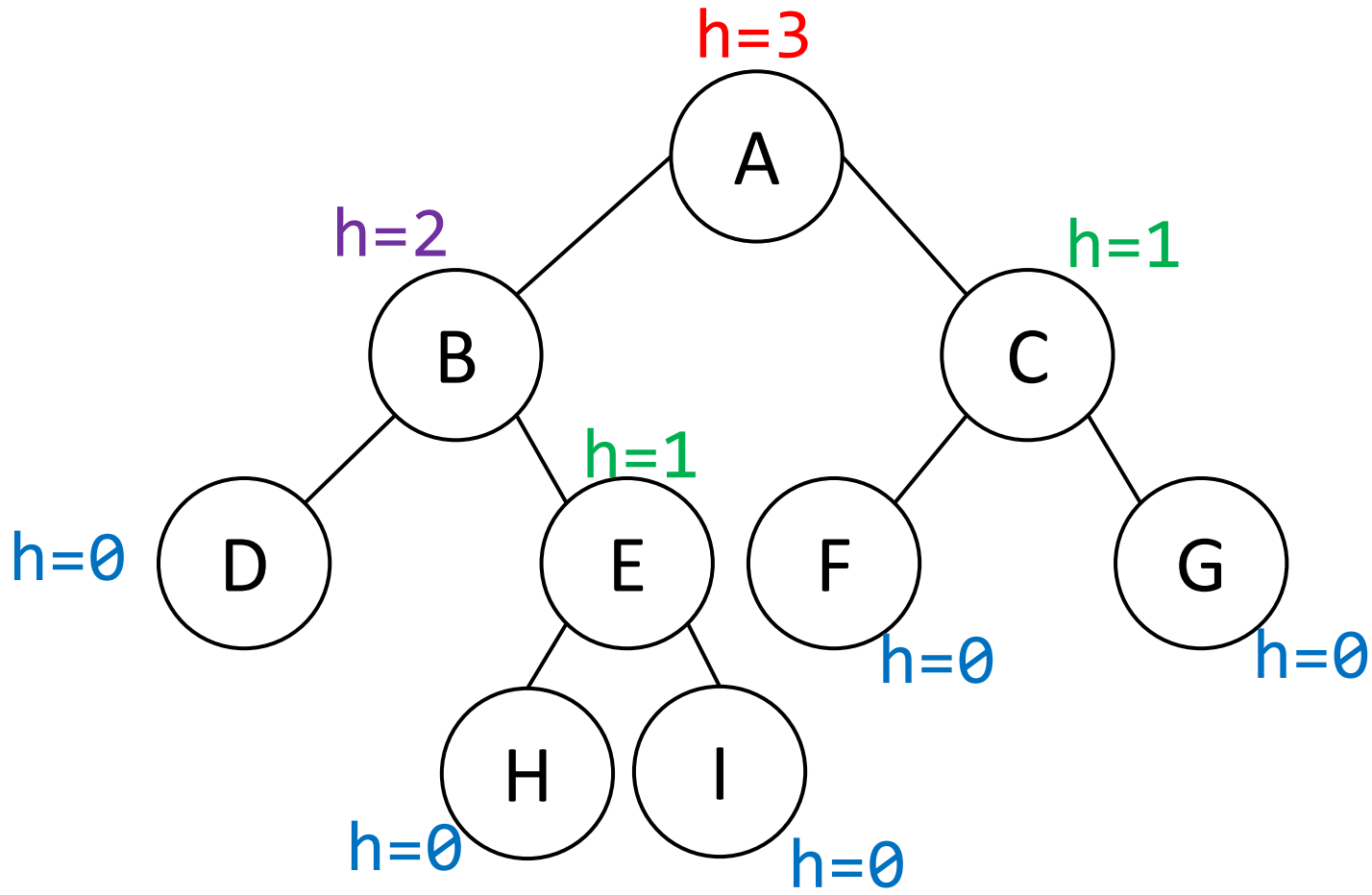


How to find the height of a node?

Height(A) depends on height(B) and height(C)

h(A) = max(h(B),h(C)) + 1

h(B) = max(h(D),h(E)) + 1

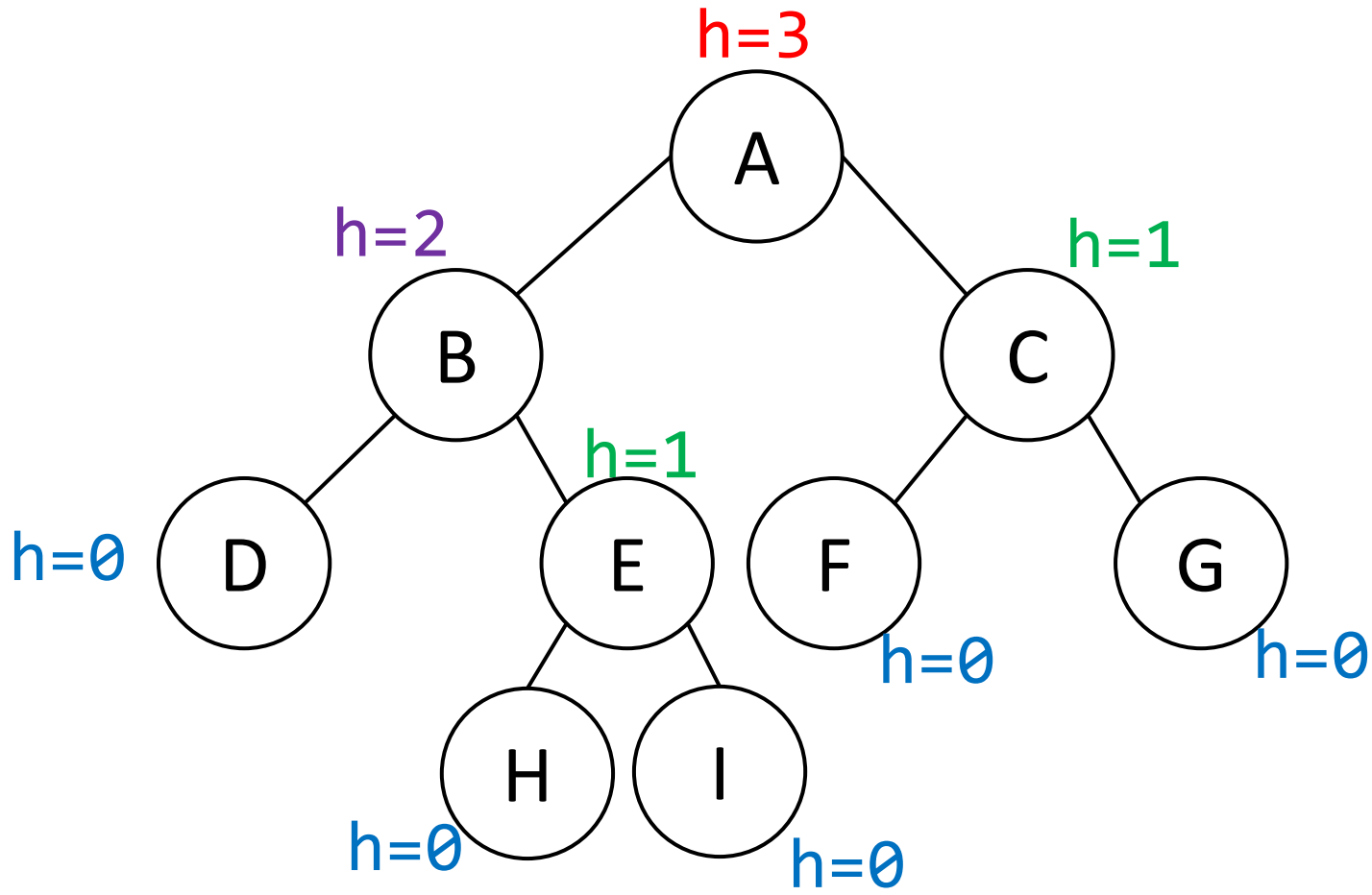


How to find the height of a node?

Height(A) depends on height(B) and height(C)

h(A) = max(h(B),h(C)) + 1

h(B) = max(h(D),h(E)) + 1




```
h(node) = max[   h(leftSubtree),
                 h(rightSubtree)
              ] + 1
```

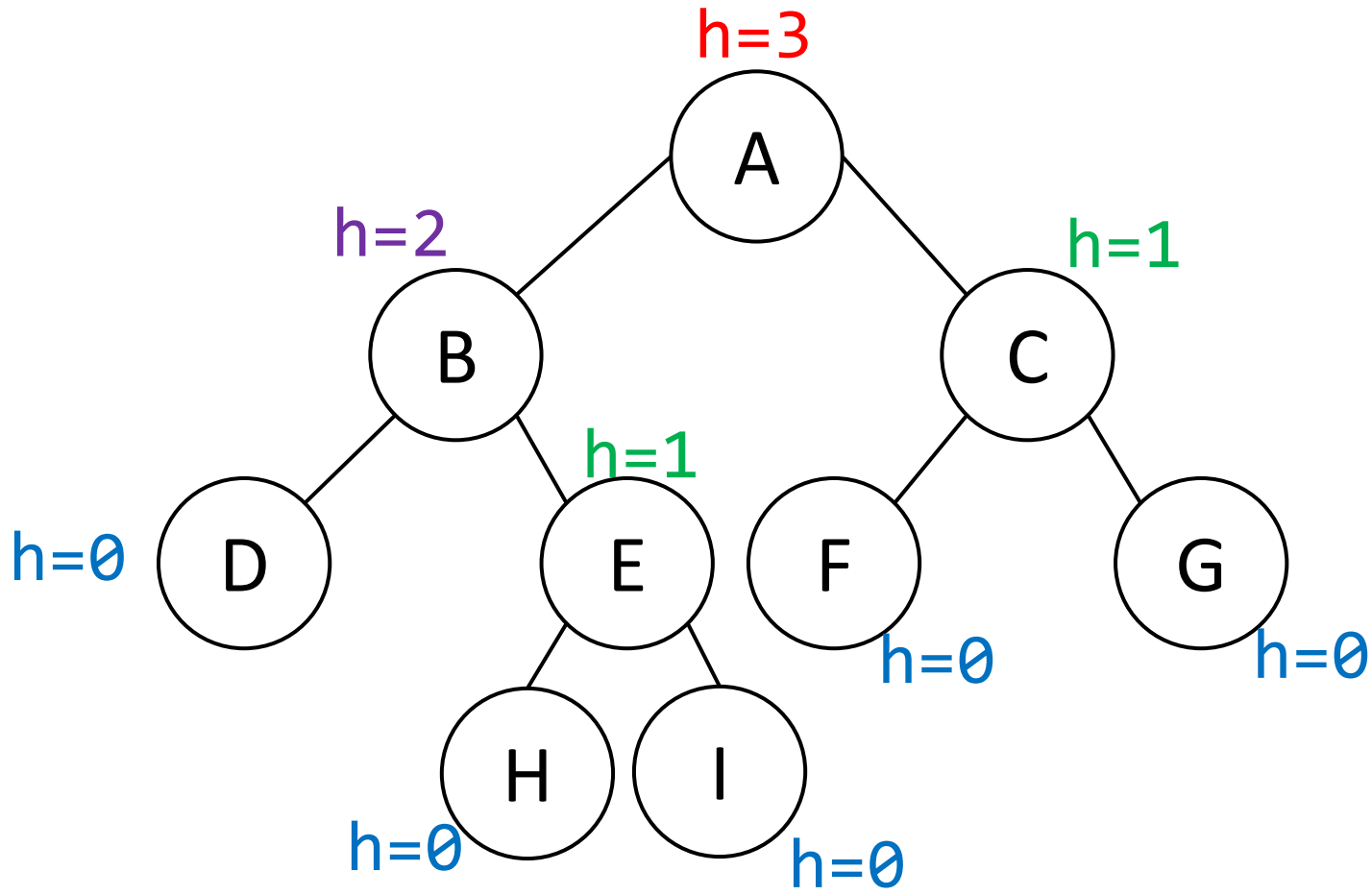
How to find the height of a node?

Height(A) depends on height(B) and height(C)

h(A) = max(h(B),h(C)) + 1

h(B) = max(h(D),h(E)) + 1

h(node) = max[h(leftSubtree),
h(rightSubtree)
] + 1



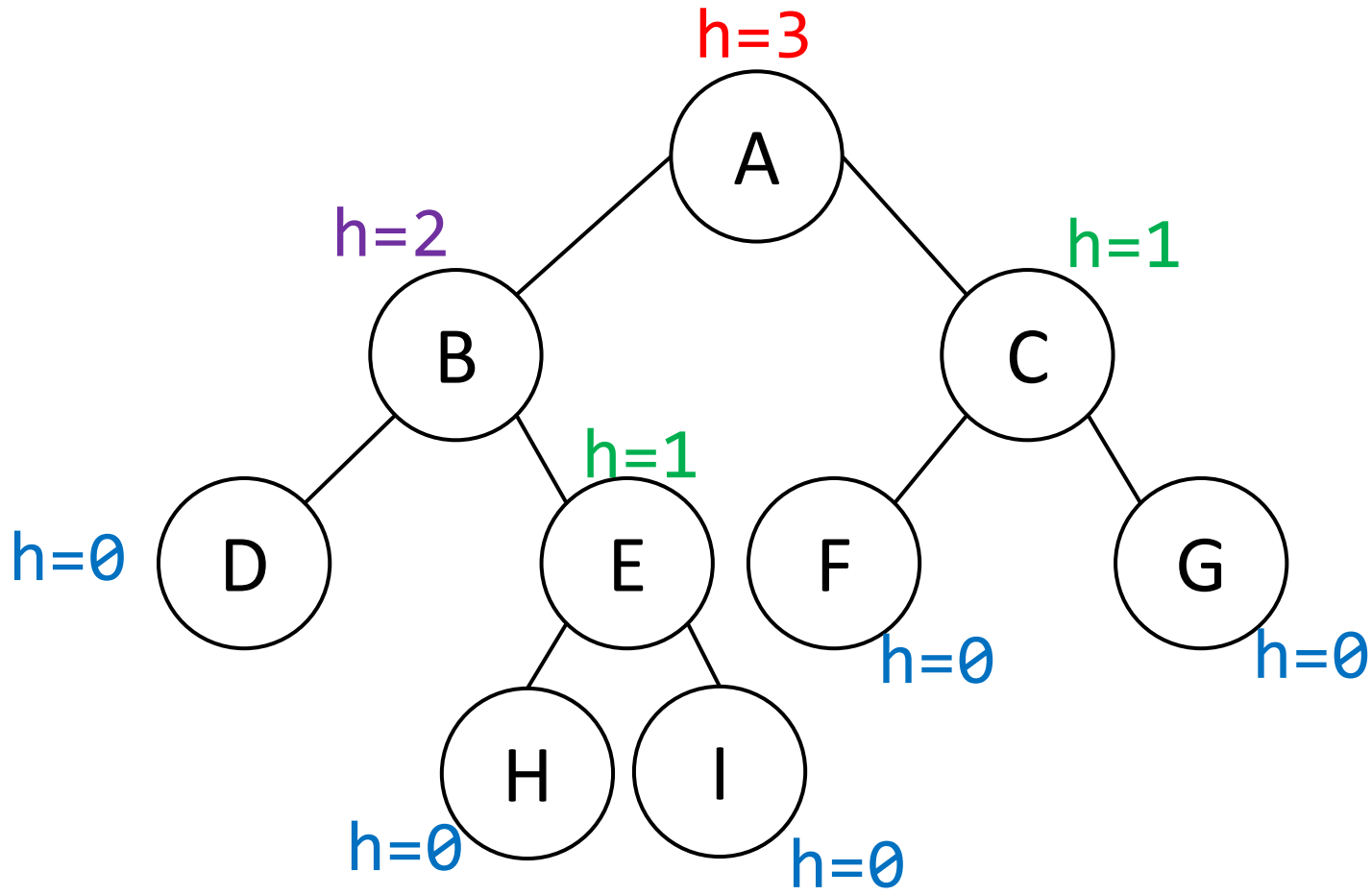
What is the height of a Null node?

How to find the height of a node?

Height(A) depends on height(B) and height(C)

h(A) = max(h(B),h(C)) + 1

h(B) = max(h(D),h(E)) + 1



h(node) = max[h(leftSubtree),
h(rightSubtree)
] + 1

What is the height of a Null node?

-1

```
int findHeight (node){   // of a binary tree




}
```

```
int findHeight (node){   // of a binary tree

    left_h  = findHeight(??)



}
```

```
int findHeight (node){   // of a binary tree

    left_h  = findHeight(node->left)


}
```

```
int findHeight (node){   // of a binary tree

    left_h  = findHeight(node->left)
    right_h = findHeight(node->right)


}
```

```
int findHeight (node){   // of a binary tree

    left_h  = findHeight(node->left)
    right_h = findHeight(node->right)

    return max(left_h, right_h)+1
}
```

```
int findHeight (node){   // of a binary tree
```

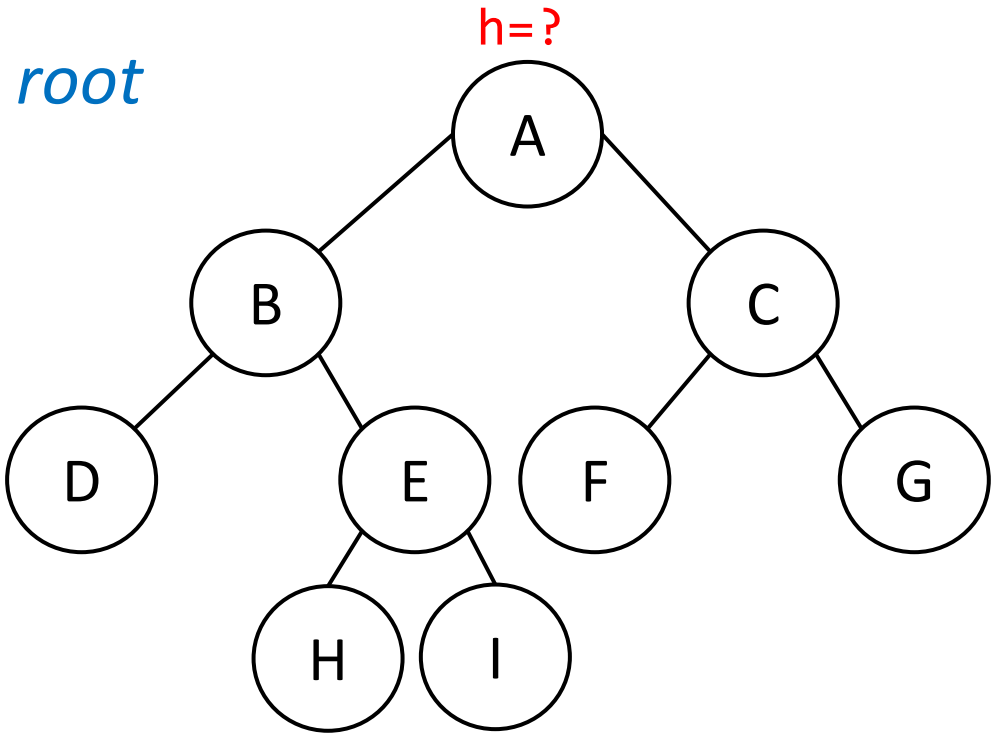
Stopping condition ?

```
    left_h  = findHeight(node->left)
    right_h = findHeight(node->right)

    return max(left_h, right_h)+1
}
```

```
int findHeight (node){   // of a binary tree
    if  (node == Null)
       ???
    left_h  = findHeight(node->left)
    right_h = findHeight(node->right)

    return max(left_h, right_h)+1
}
```

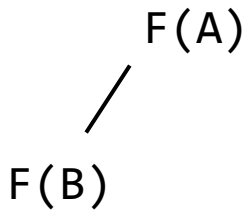
```
int findHeight (node){   // of a binary tree
    if  (node == Null)
        return -1
    left_h  = findHeight(node->left)
    right_h = findHeight(node->right)

    return max(left_h, right_h)+1
}
```

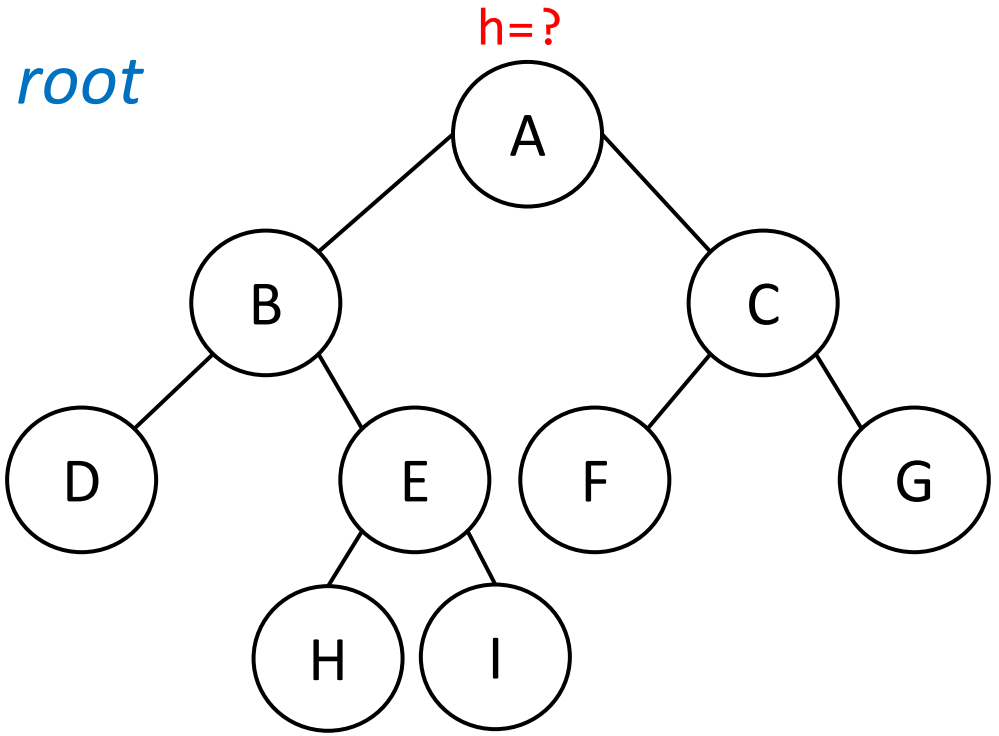
Let's find the height of this tree by passing the root into the findHeight function



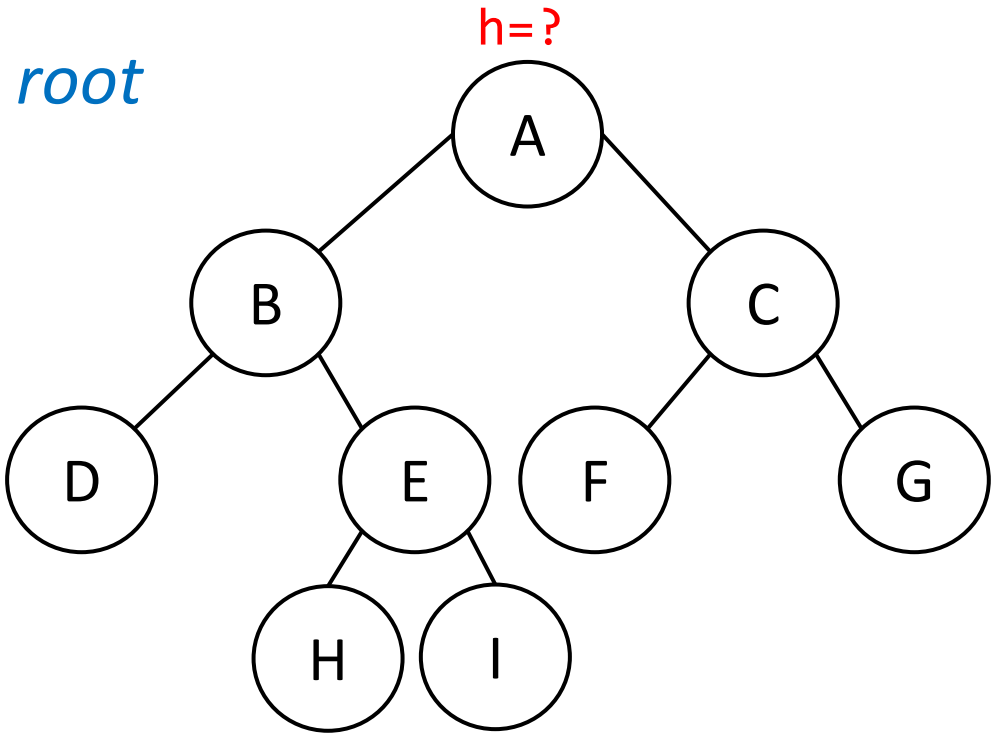
F(A)

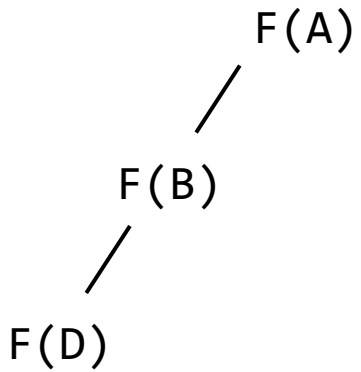
Let's find the height of this tree by passing the root into the findHeight function



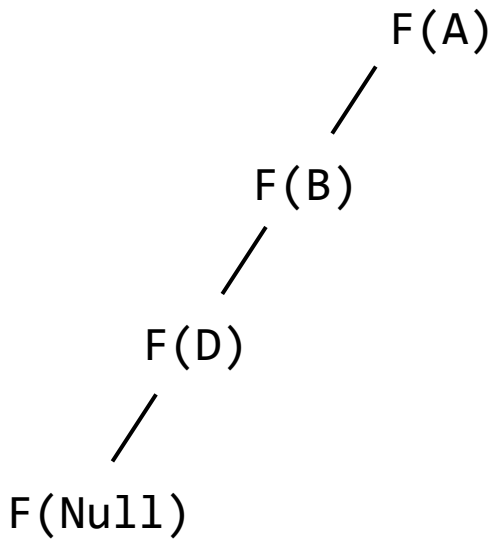


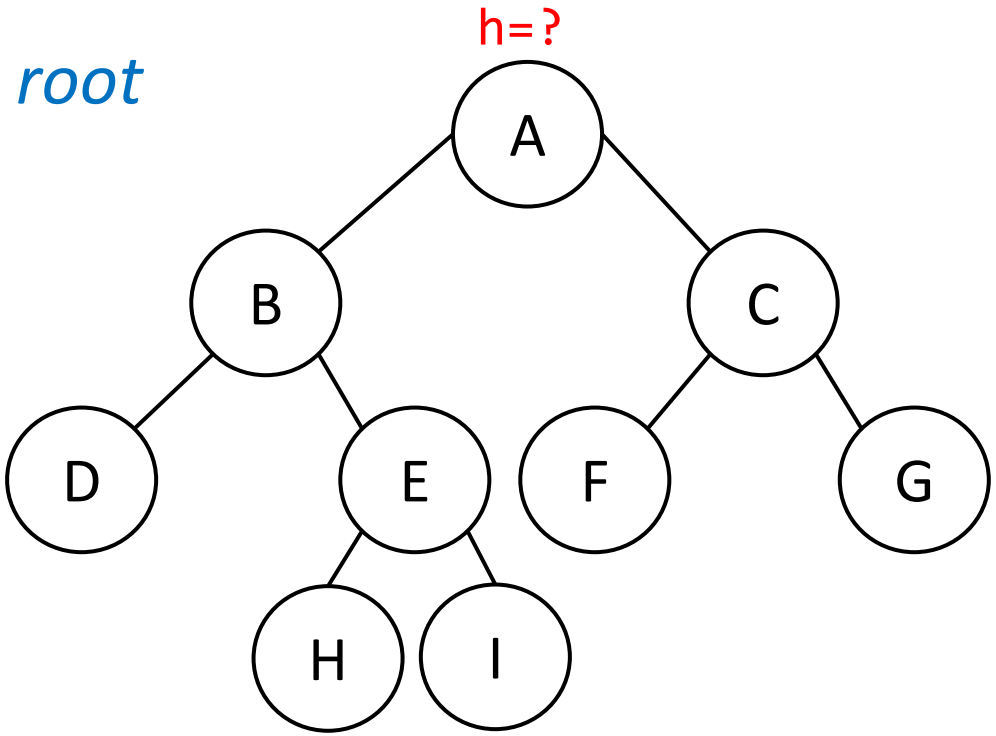
Let's find the height of this tree by passing the root into the findHeight function



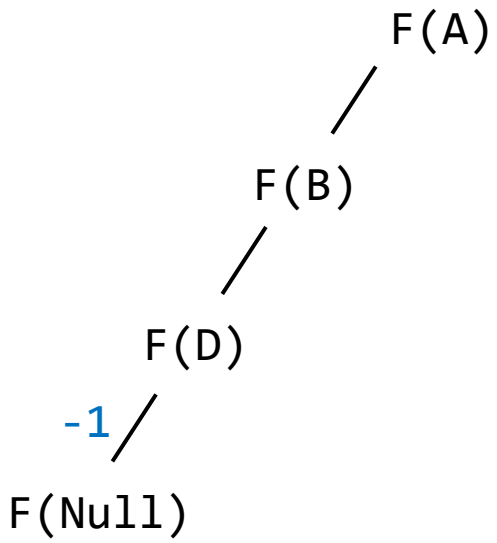


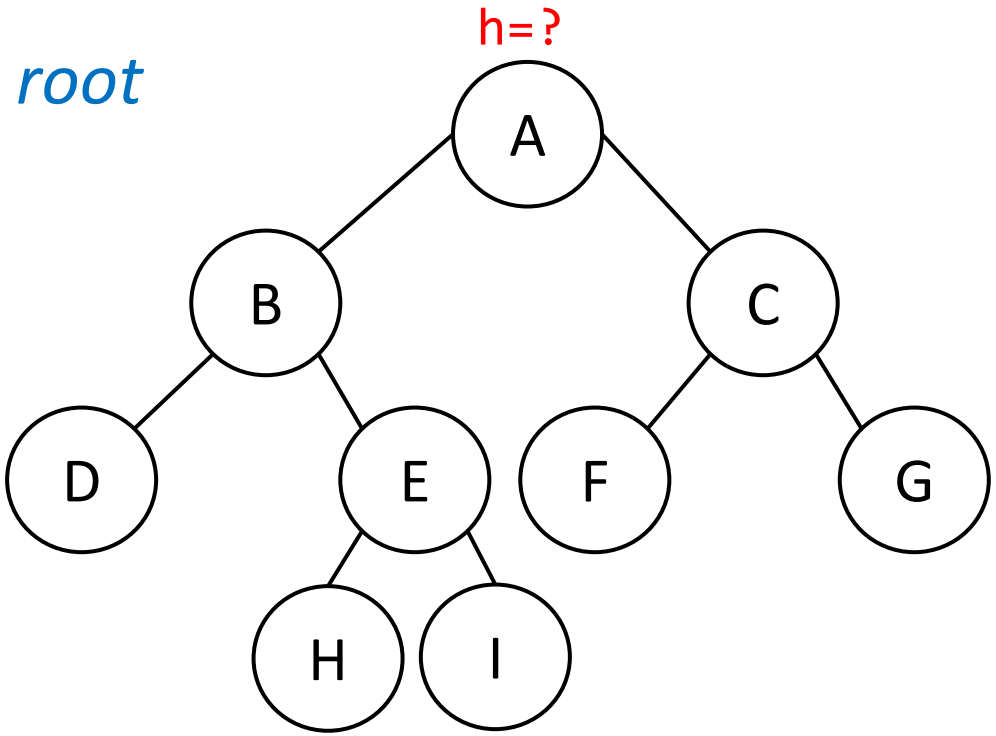
Let's find the height of this tree by passing the root into the findHeight function



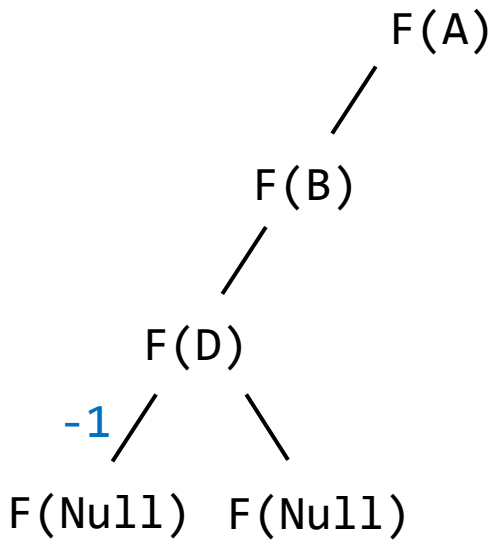


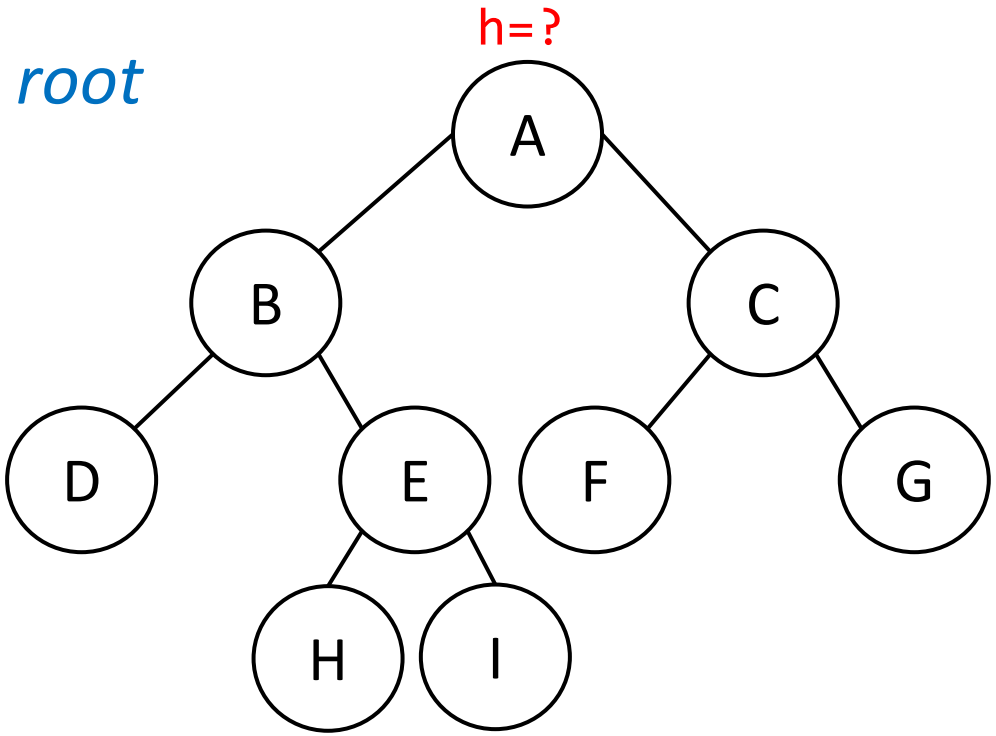
Let's find the height of this tree by passing the root into the findHeight function



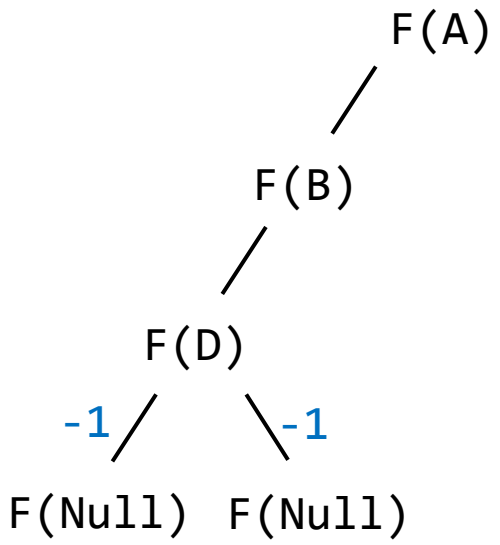


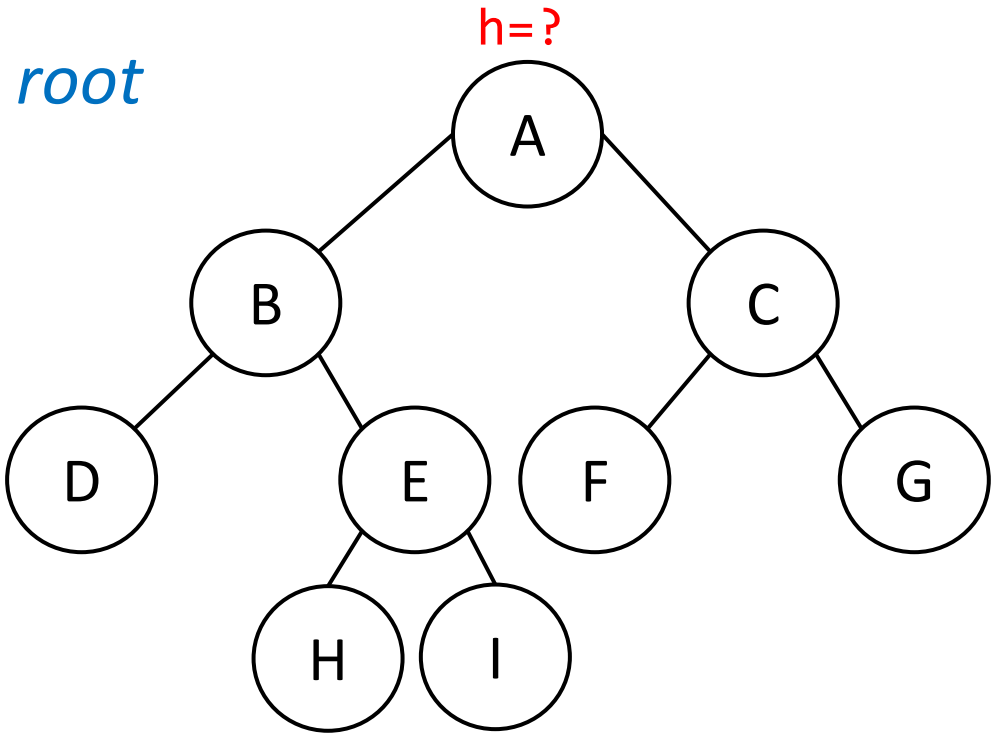
Let's find the height of this tree by passing the root into the findHeight function



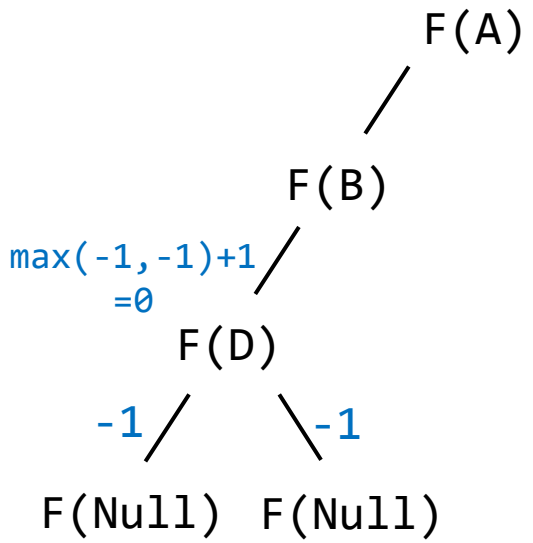


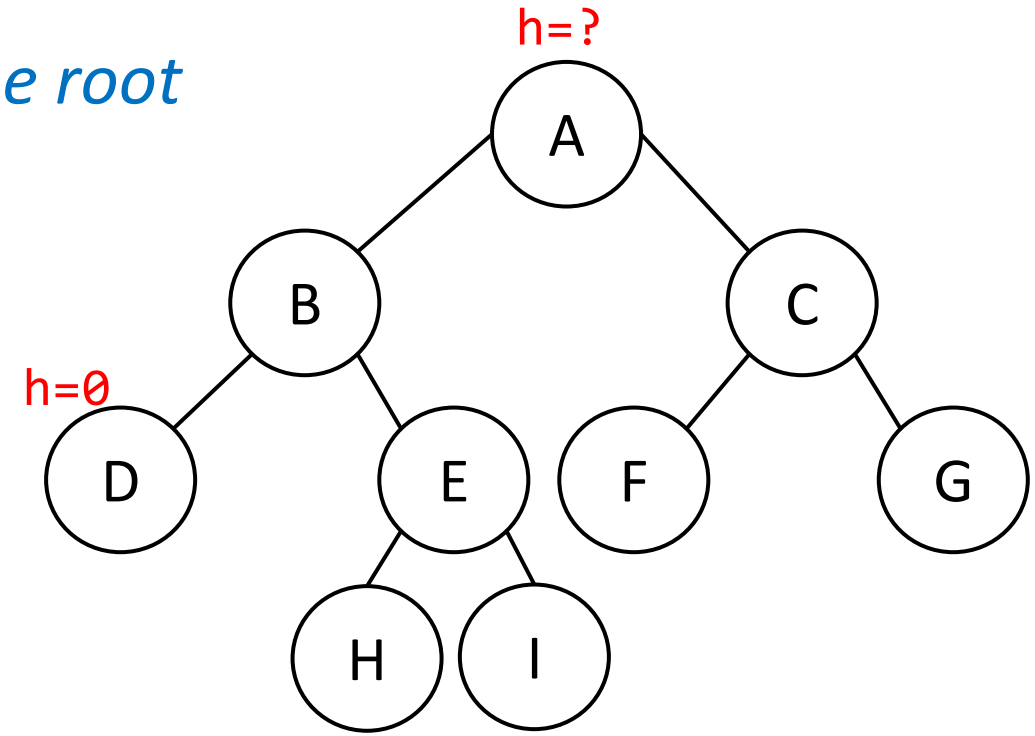
Let's find the height of this tree by passing the root into the findHeight function



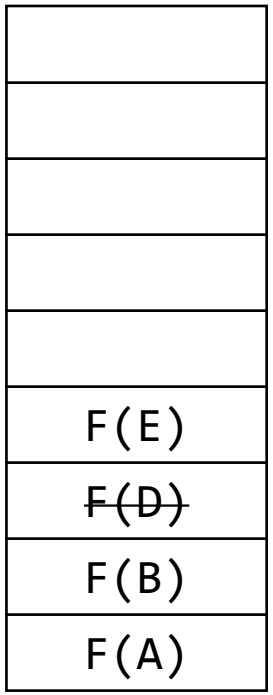


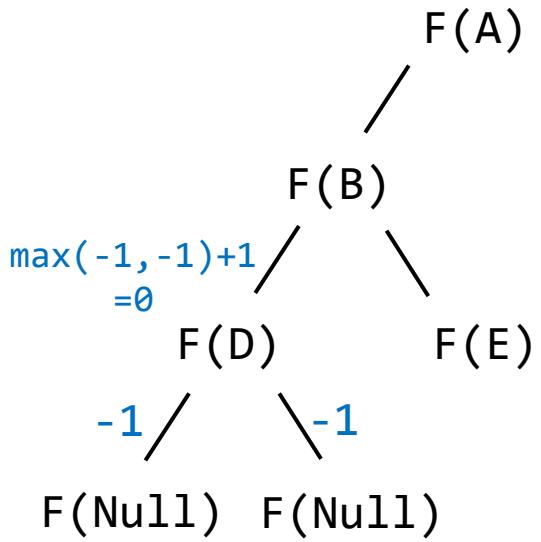
Let's find the height of this tree by passing the root into the findHeight function

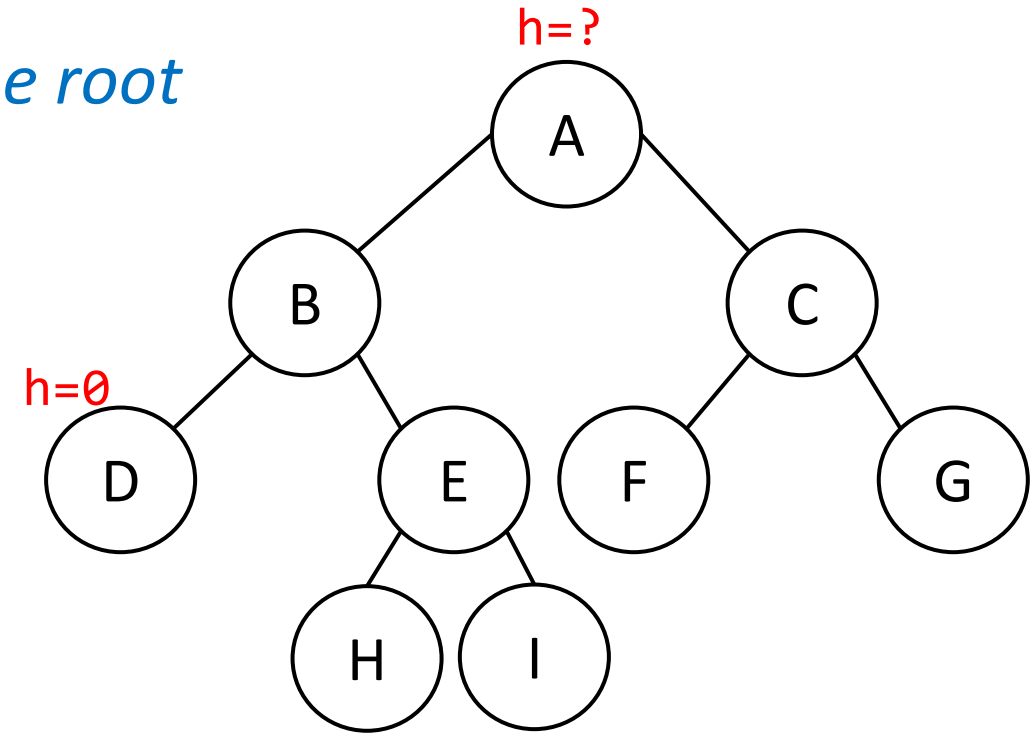




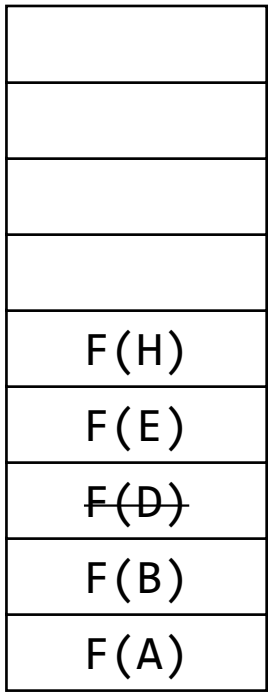
Let's find the height of this tree by passing the root into the findHeight function

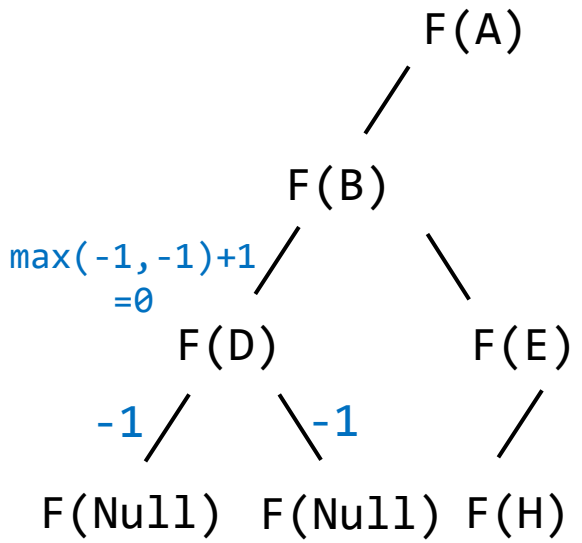


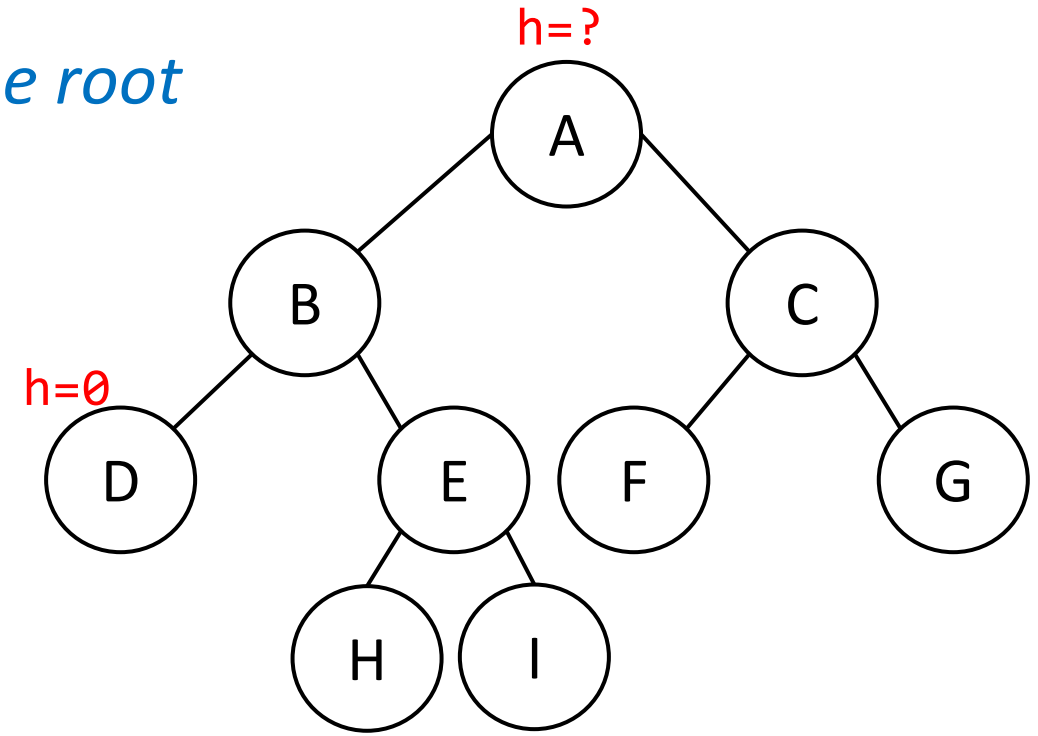




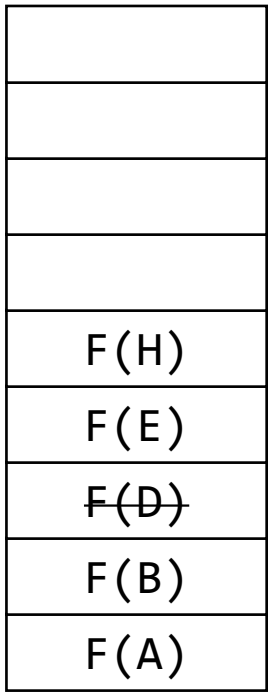
Let's find the height of this tree by passing the root into the findHeight function

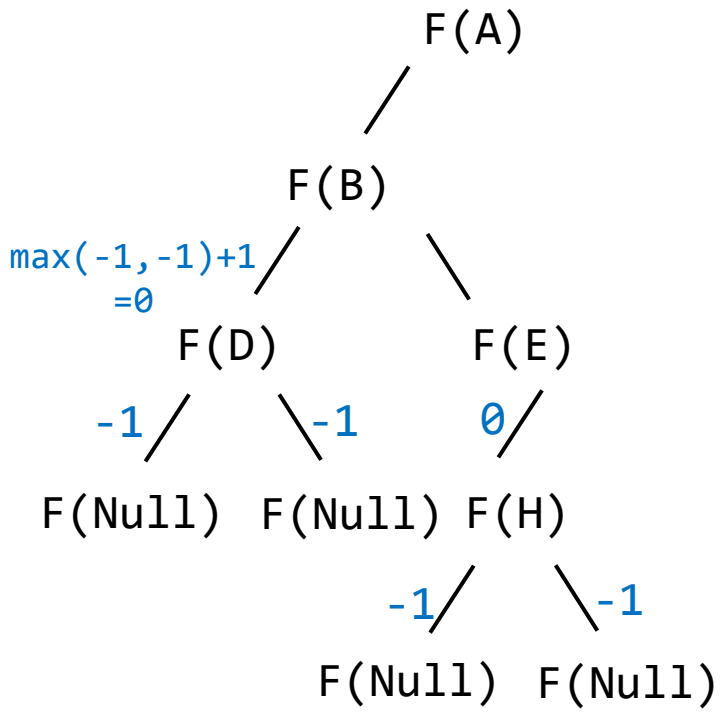


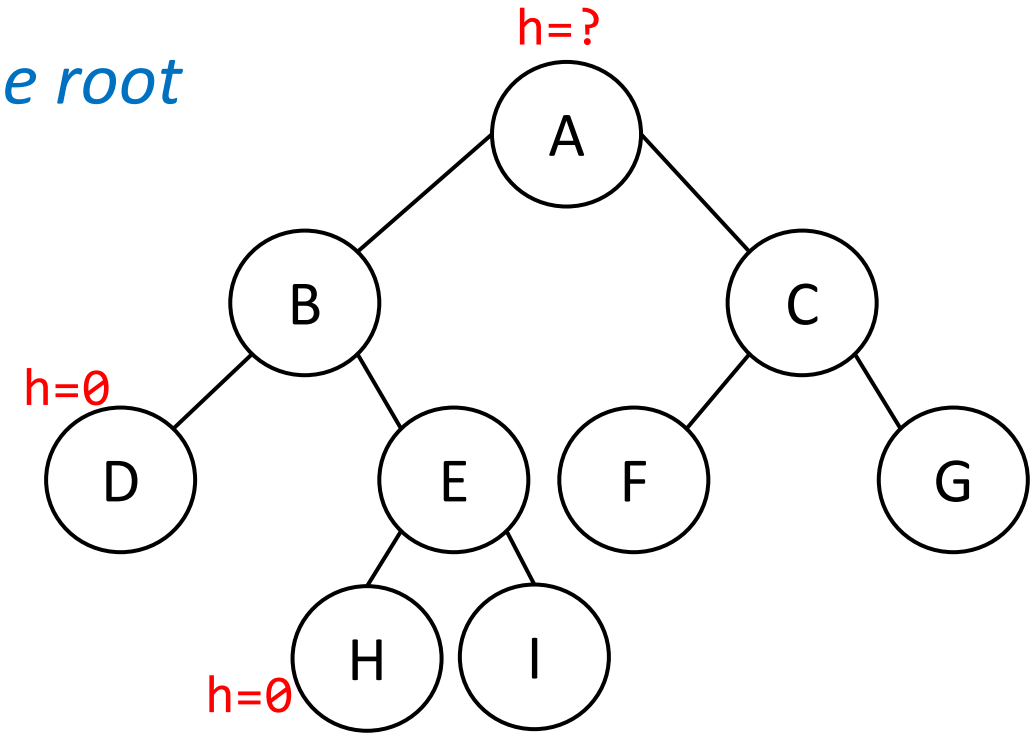




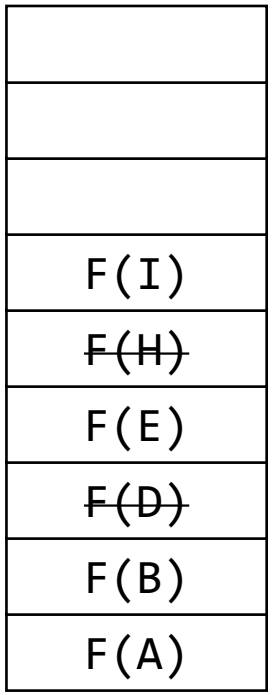
Let's find the height of this tree by passing the root into the findHeight function

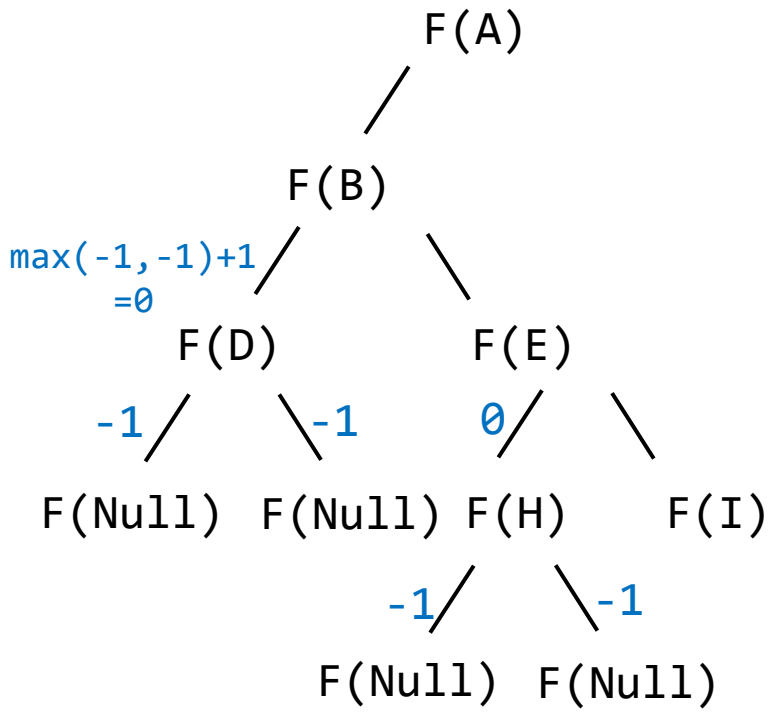


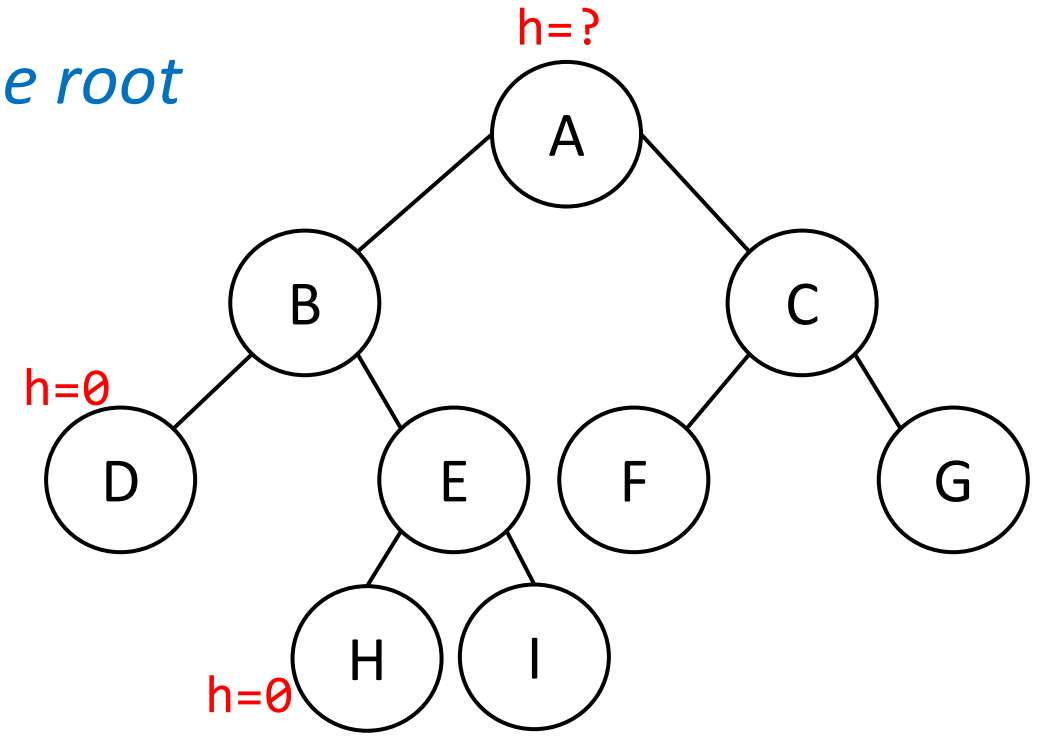




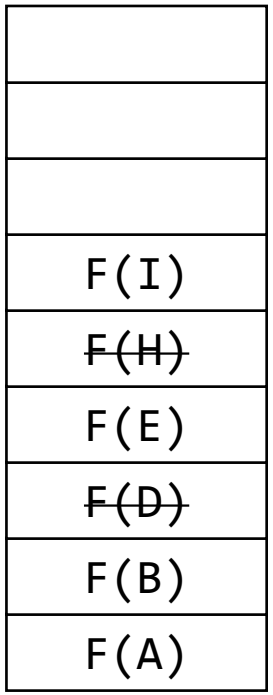
Let's find the height of this tree by passing the root into the findHeight function

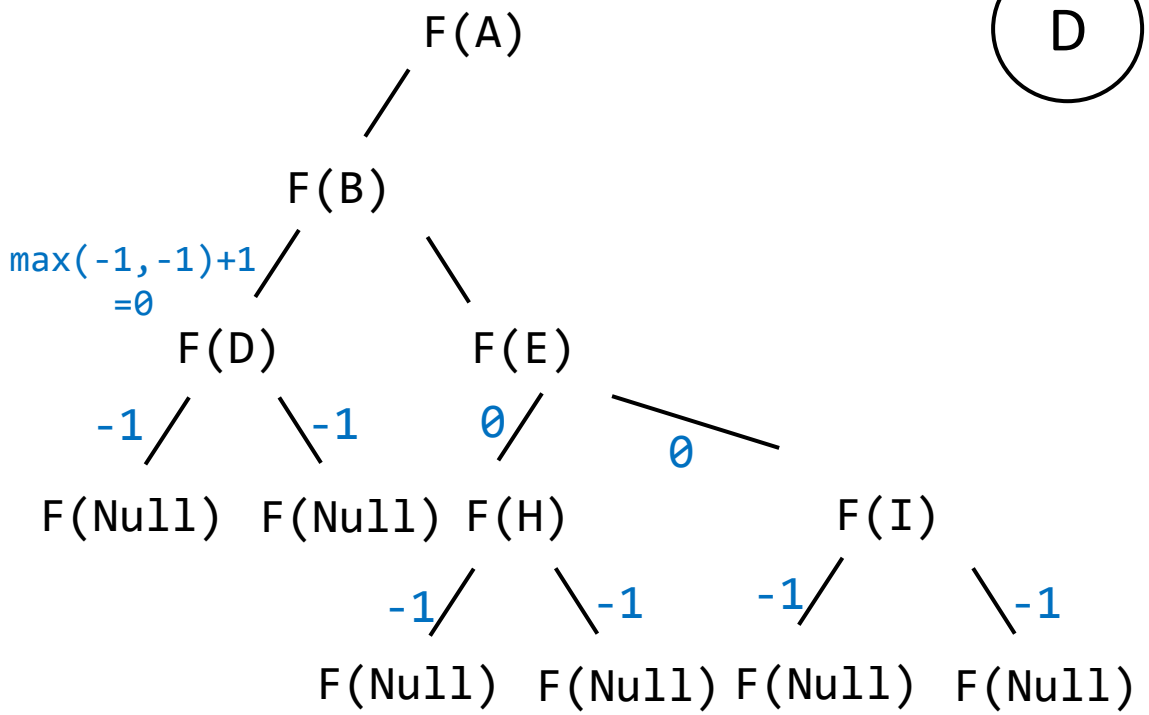


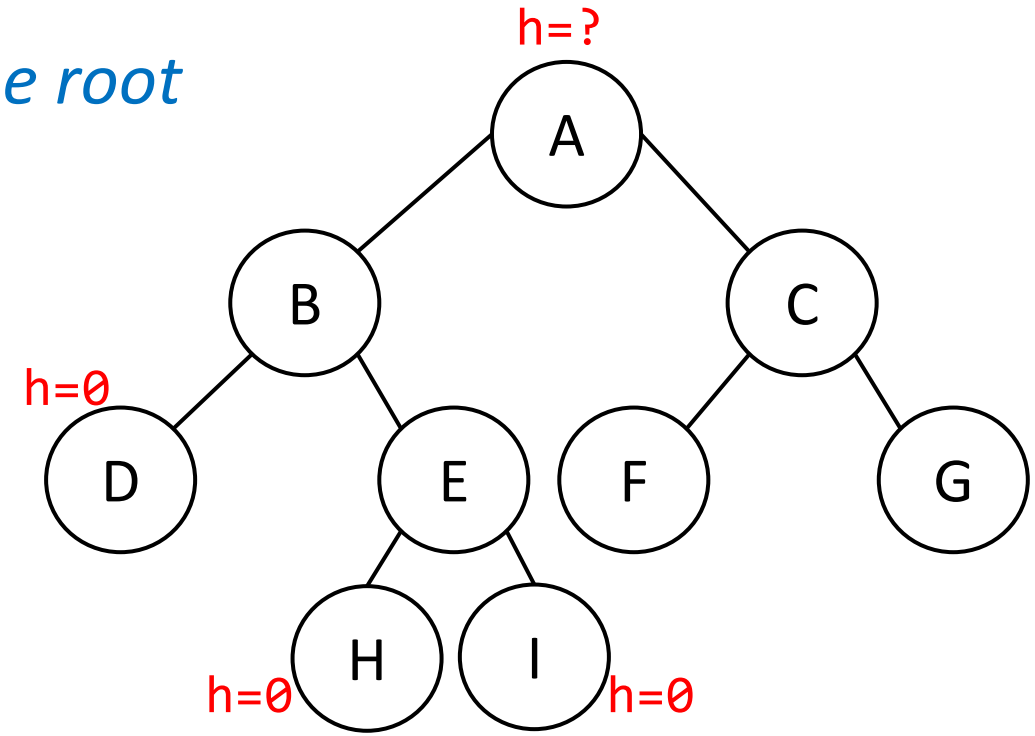




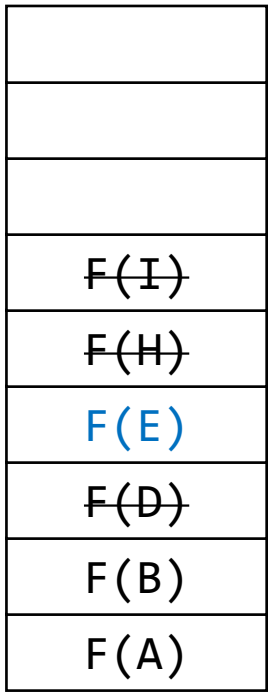
Let's find the height of this tree by passing the root into the findHeight function

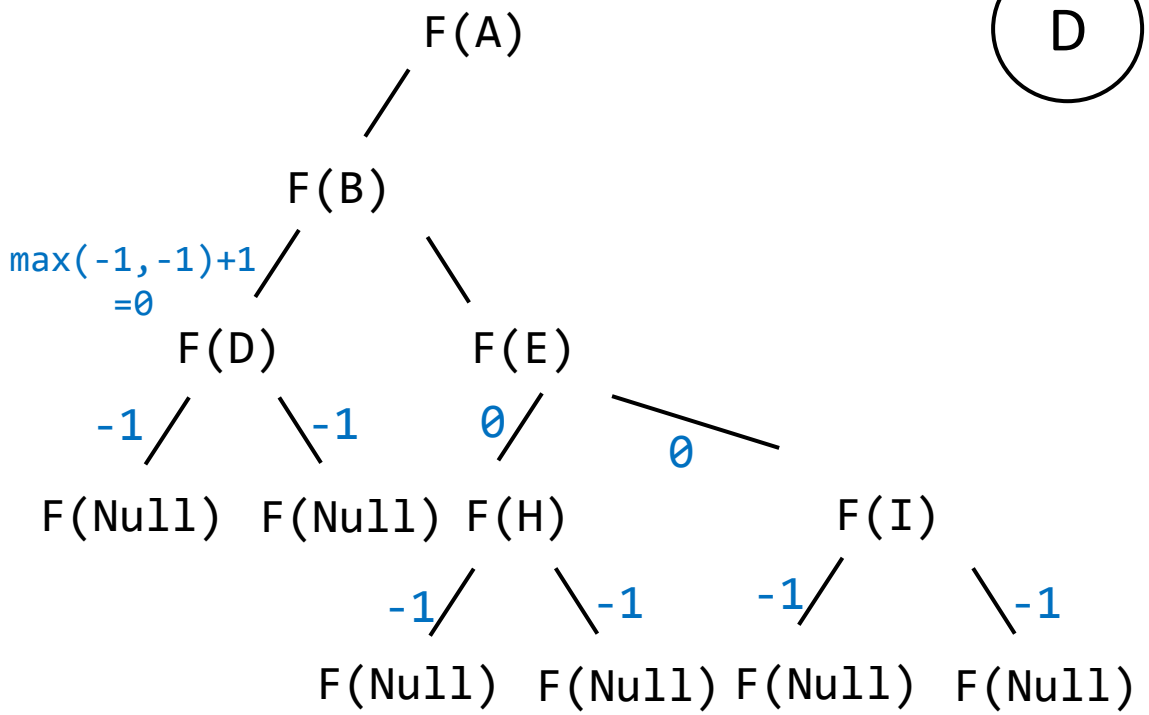


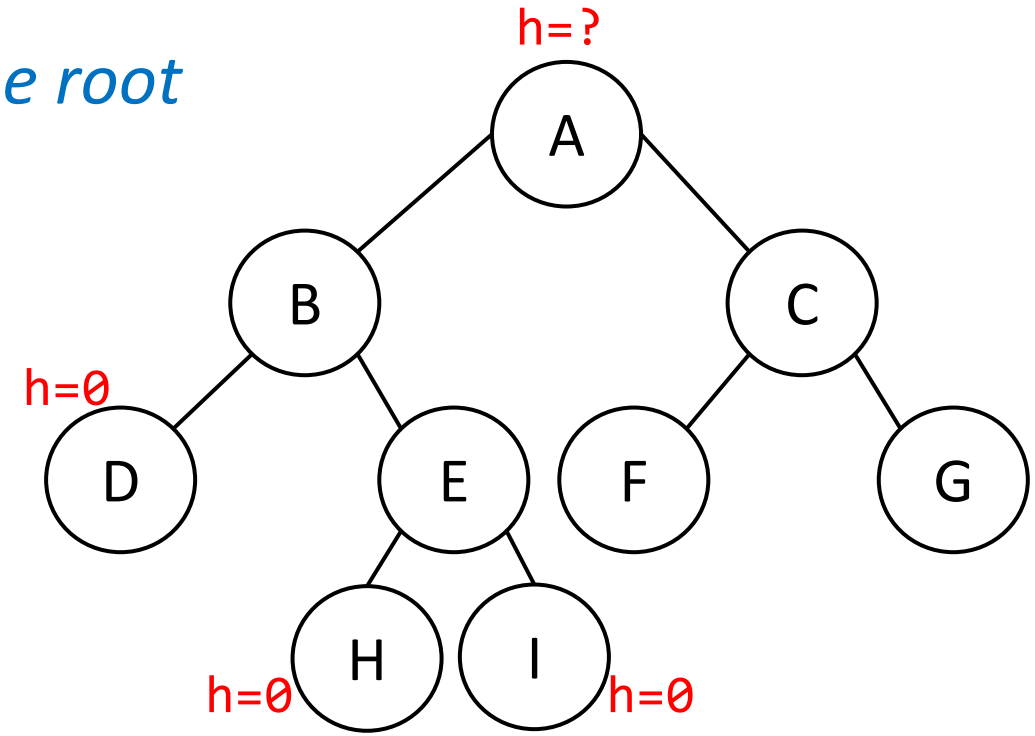




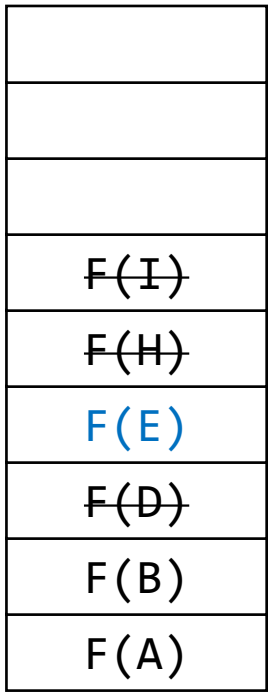
Let's find the height of this tree by passing the root into the findHeight function

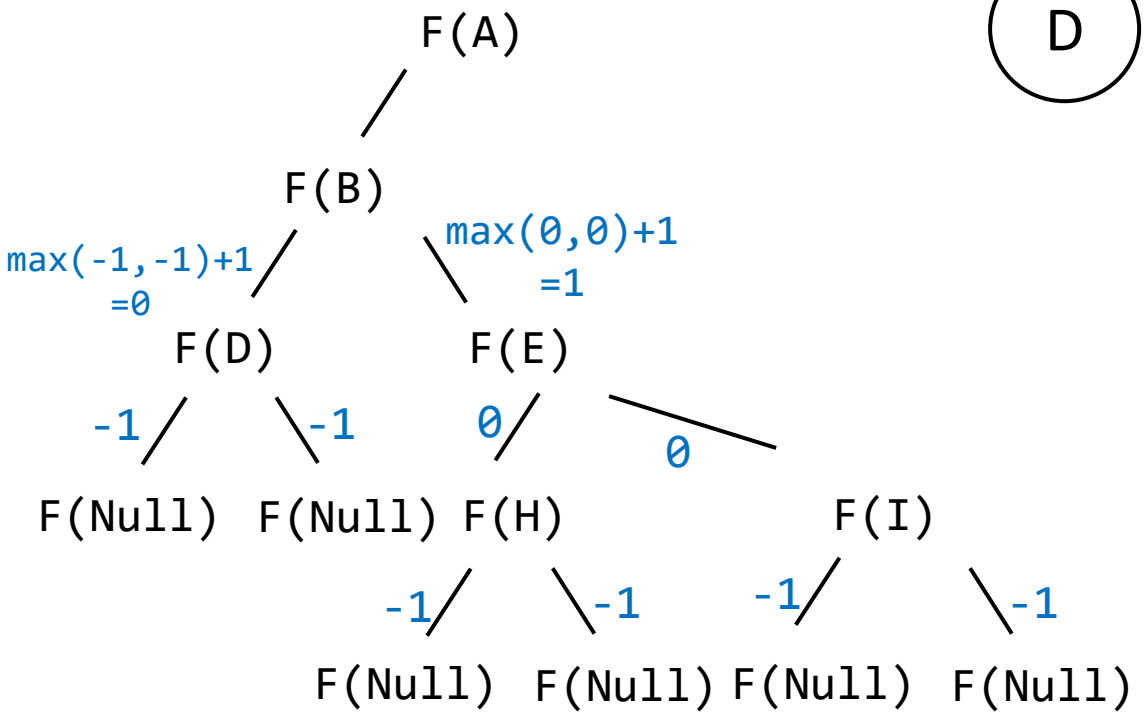


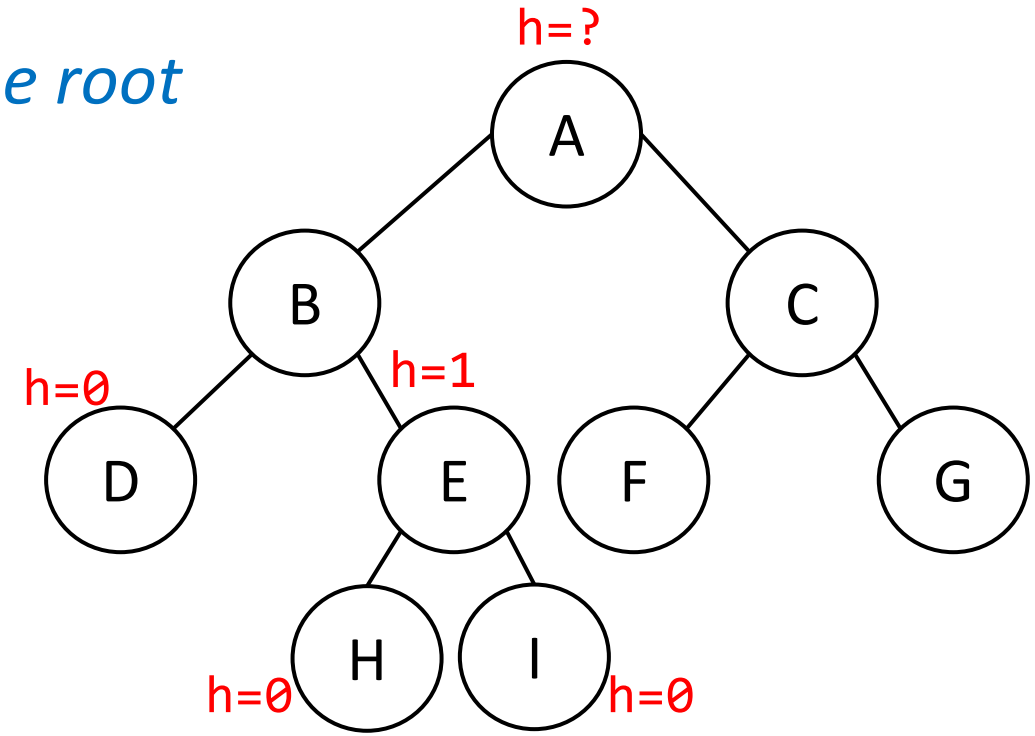




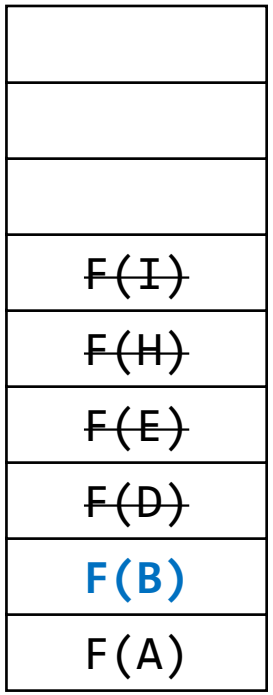
Let's find the height of this tree by passing the root into the findHeight function

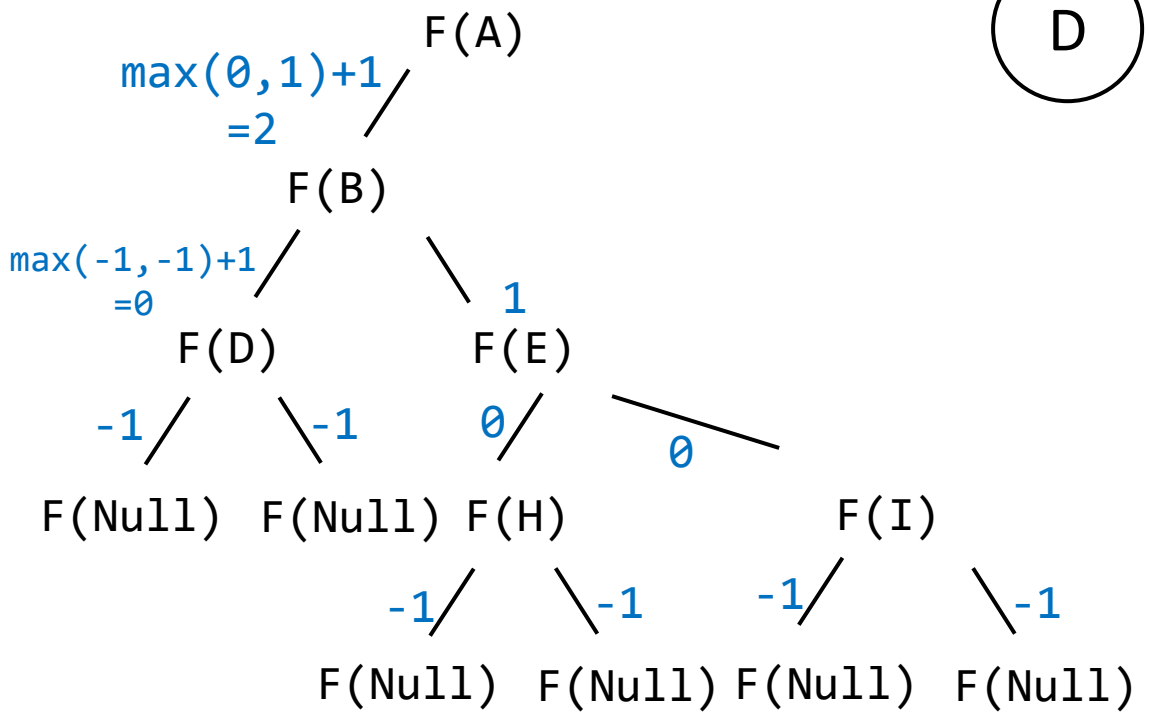


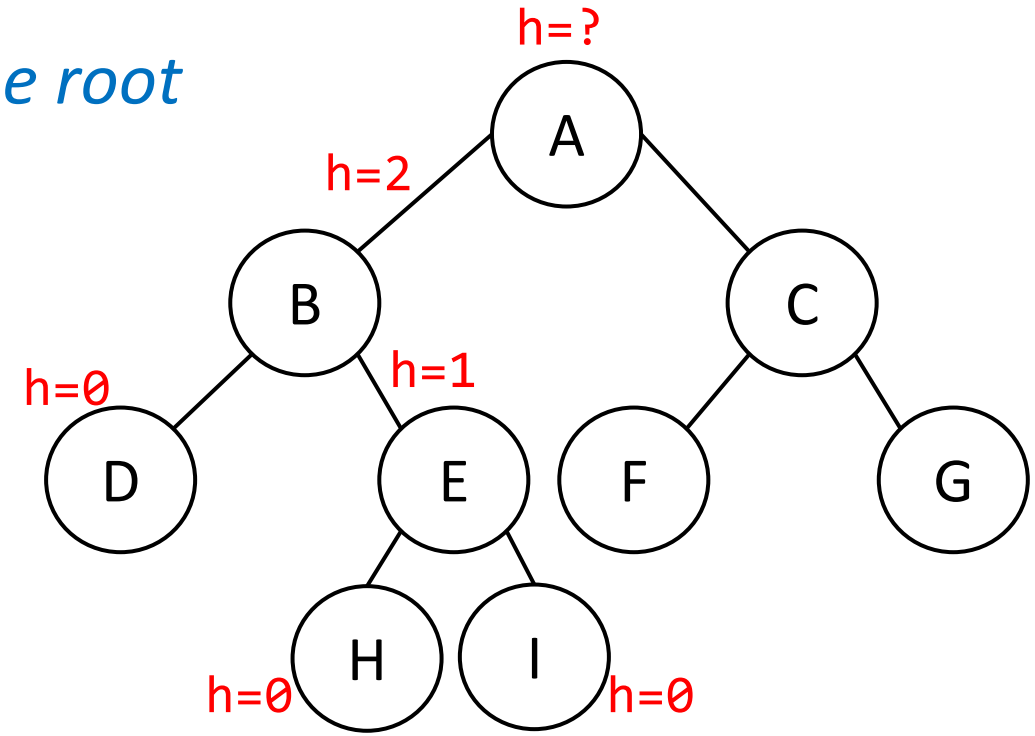




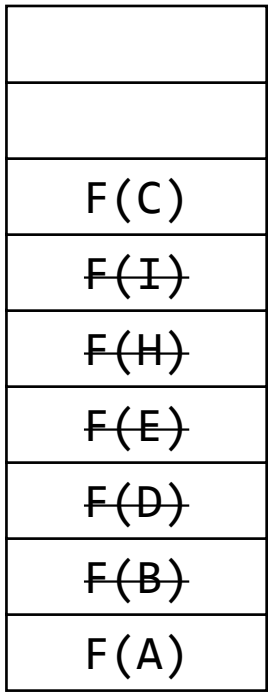
Let's find the height of this tree by passing the root into the findHeight function

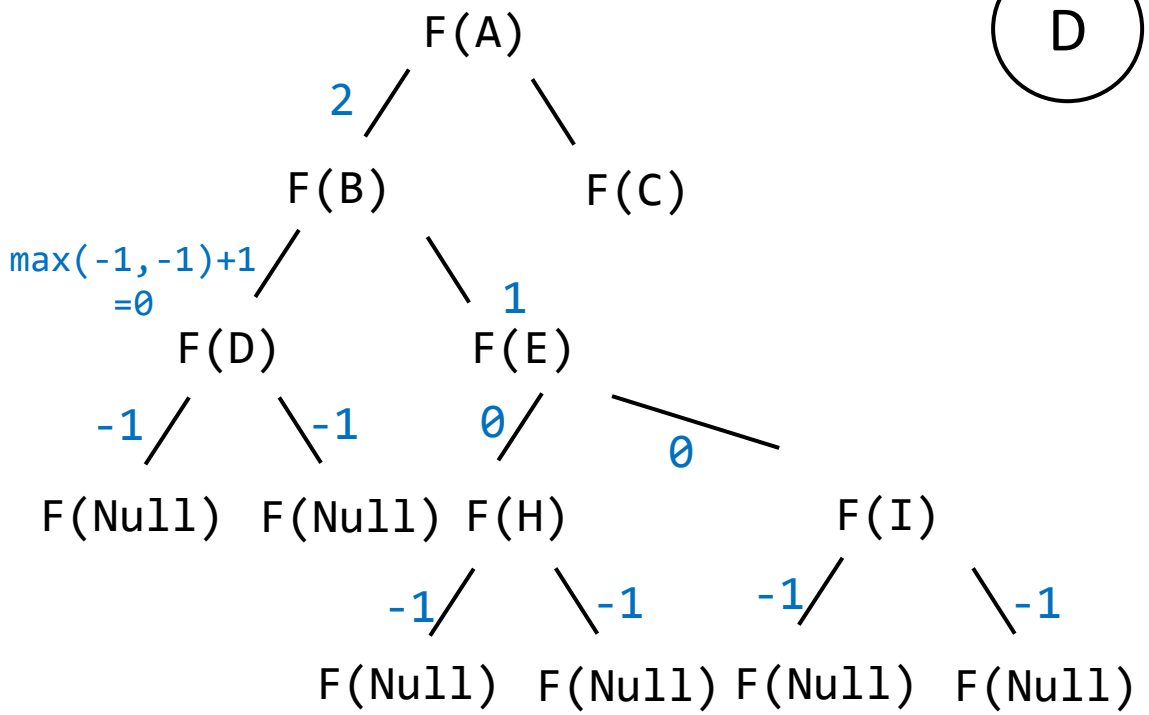


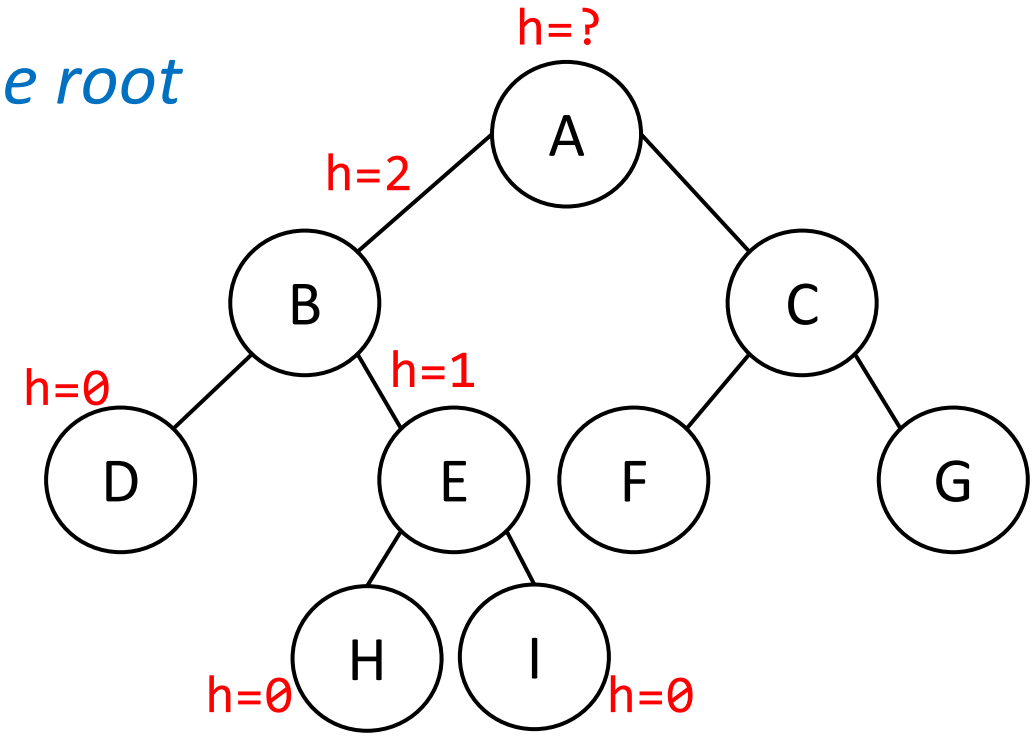




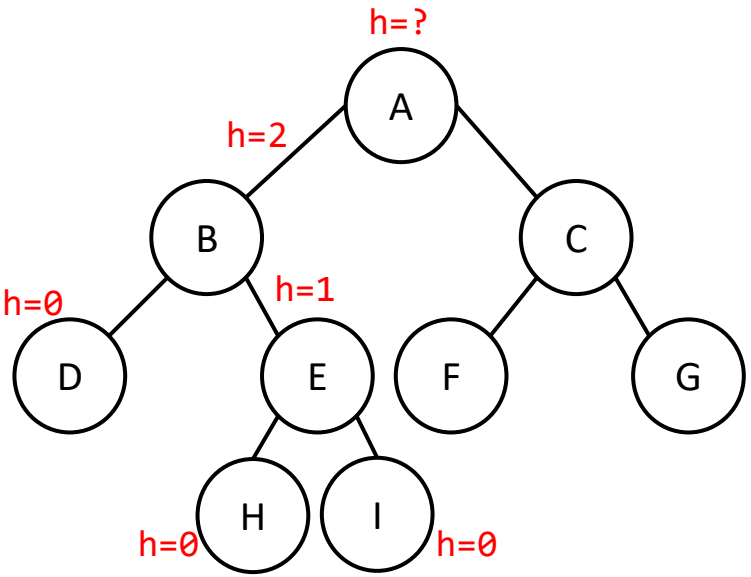
Let's find the height of this tree by passing the root into the findHeight function



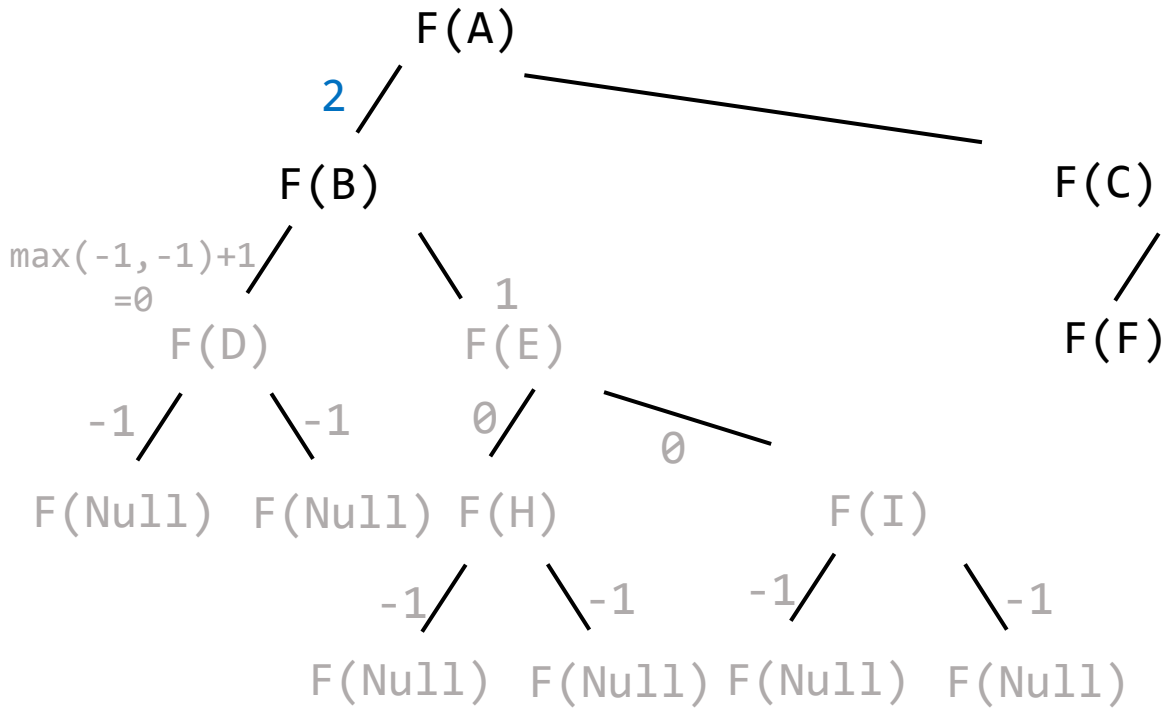




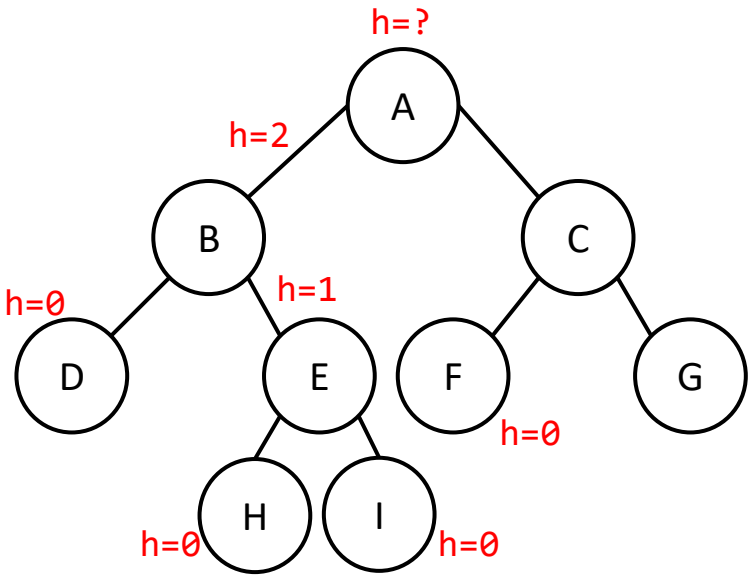
Let's find the height of this tree by passing the root into the findHeight function



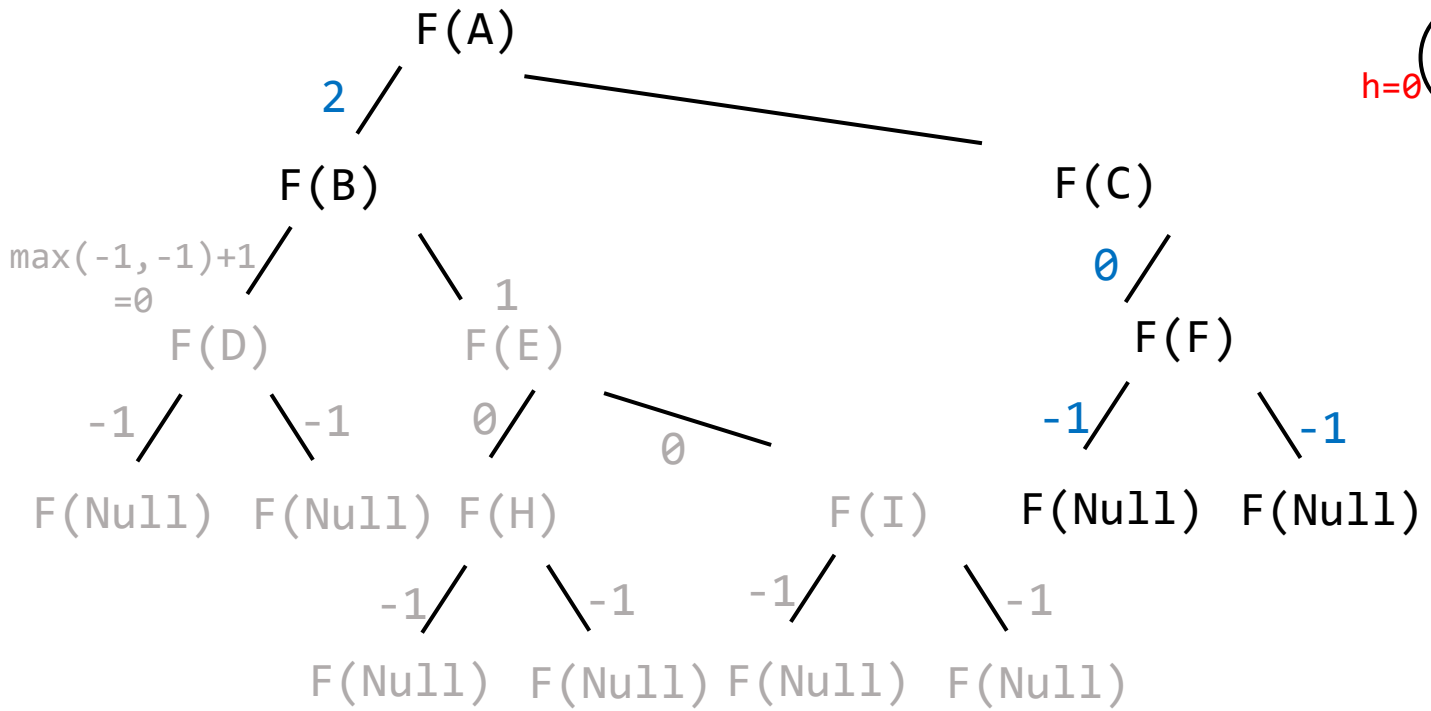
F(F)
F(C)
~~F(I)~~
~~F(H)~~
~~F(E)~~
~~F(D)~~
~~F(B)~~
F(A)



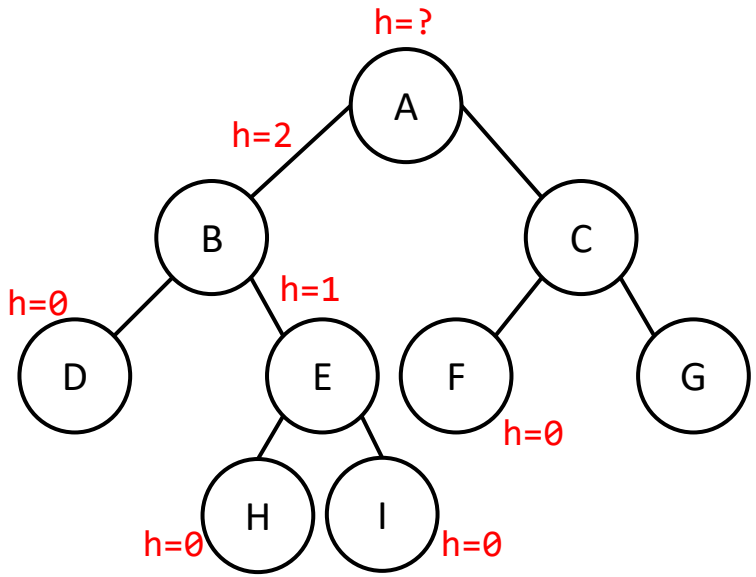
Let's find the height of this tree by passing the root into the findHeight function



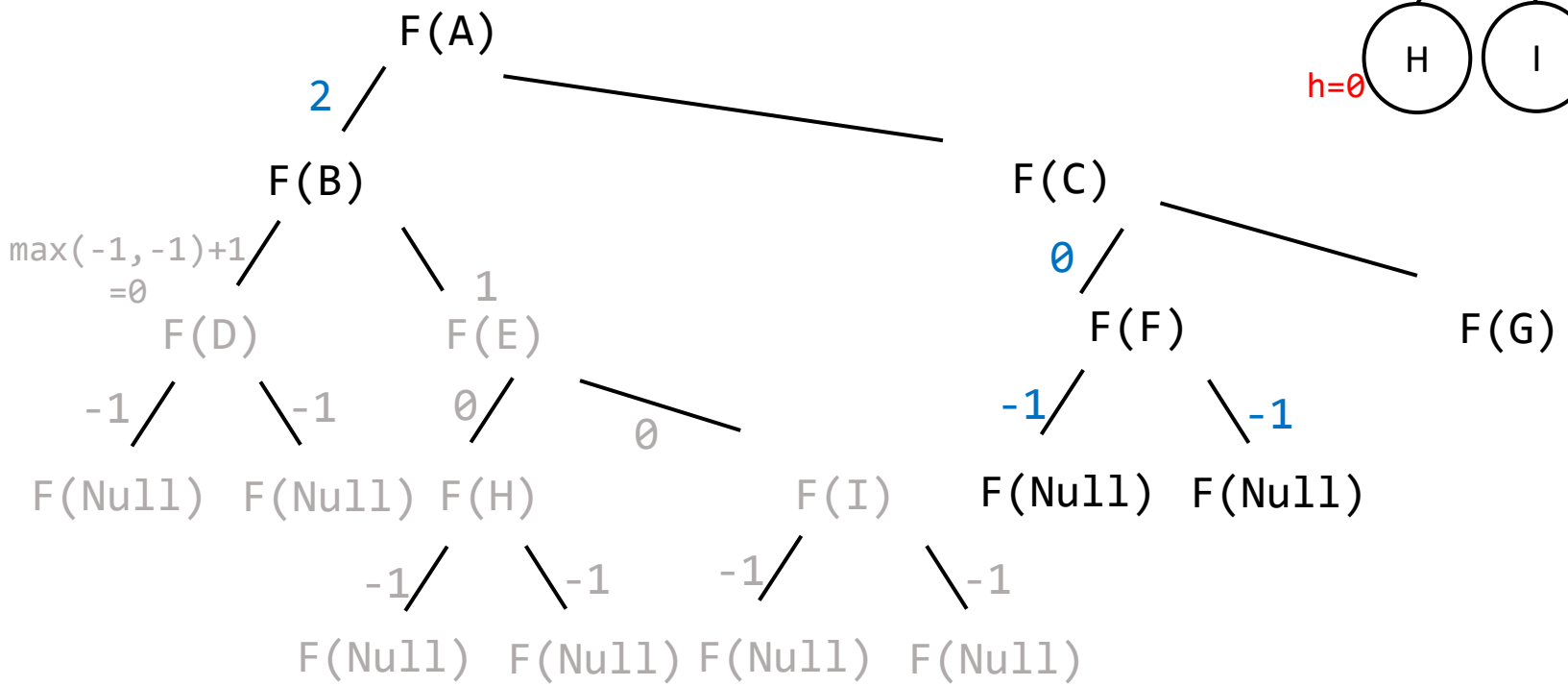
F(F)
F(C)
~~F(I)~~
~~F(H)~~
~~F(E)~~
~~F(D)~~
~~F(B)~~
F(A)



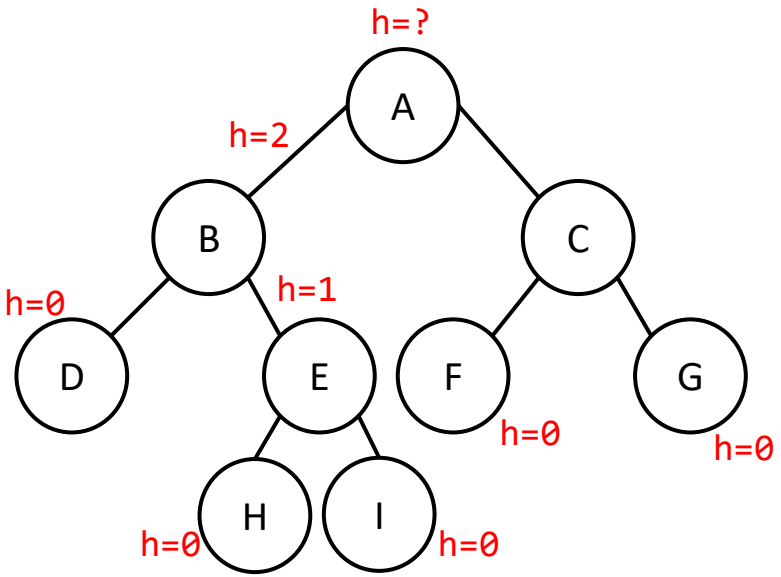
Let's find the height of this tree by passing the root into the findHeight function



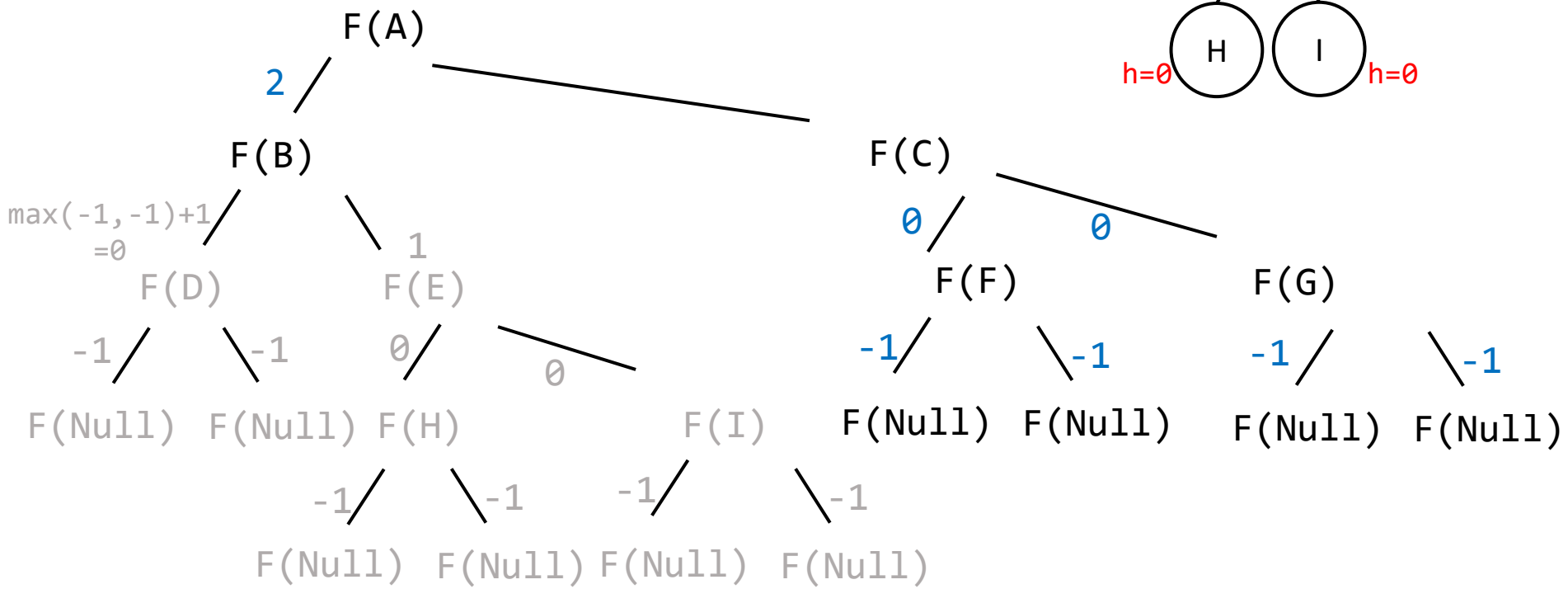
F(G)
~~F(F)~~
F(C)
~~F(I)~~
~~F(H)~~
~~F(E)~~
~~F(D)~~
~~F(B)~~
F(A)



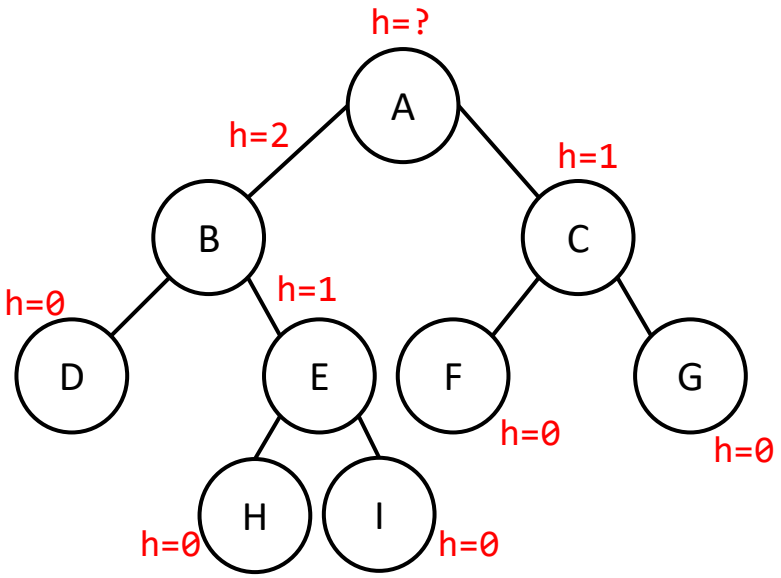
Let's find the height of this tree by passing the root into the findHeight function



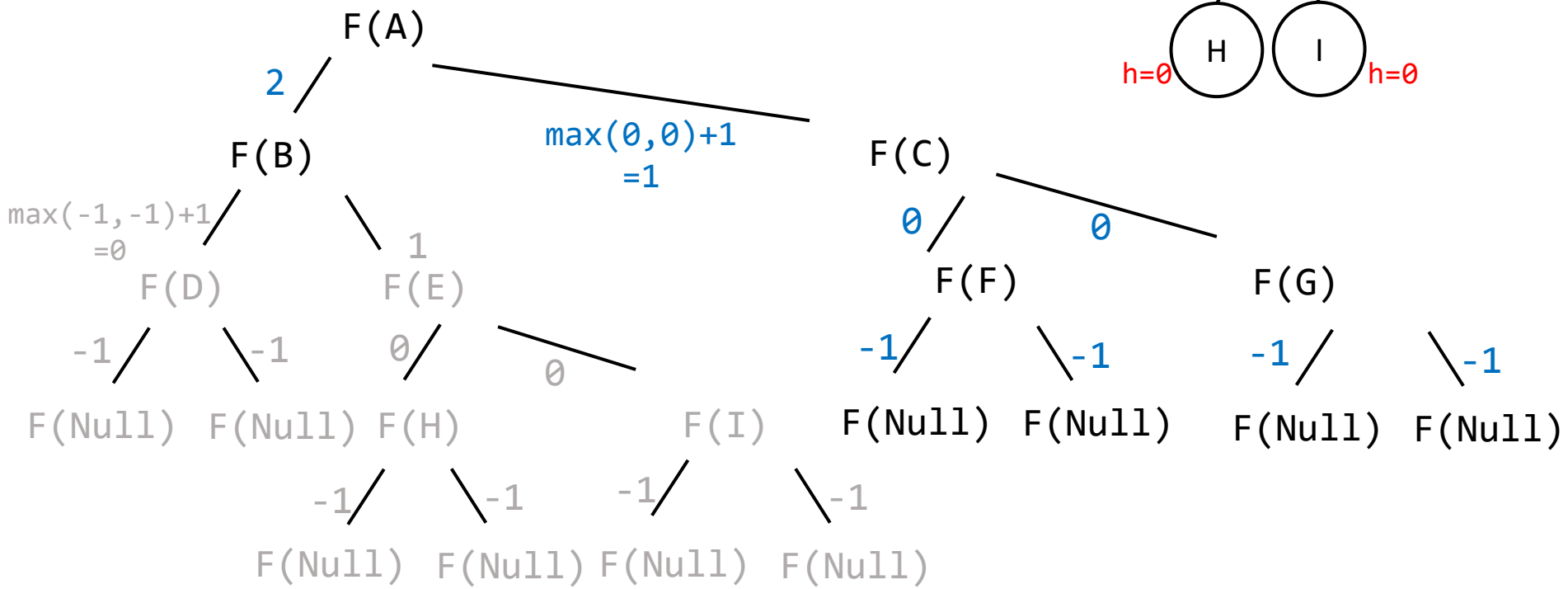
F(G)
~~F(F)~~
F(C)
~~F(I)~~
~~F(H)~~
~~F(E)~~
~~F(D)~~
~~F(B)~~
F(A)



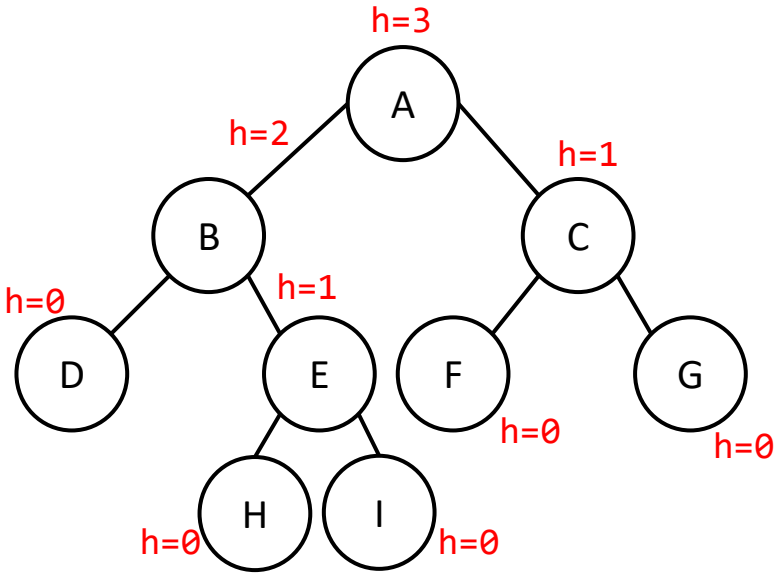
Let's find the height of this tree by passing the root into the findHeight function



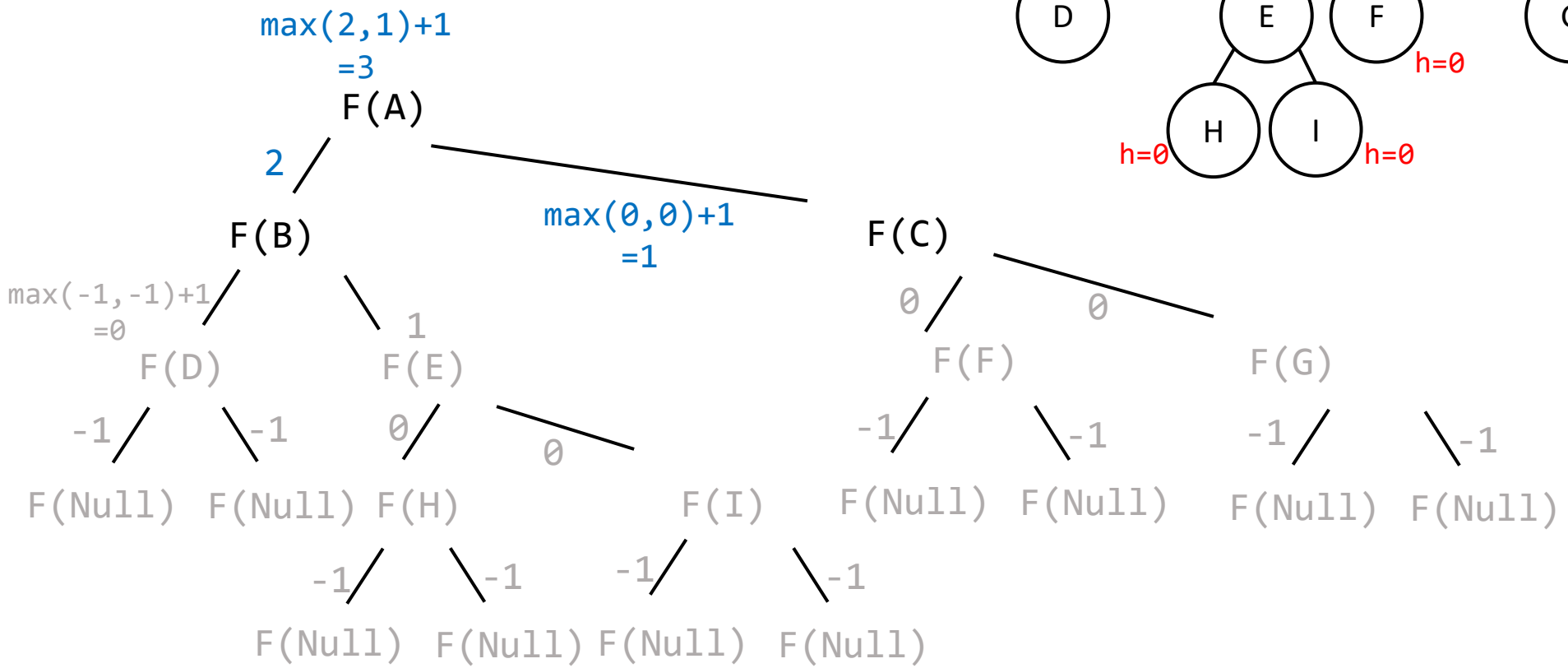
~~F(G)~~
~~F(F)~~
F(C)
~~F(I)~~
~~F(H)~~
~~F(E)~~
~~F(D)~~
~~F(B)~~
F(A)



Let's find the height of this tree by passing the root into the findHeight function



~~F(G)~~
~~F(F)~~
~~F(C)~~
~~F(I)~~
~~F(H)~~
~~F(E)~~
~~F(D)~~
~~F(B)~~
F(A)



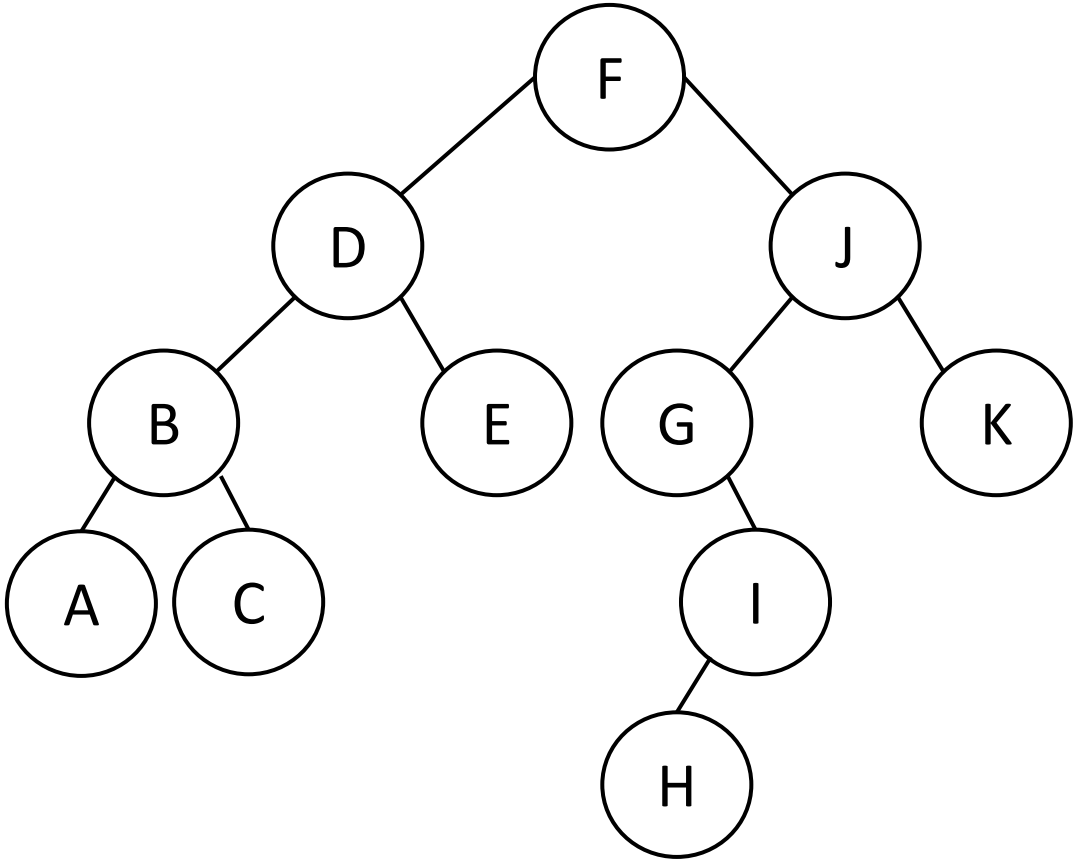
Tree traversal

Tree traversal

- Breadth first/ level order
- Depth first
  - Preorder `<data> <left> <right>`
  - Inorder `<left> <data> <right>`
  - Post order `<left> <right> <data>`

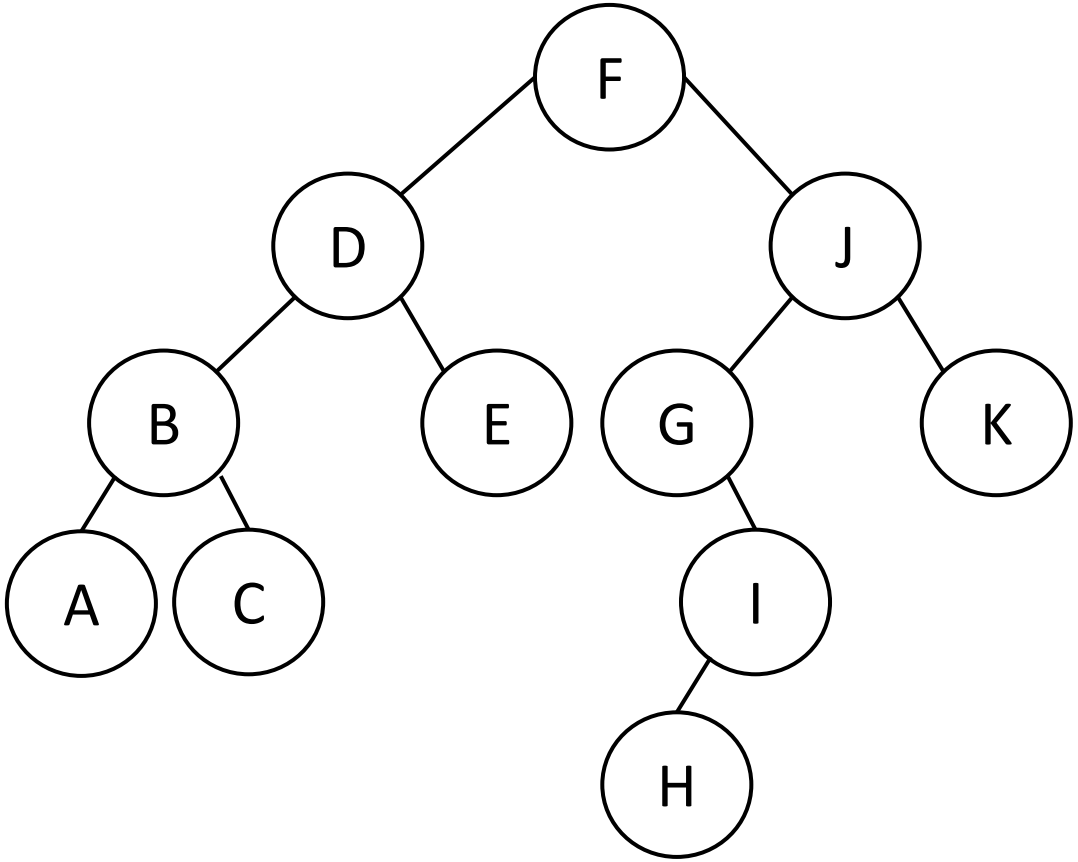
Level order traversal

Order is :



Level order traversal

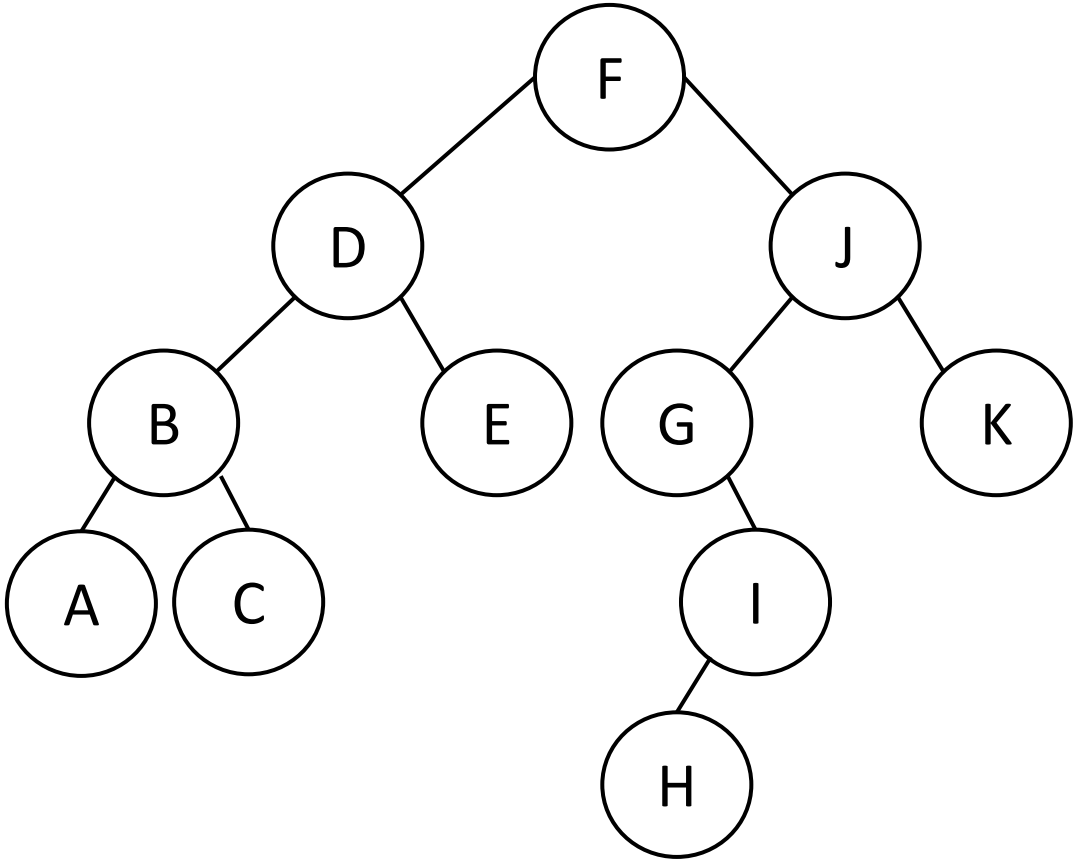
F							



Order is :

Level order traversal

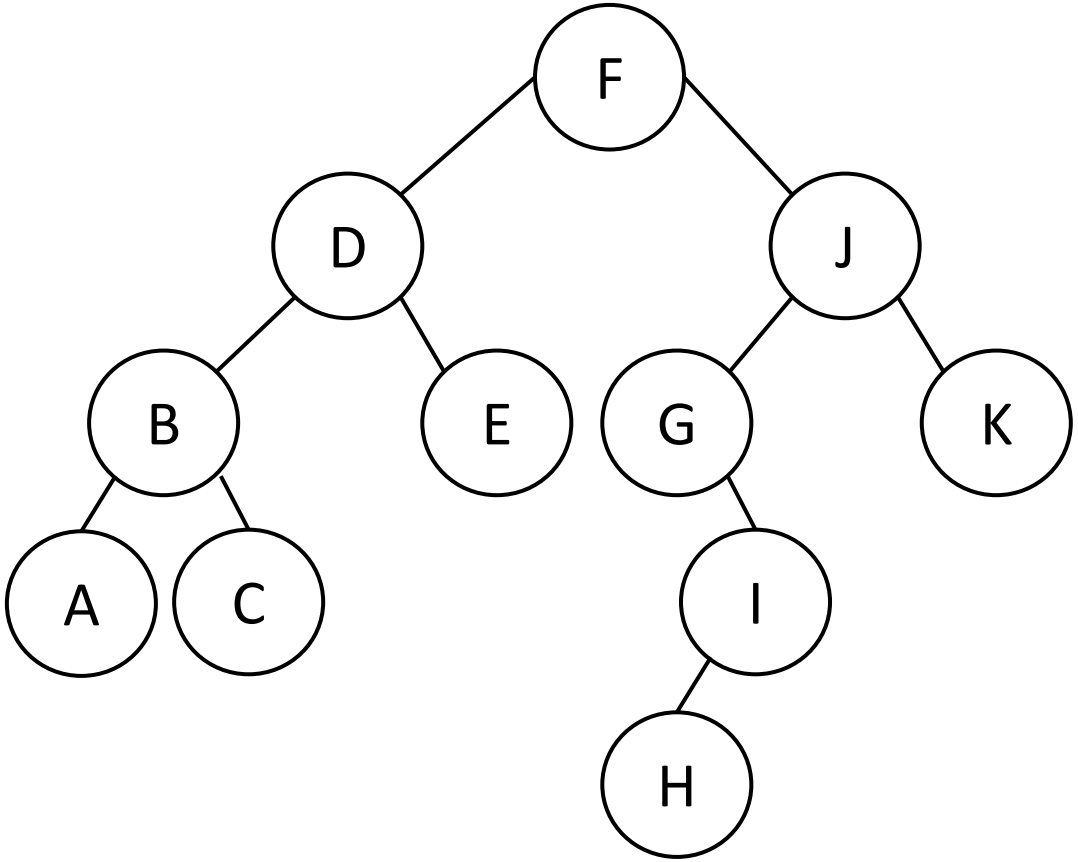
F	D	J					



Order is : F

Level order traversal

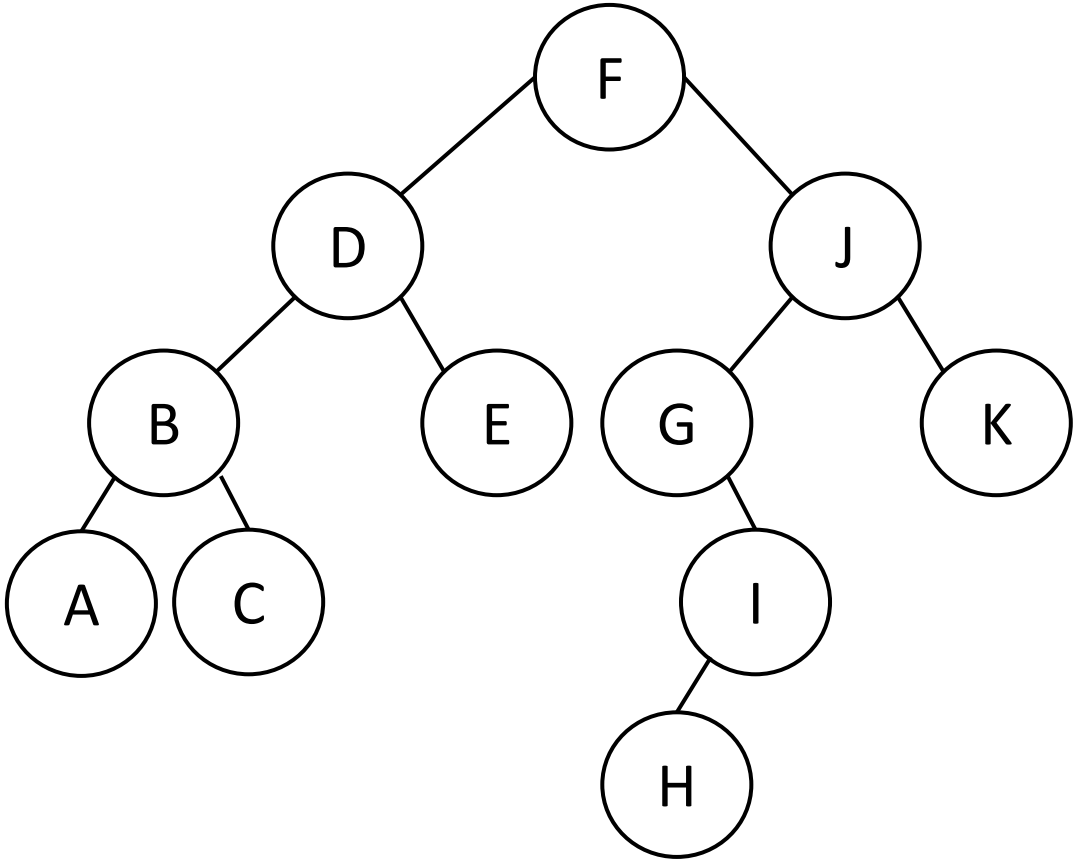
~~F~~	~~D~~	J	B	E			



Order is : F D

Level order traversal

~~F~~	~~D~~	~~J~~	B	E	G	K	



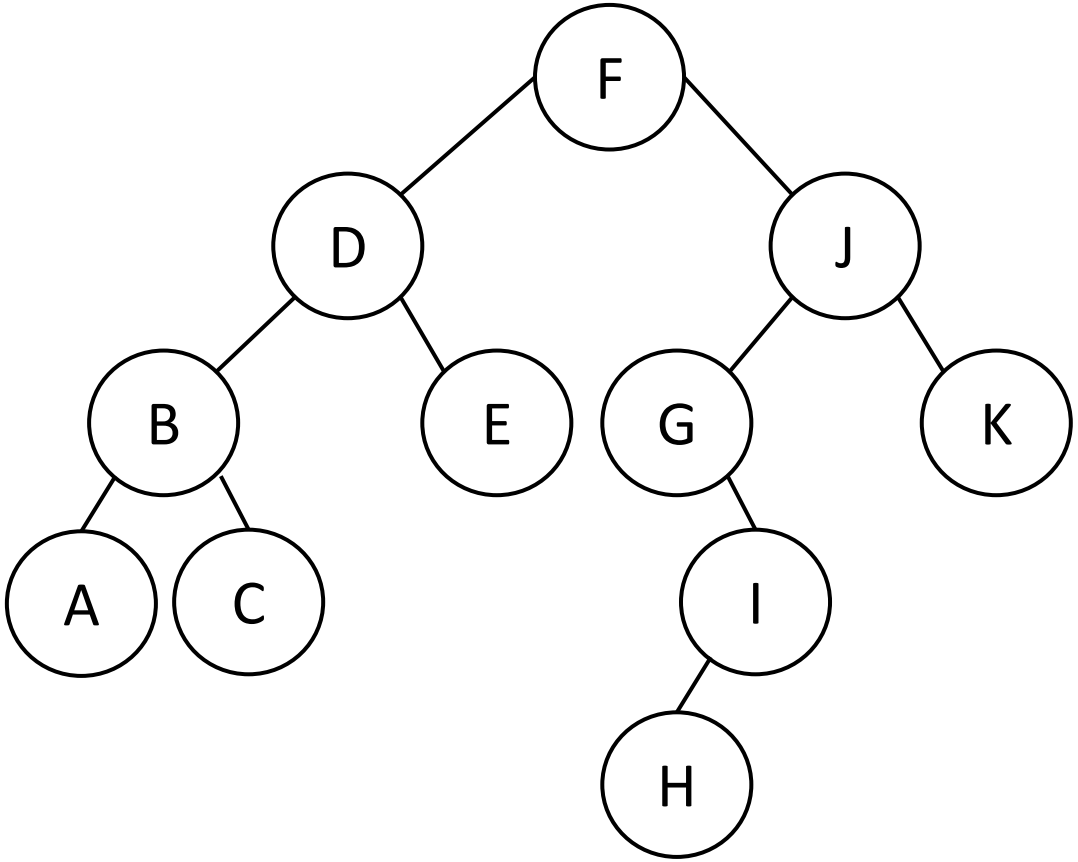
Order is : F D J

Level order traversal

~~F~~	~~D~~	~~J~~	~~B~~	E	G	K	A

C							

Order is : F D J B

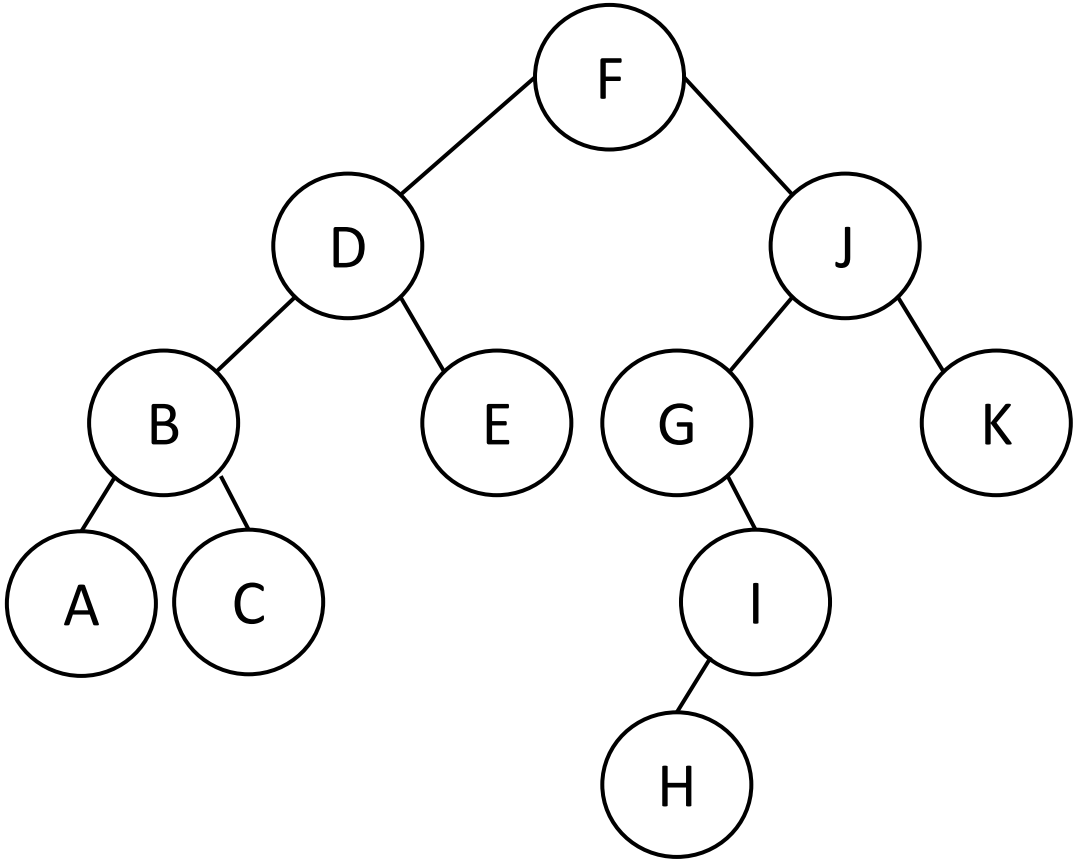


Level order traversal

~~F~~	~~D~~	~~J~~	~~B~~	~~E~~	G	K	A

C							

Order is : F D J B E

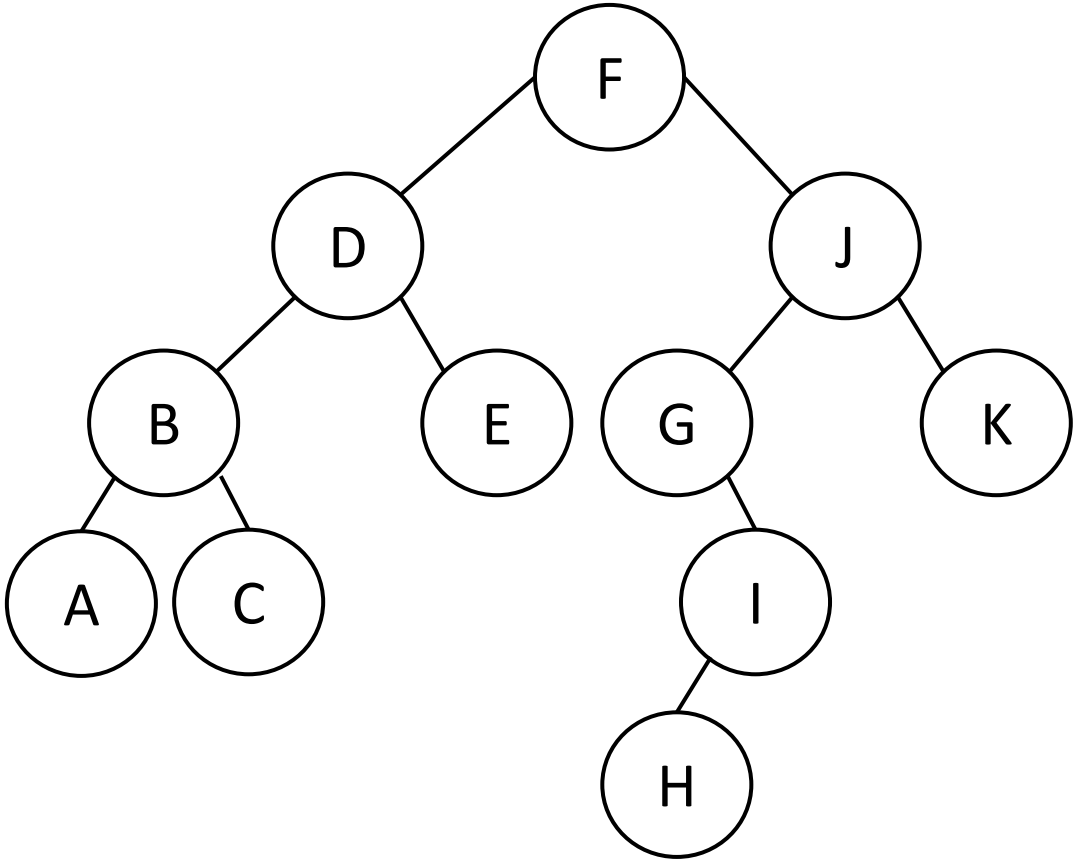


Level order traversal

~~F~~	~~D~~	~~J~~	~~B~~	~~E~~	~~G~~	K	A

C	I						

Order is : F D J B E G

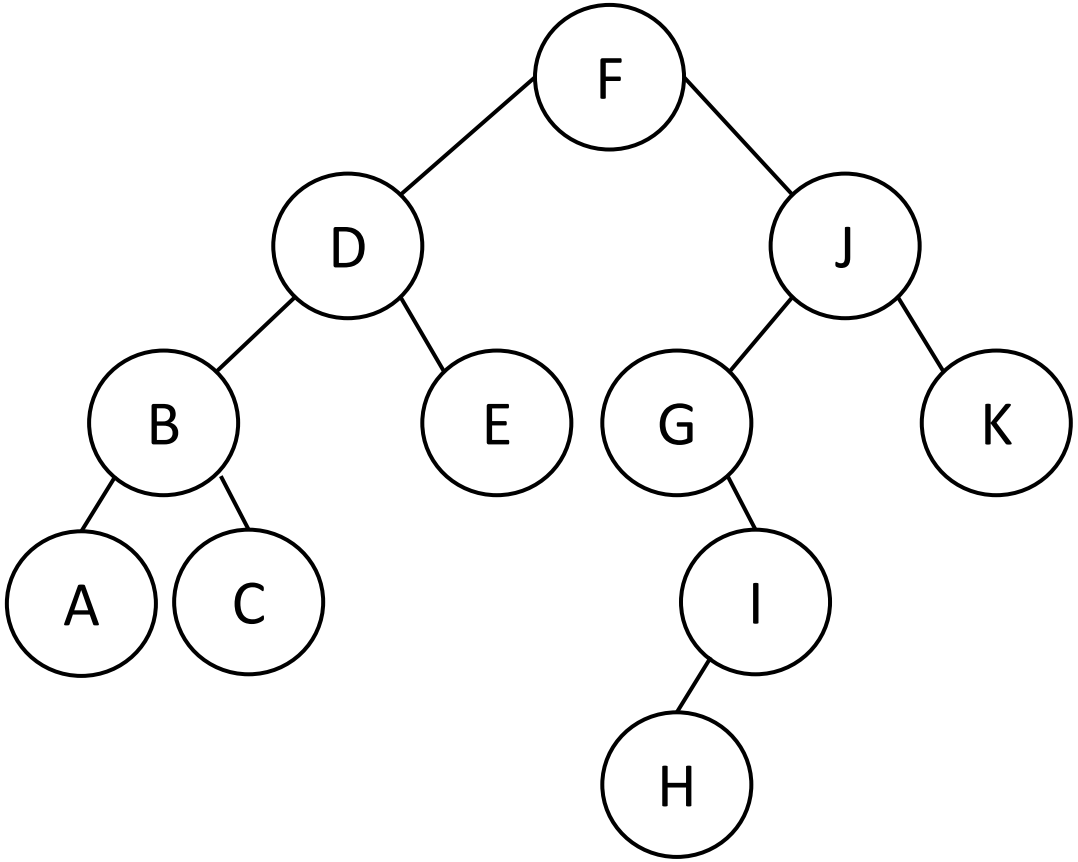


Level order traversal

~~F~~	~~D~~	~~J~~	~~B~~	~~E~~	~~G~~	~~K~~	A

C	I						

Order is : F D J B E G K

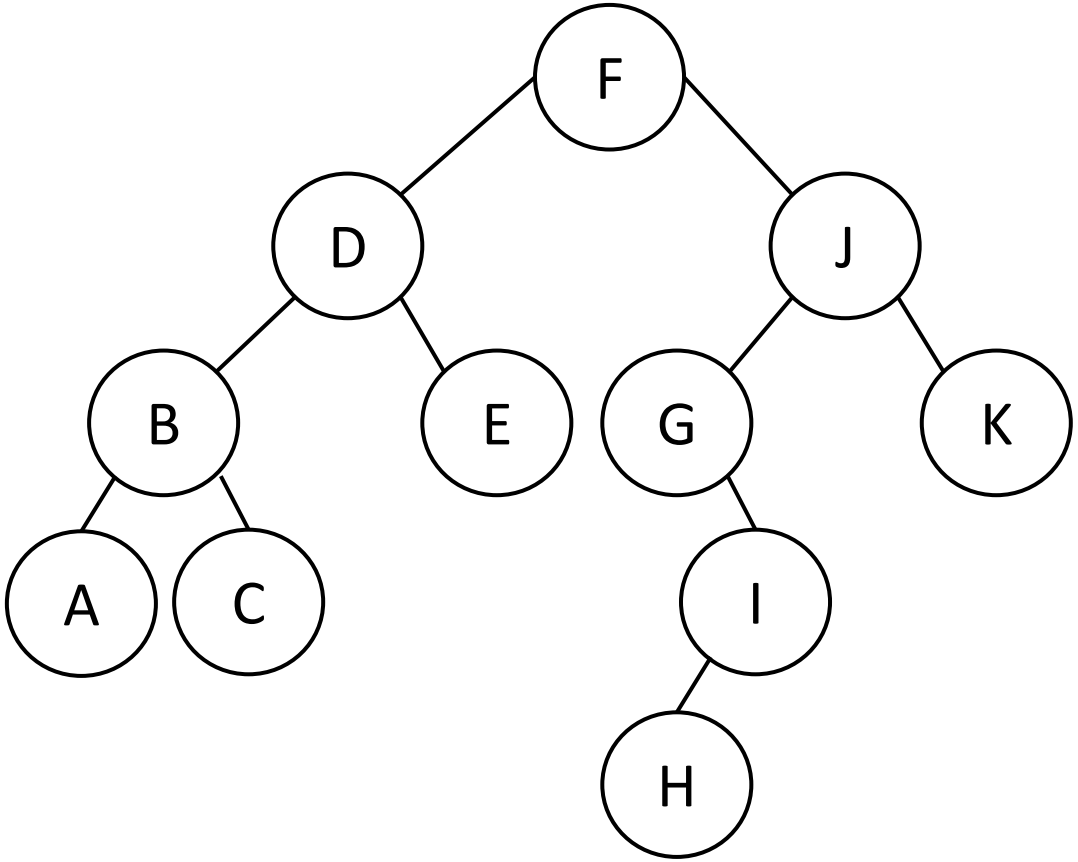


Level order traversal

~~F~~	~~D~~	~~J~~	~~B~~	~~E~~	~~G~~	~~K~~	~~A~~

C	I						

Order is : F D J B E G K A

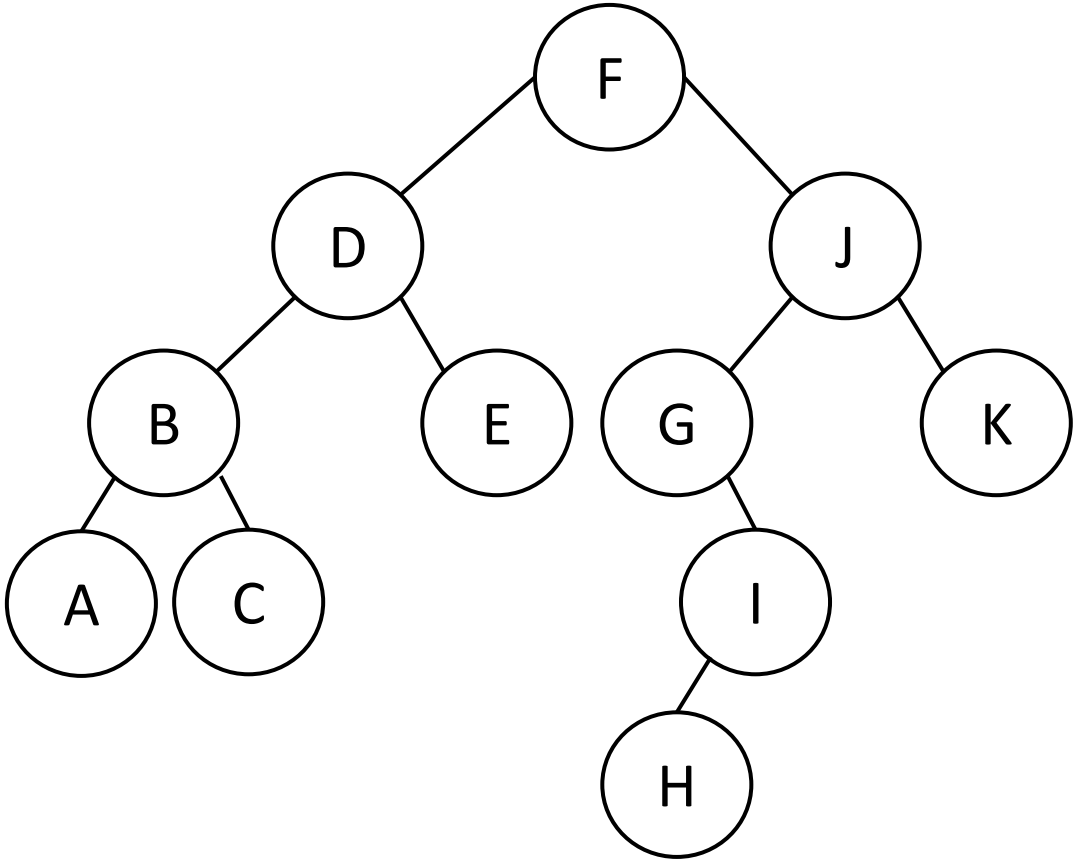


Level order traversal

~~F~~	~~D~~	~~J~~	~~B~~	~~E~~	~~G~~	~~K~~	~~A~~

~~C~~	I						

Order is : F D J B E G K A C

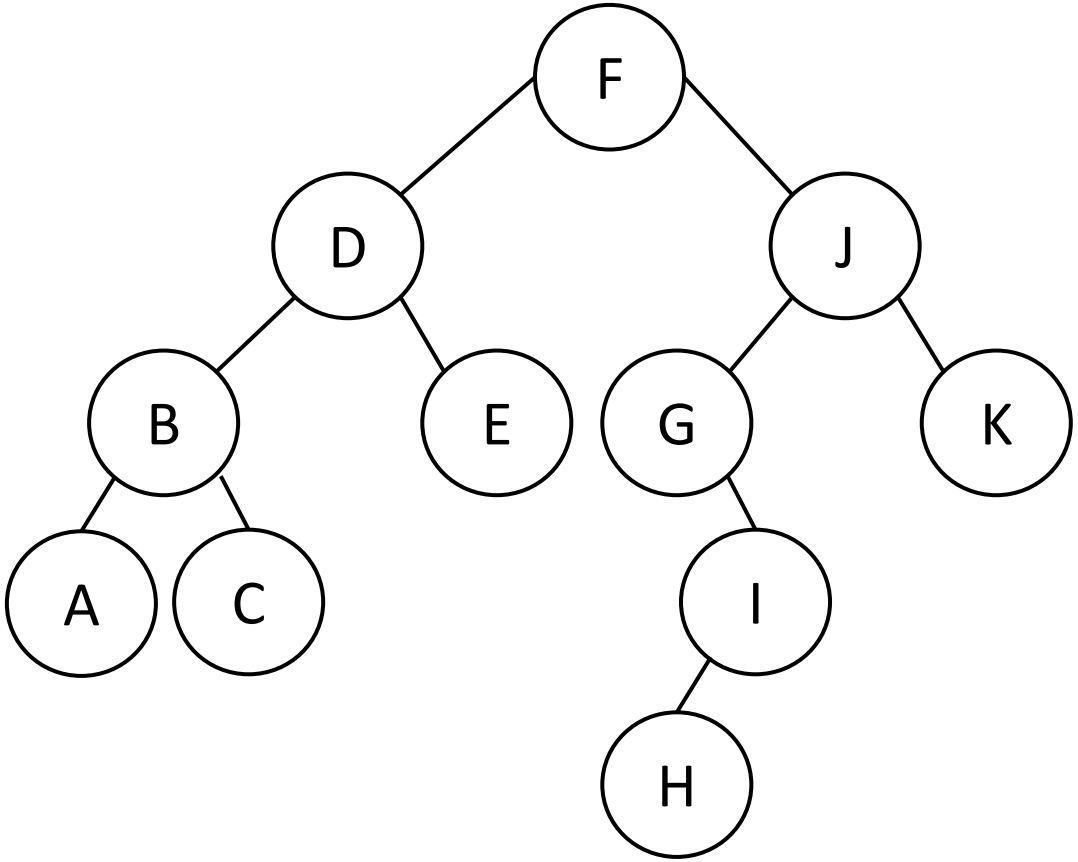


Level order traversal

~~F~~	~~D~~	~~J~~	~~B~~	~~E~~	~~G~~	~~K~~	~~A~~

~~C~~	~~I~~	H					

Order is : F D J B E G K A C I



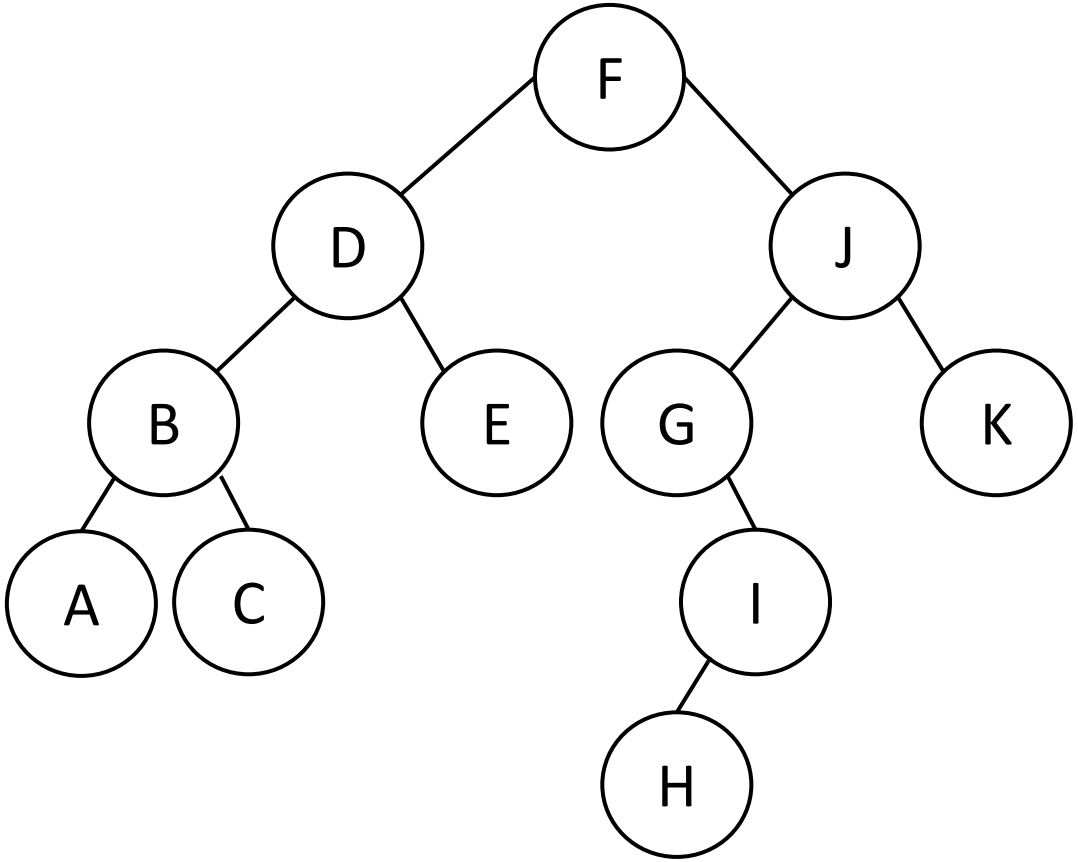
Level order traversal

~~F~~	~~D~~	~~J~~	~~B~~	~~E~~	~~G~~	~~K~~	~~A~~

~~C~~	~~I~~	~~H~~					

Order is : F D J B E G K A C I H

Queue is empty!



```
void level_order(Node * node){
```

```
void level_order(Node * node){
      queue <Node*> Q
      Q.push(node)
```

```
void level_order(Node * node){
      queue <Node*> Q
      Q.push(node)
      while(!Q.empty())
```

```
void level_order(Node * node){
     queue <Node*> Q
     Q.push(node)
     while(!Q.empty())
          Node* current = Q.front()
          Q.pop()
          print(current->data)
```

```
void level_order(Node * node){
     queue <Node*> Q
     Q.push(node)
     while(!Q.empty())
          Node* current = Q.front()
          Q.pop()
          print(current->data)
          if(current->left != Null)
               Q.push(current->left)
```

```
void level_order(Node * node){
     queue <Node*> Q
     Q.push(node)
     while(!Q.empty())
          Node* current = Q.front()
          Q.pop()
          print(current->data)
          if(current->left != Null)
               Q.push(current->left)
          if(current->right!=Null)
               Q.push(current->right)
```

```
void level_order(Node * node){
    queue <Node*> Q
    Q.push(node)
    while(!Q.empty())
        Node* current = Q.front()
        Q.pop()
        print(current->data)
        if(current->left != Null)
            Q.push(current->left)
        if(current->right!=Null)
            Q.push(current->right)
```

Time complexity?

```
void level_order(Node * node){
      queue <Node*> Q
      Q.push(node)
      while(!Q.empty())
            Node* current = Q.front()
            Q.pop()
            print(current->data)
            if(current->left != Null)
                  Q.push(current->left)
            if(current->right!=Null)
                  Q.push(current->right)
```

Time complexity?

O(N) for any traversal approach!

```
void level_order(Node * node){
    queue <Node*> Q
    Q.push(node)
    while(!Q.empty())
        Node* current = Q.front()
        Q.pop()
        print(current->data)
        if(current->left != Null)
            Q.push(current->left)
        if(current->right!=Null)
            Q.push(current->right)
```

Space complexity?

How much additional space is required?

```
void level_order(Node * node){
      queue <Node*> Q
      Q.push(node)
      while(!Q.empty())
            Node* current = Q.front()
            Q.pop()
            print(current->data)
            if(current->left != Null)
                  Q.push(current->left)
            if(current->right!=Null)
                  Q.push(current->right)
```

Space complexity?

How much additional space is required?

Need to find what portion of the Queue was occupied!

$$O(N/2) \sim O(N)$$

```
void level_order(Node * node){
    queue <Node*> Q
    Q.push(node)
    while(!Q.empty())
        Node* current = Q.front()
        Q.pop()
        print(current->data)
        if(current->left != Null)
            Q.push(current->left)
        if(current->right!=Null)
            Q.push(current->right)
```

Space complexity?

How much additional space is required?

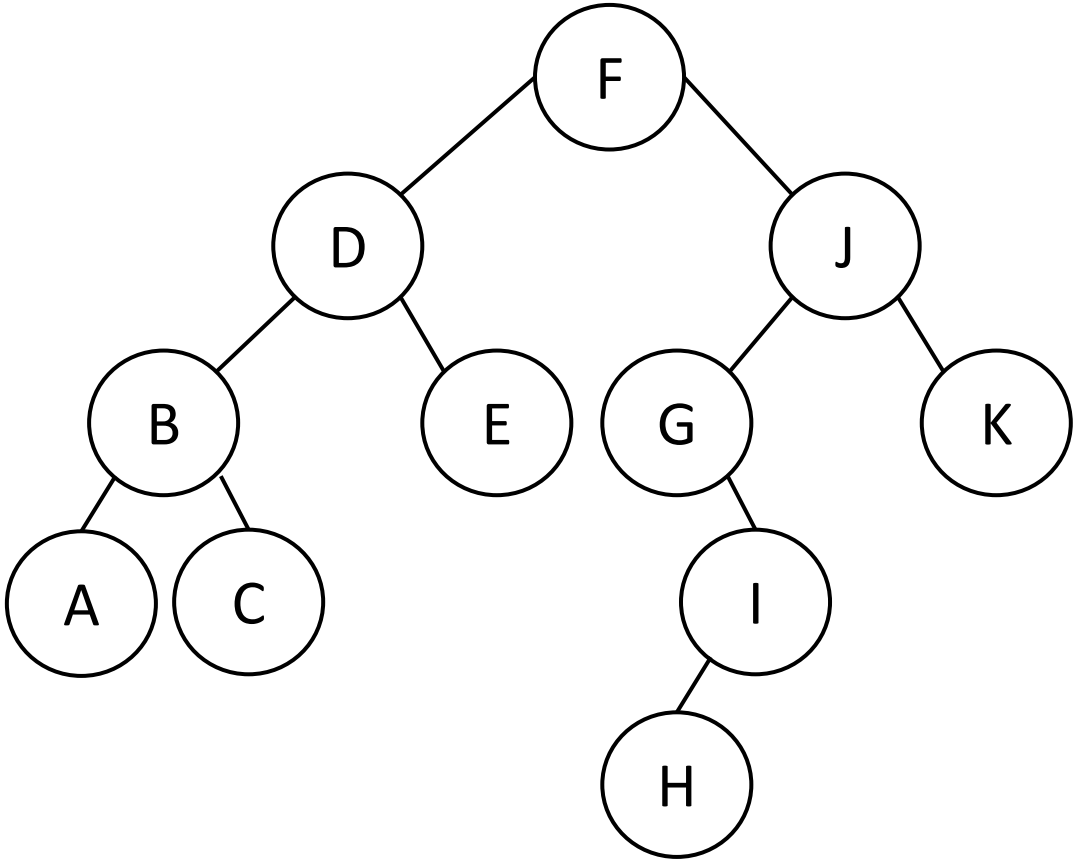
Need to find what portion of the Queue was occupied!

$$O(N/2) \sim O(N)$$

(max length of a level)

Preorder traversal

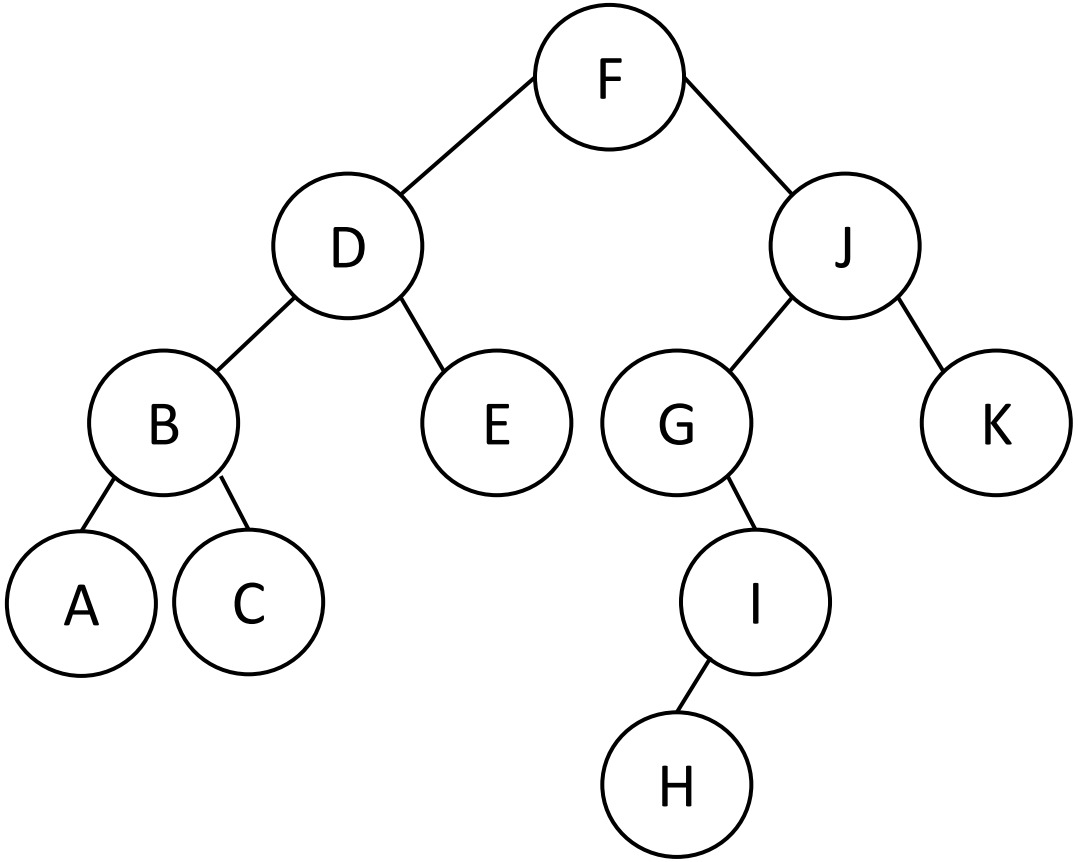
Order is :



Preorder traversal

Order is : F

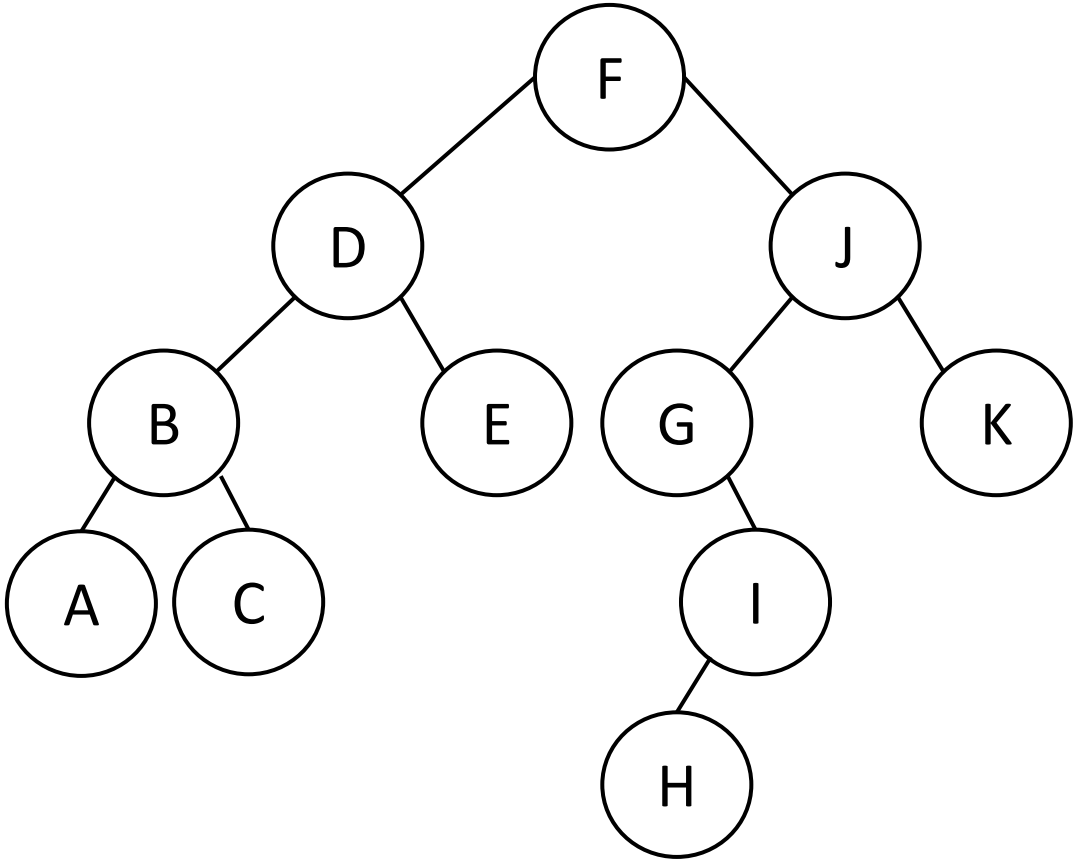
F(D)
F(F)



Preorder traversal

Order is : F D

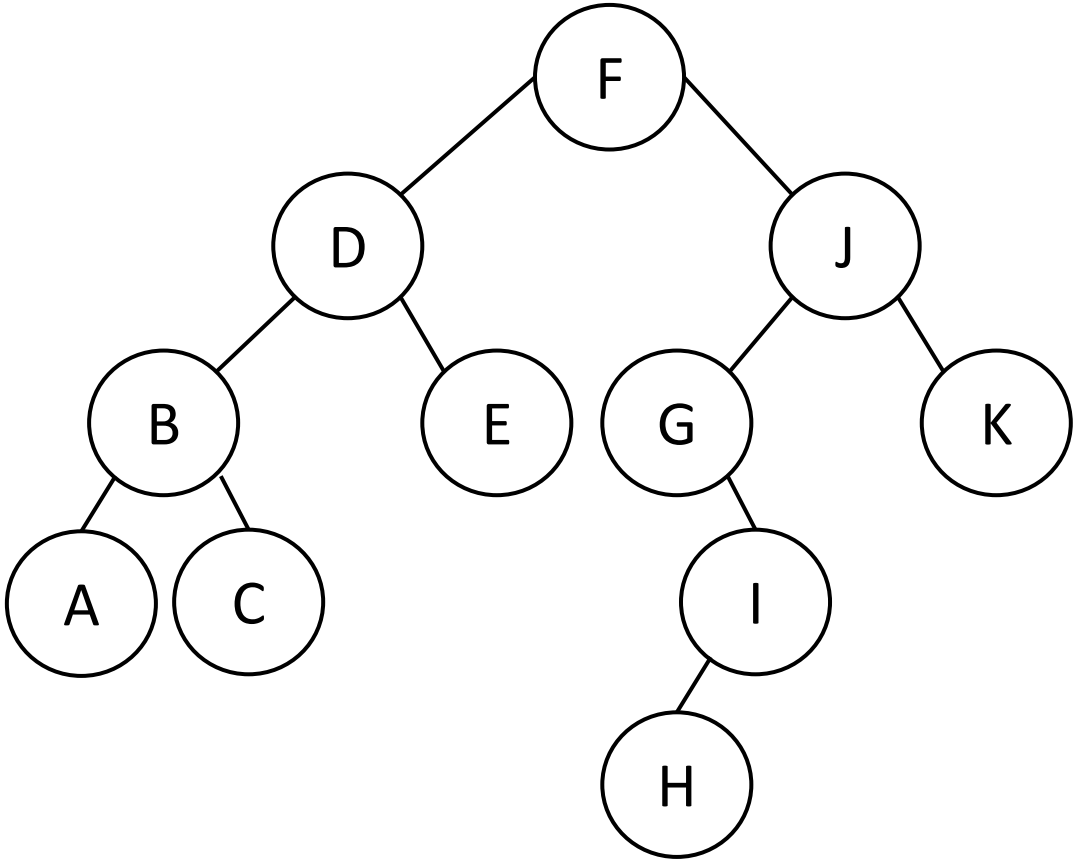
F(B)
F(D)
F(F)



Preorder traversal

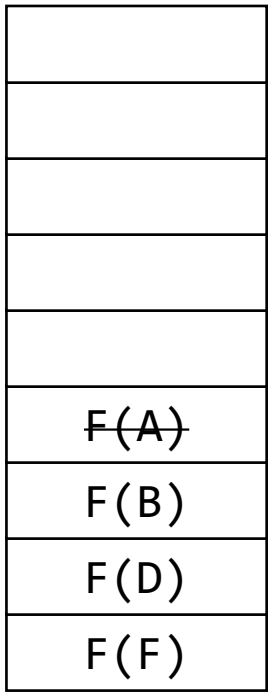
Order is : F D B

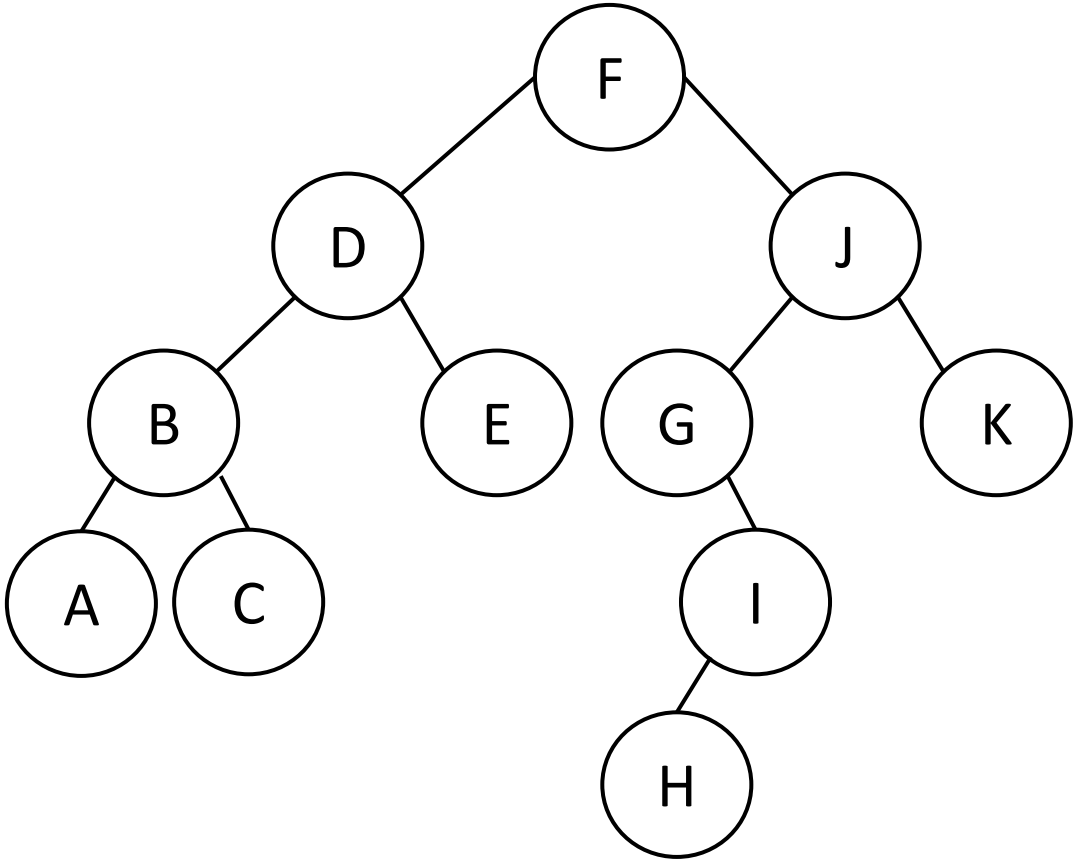
F(A)
F(B)
F(D)
F(F)



Preorder traversal

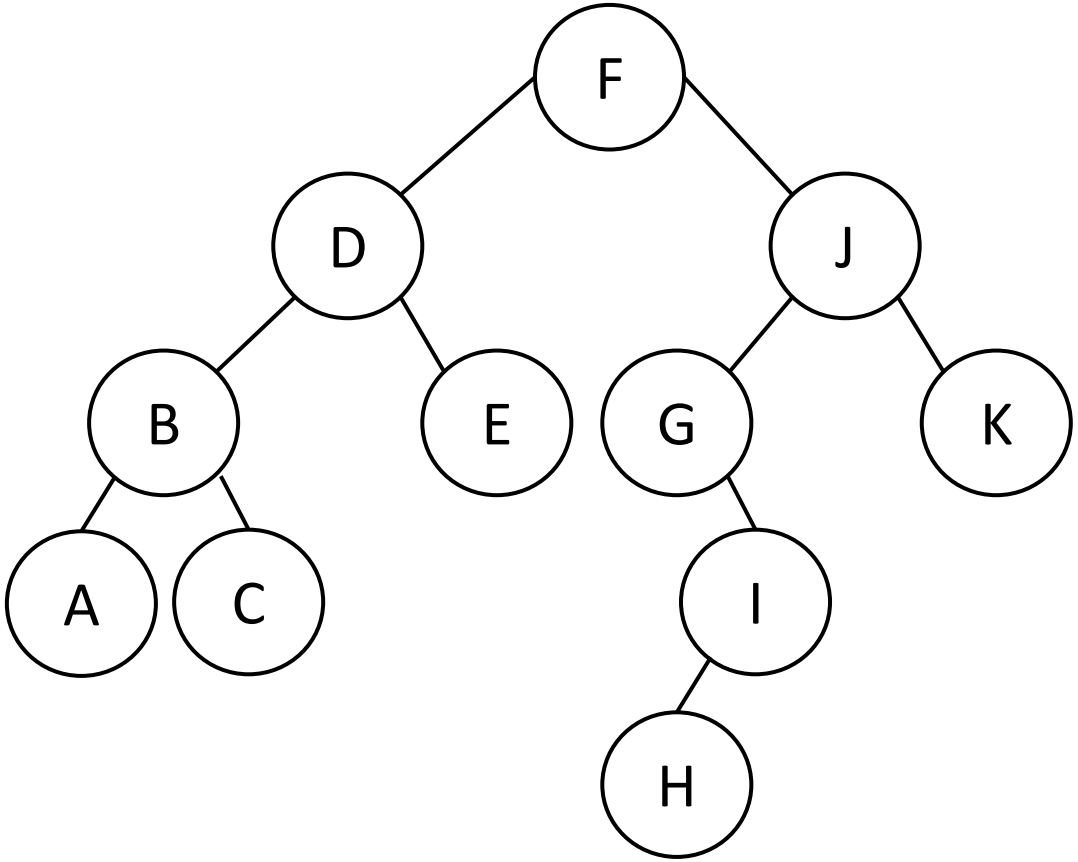
Order is : F D B A





Preorder traversal

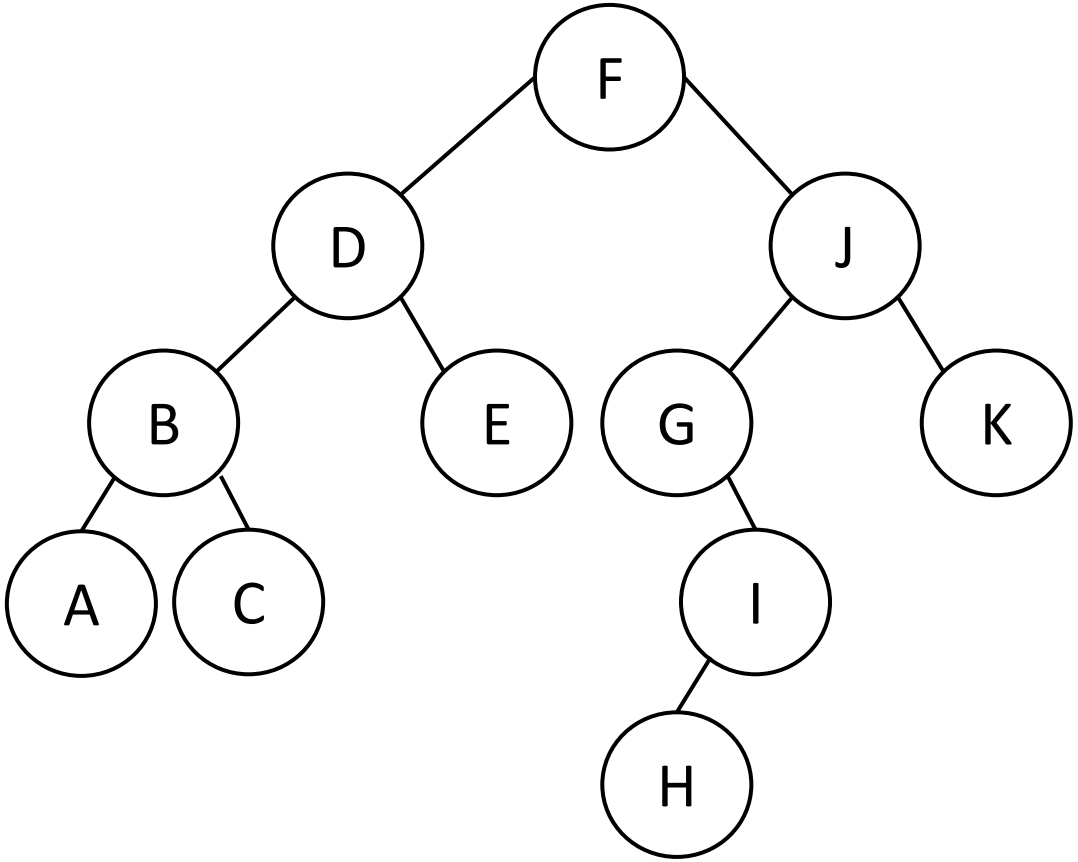
Order is : F D B A



F(C)
~~F(A)~~
F(B)
F(D)
F(F)

Preorder traversal

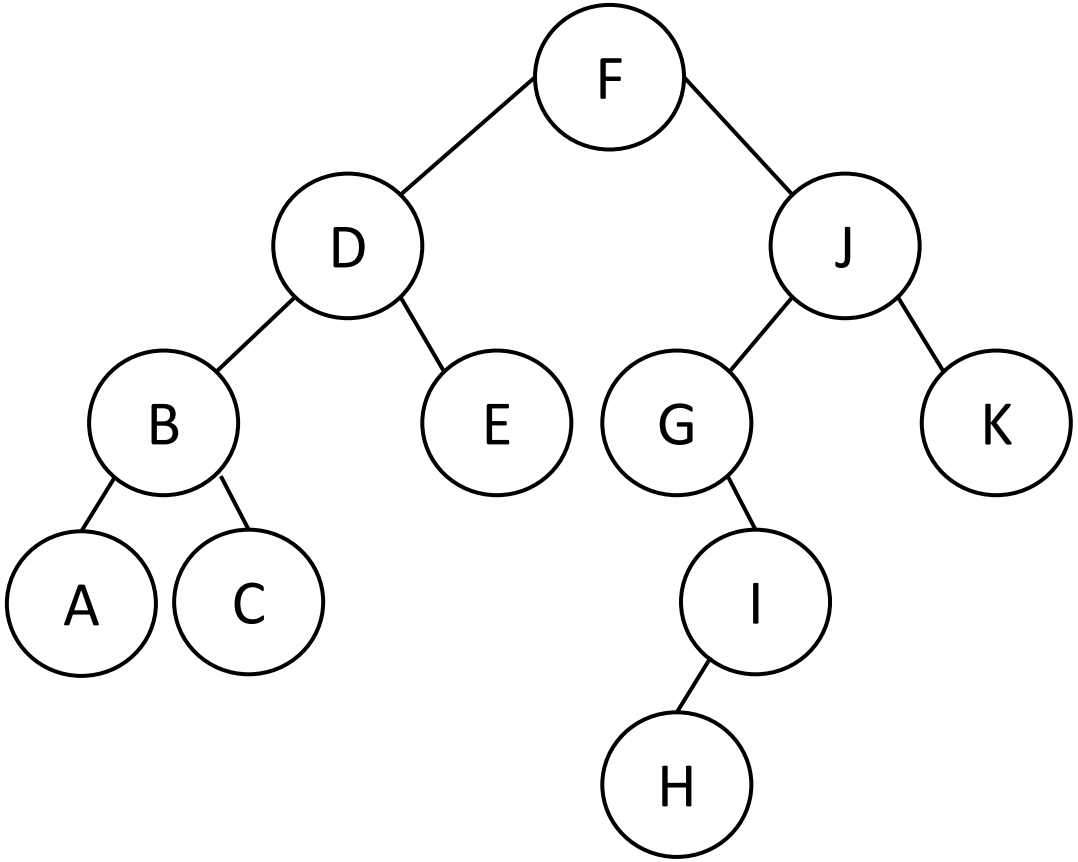
Order is : F D B A C



~~F(C)~~
~~F(A)~~
F(B)
F(D)
F(F)

Preorder traversal

Order is : F D B A C

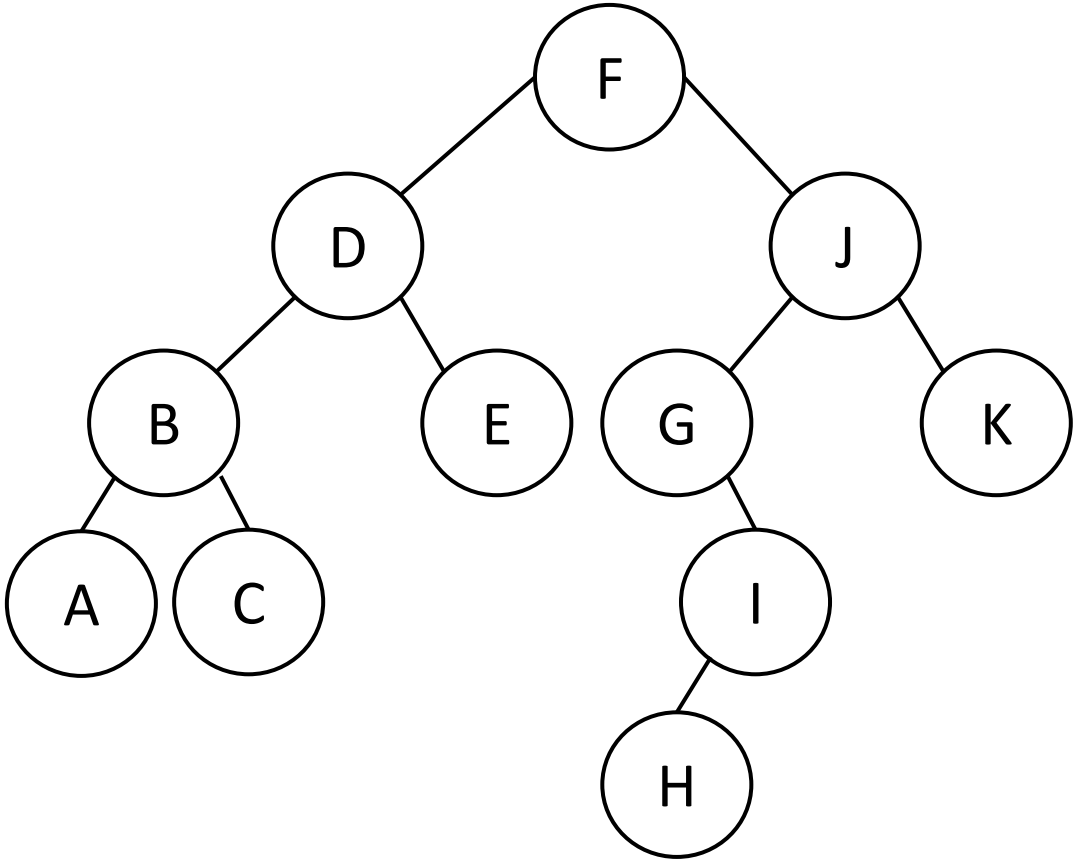


~~F(C)~~
~~F(A)~~
~~F(B)~~
F(D)
F(F)

Preorder traversal

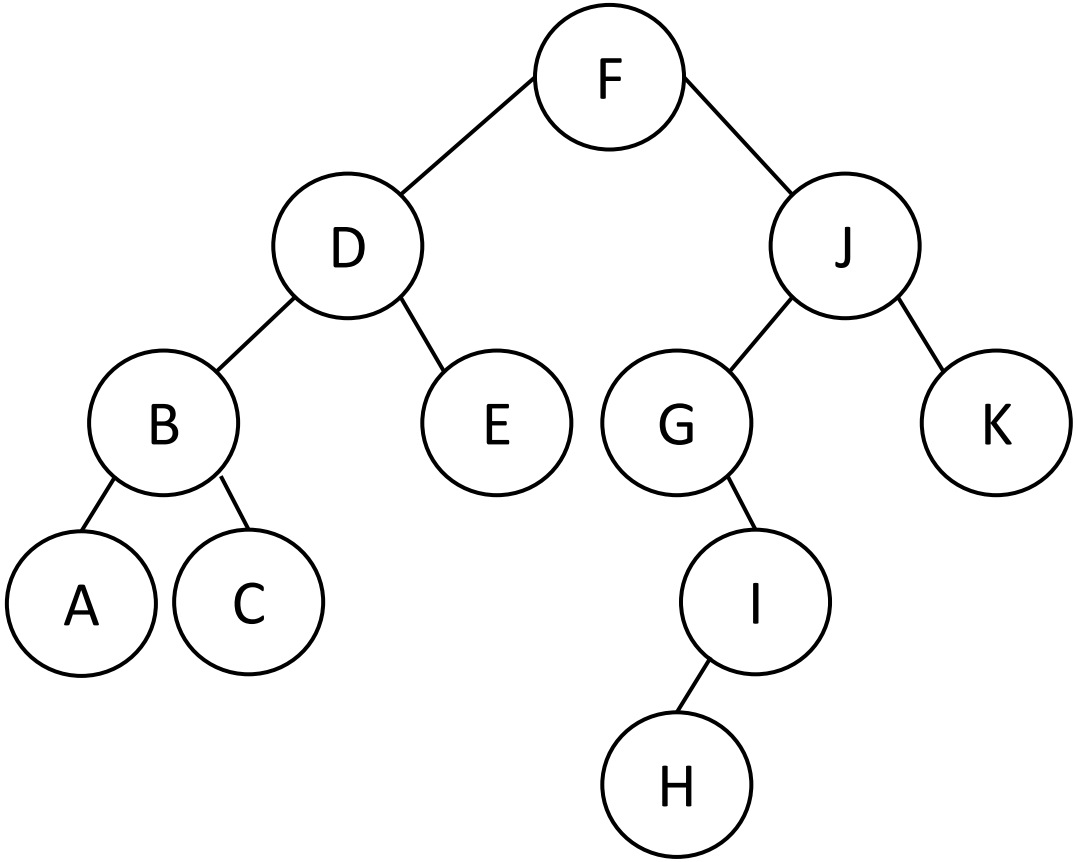
Order is : F D B A C

F(E)
~~F(C)~~
~~F(A)~~
~~F(B)~~
F(D)
F(F)



Preorder traversal

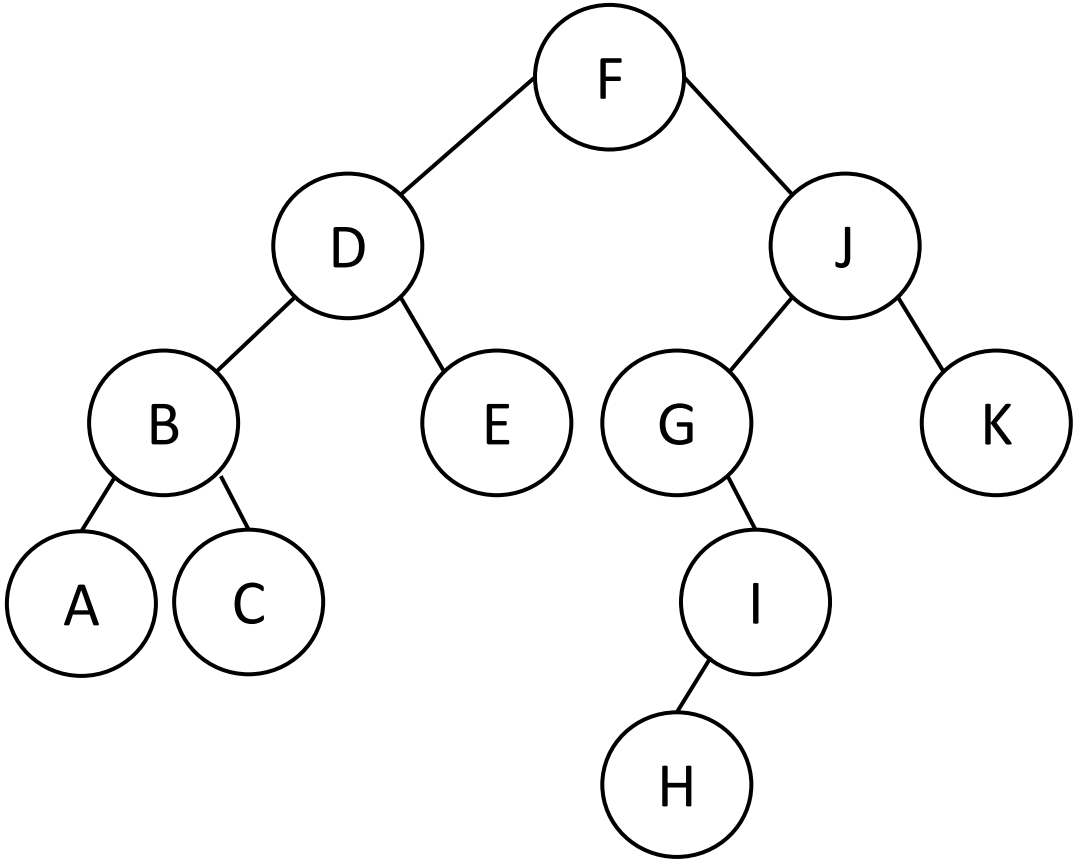
Order is : F D B A C E



~~F(E)~~
~~F(C)~~
~~F(A)~~
~~F(B)~~
F(D)
F(F)

Preorder traversal

Order is : F D B A C E

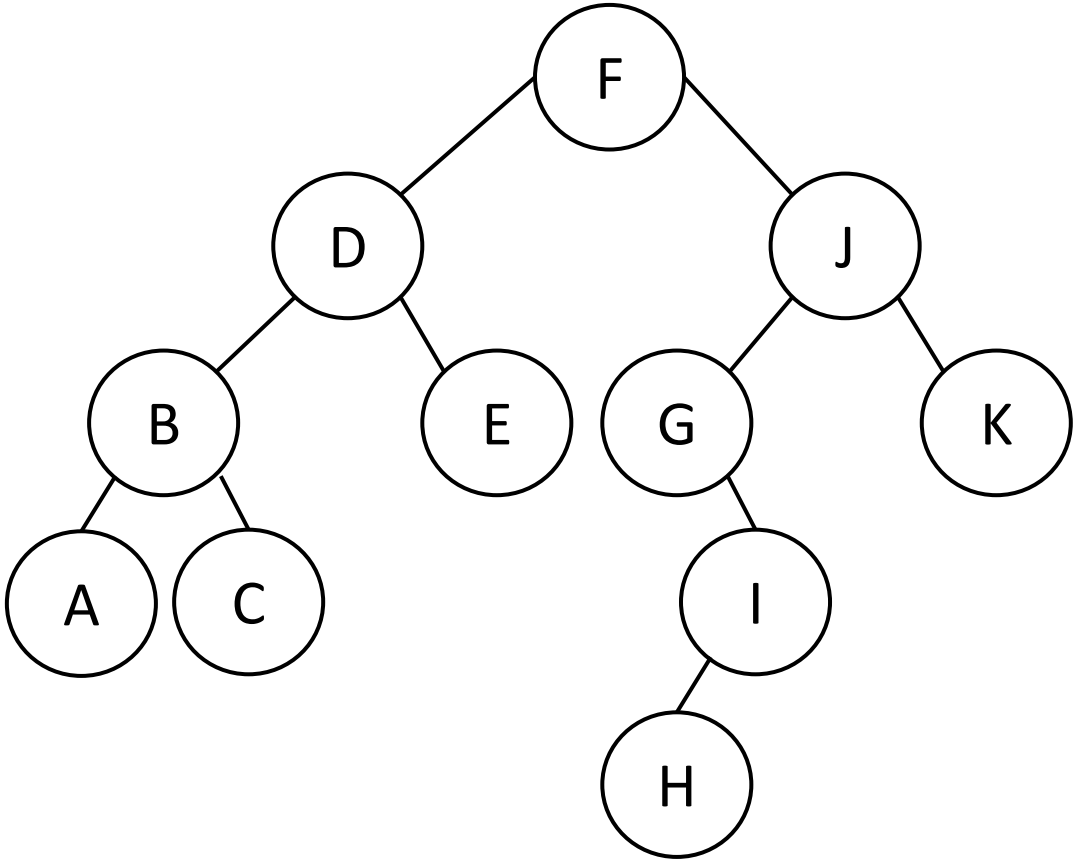


~~F(E)~~
~~F(C)~~
~~F(A)~~
~~F(B)~~
~~F(D)~~
F(F)

Preorder traversal

Order is : F D B A C E

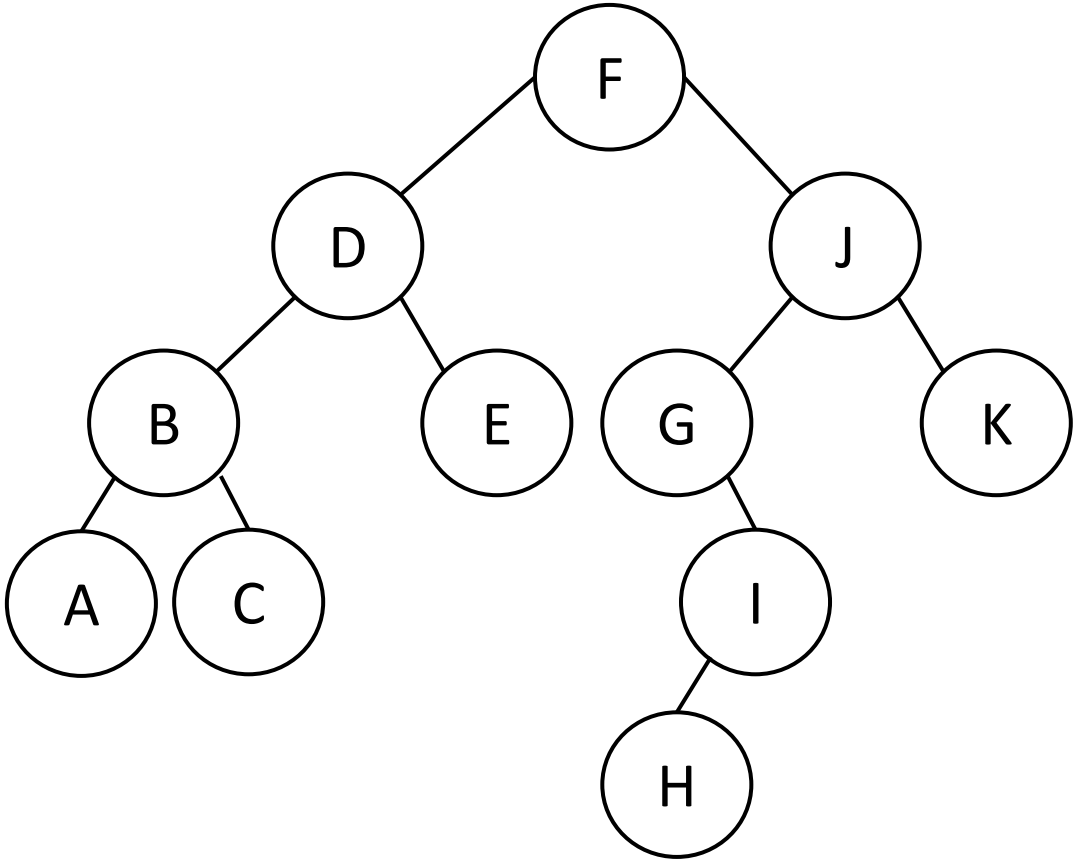
F(J)
~~F(E)~~
~~F(C)~~
~~F(A)~~
~~F(B)~~
~~F(D)~~
F(F)



Preorder traversal

Order is : F D B A C E J

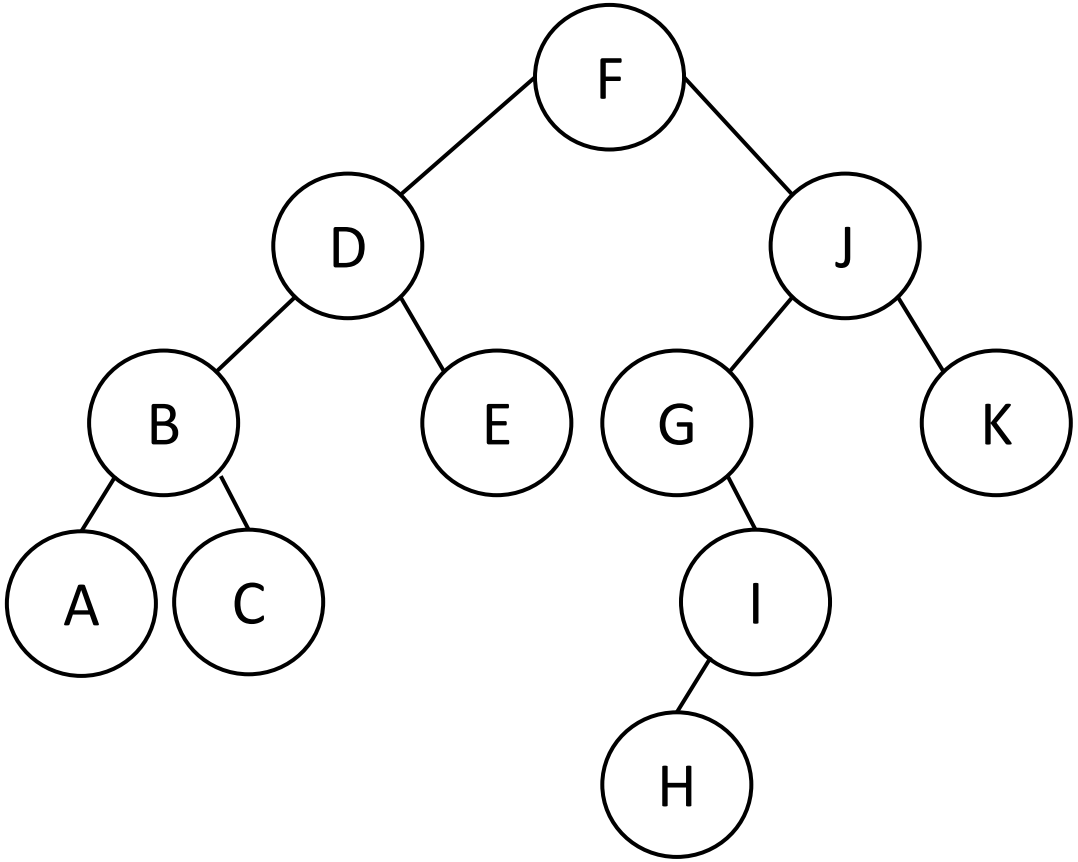
F(G)
F(J)
~~F(E)~~
~~F(C)~~
~~F(A)~~
~~F(B)~~
~~F(D)~~
F(F)



Preorder traversal

Order is : F D B A C E J G

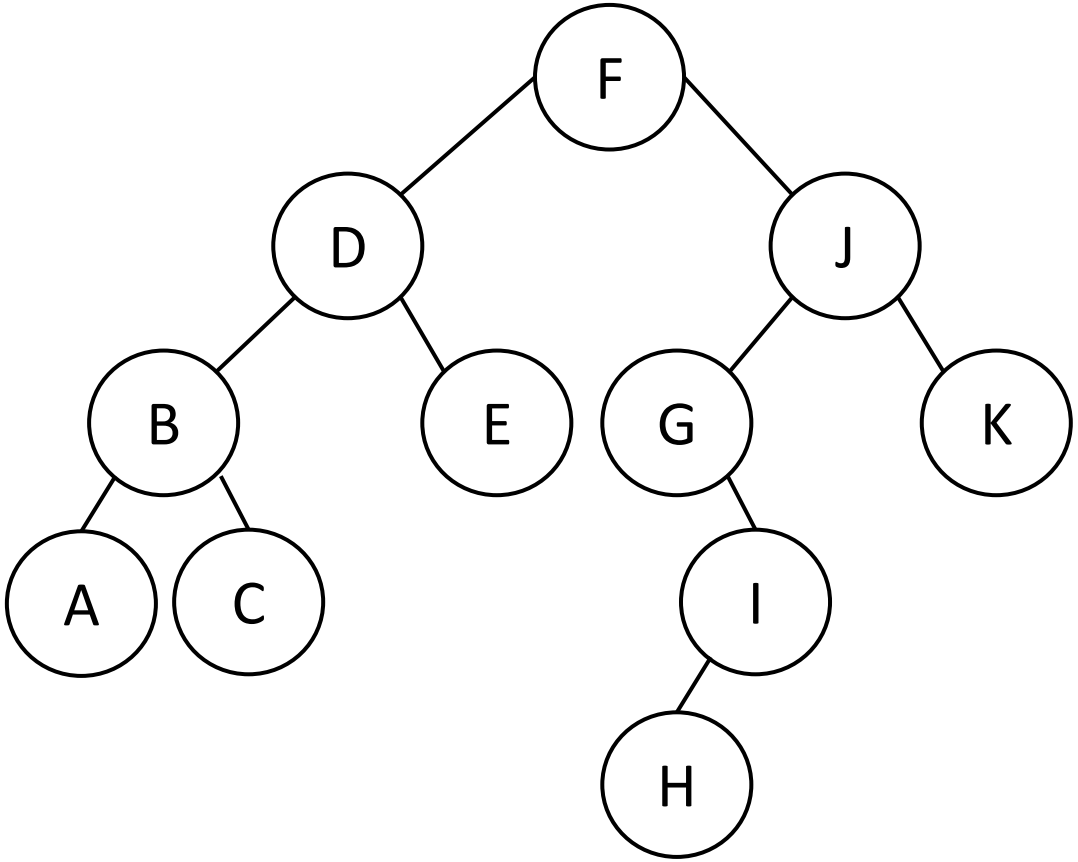
F(I)
F(G)
F(J)
~~F(E)~~
~~F(C)~~
~~F(A)~~
~~F(B)~~
~~F(D)~~
F(F)



Preorder traversal

Order is : F D B A C E J G I

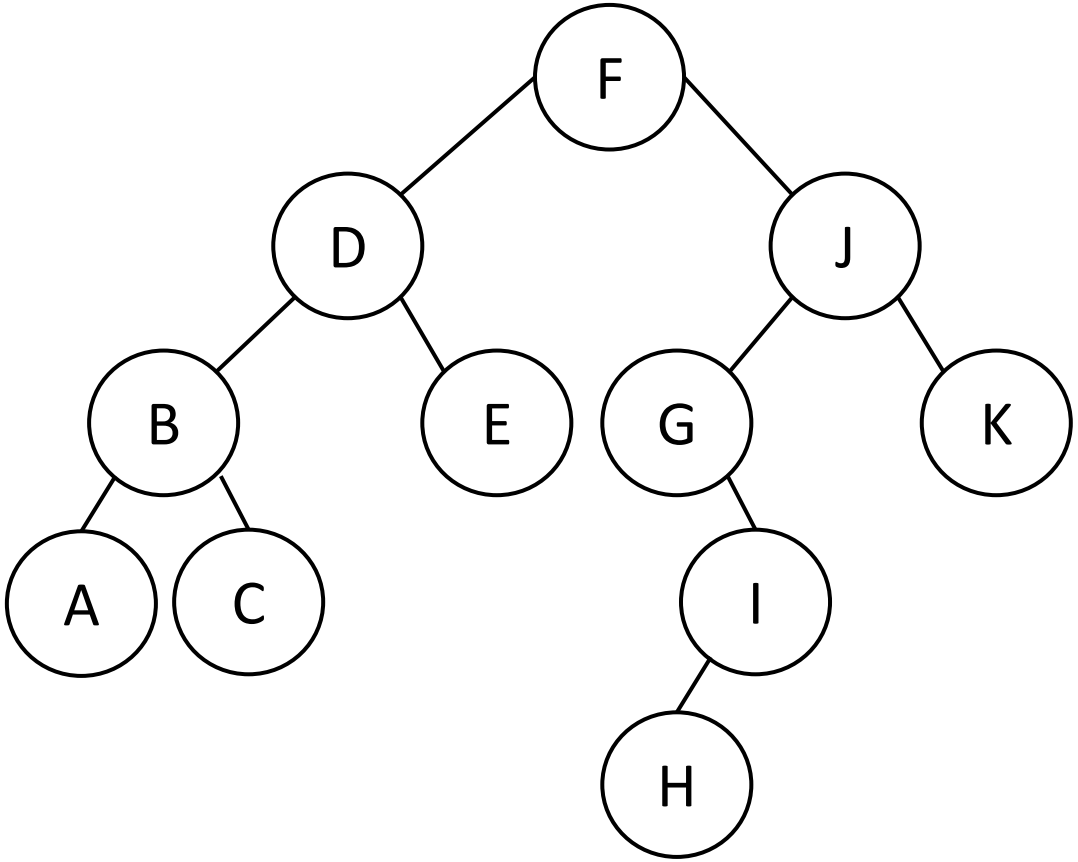
F(H)
F(I)
F(G)
F(J)
~~F(E)~~
~~F(C)~~
~~F(A)~~
~~F(B)~~
~~F(D)~~
F(F)



Preorder traversal

Order is : F D B A C E J G I H

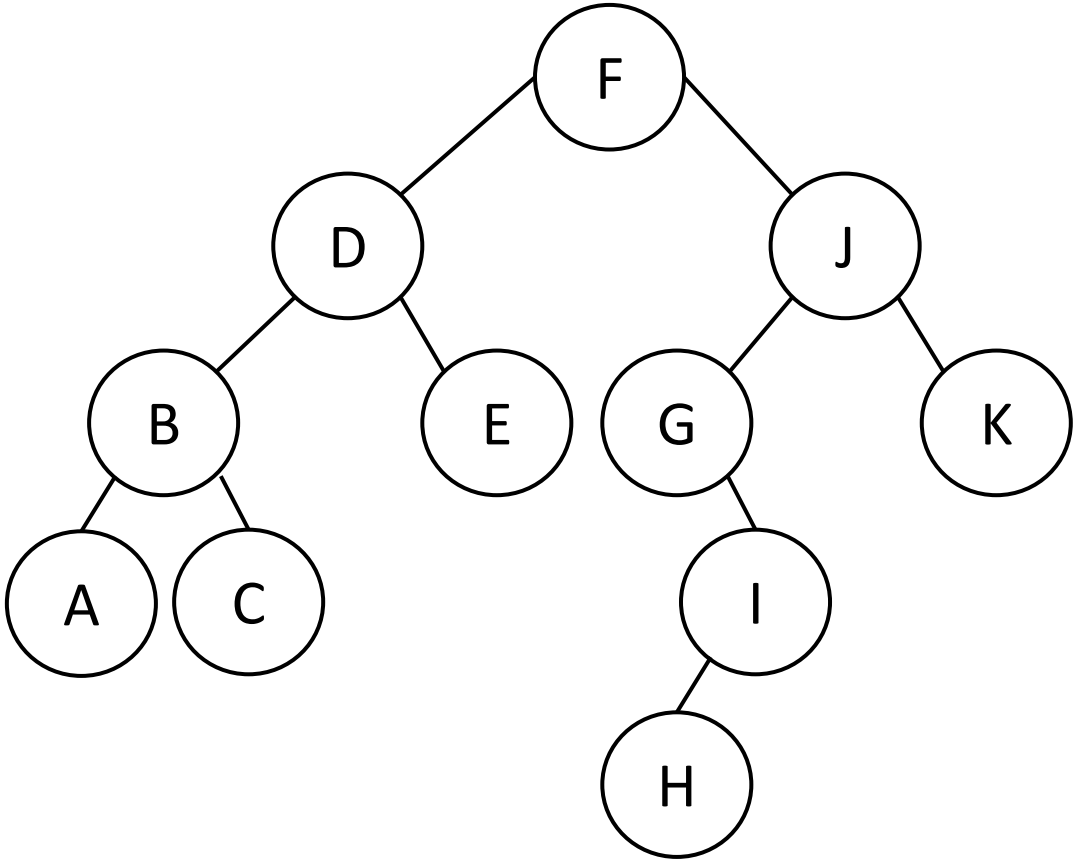
~~F(H)~~
F(I)
F(G)
F(J)
~~F(E)~~
~~F(C)~~
~~F(A)~~
~~F(B)~~
~~F(D)~~
F(F)



Preorder traversal

Order is : F D B A C E J G I H

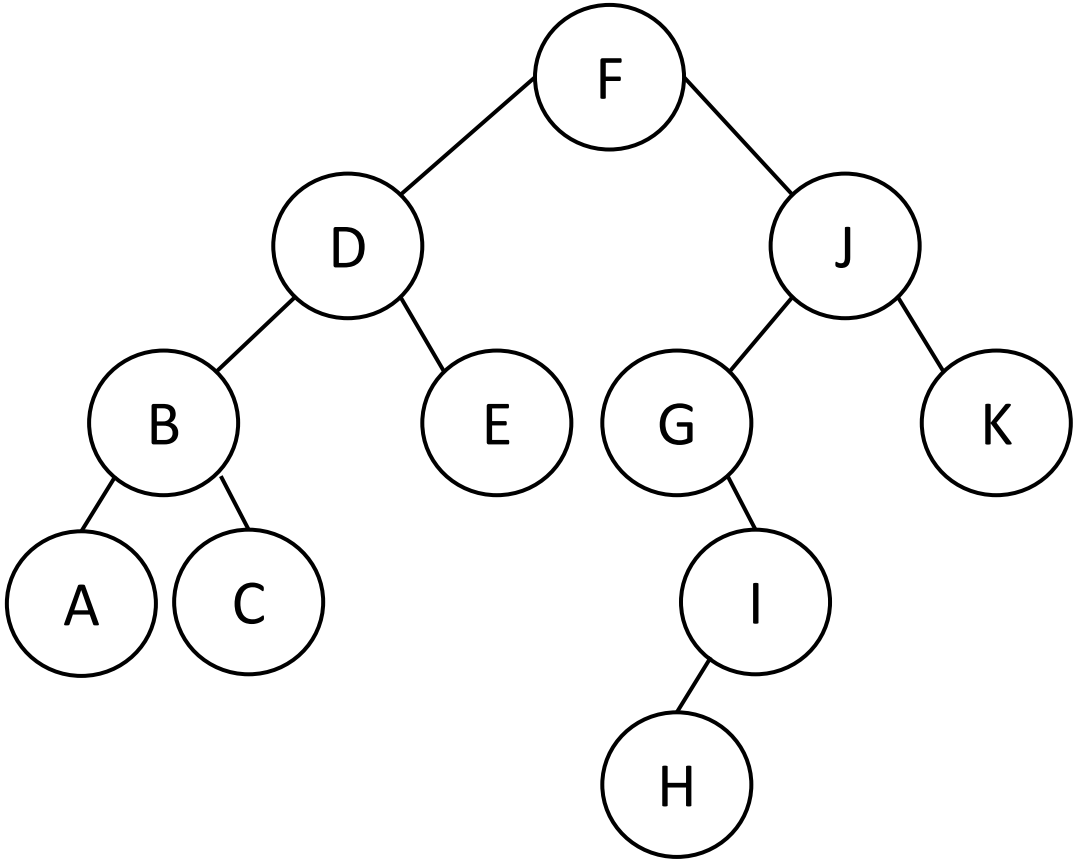
~~F(H)~~
~~F(I)~~
F(G)
F(J)
~~F(E)~~
~~F(C)~~
~~F(A)~~
~~F(B)~~
~~F(D)~~
F(F)



Preorder traversal

Order is : F D B A C E J G I H

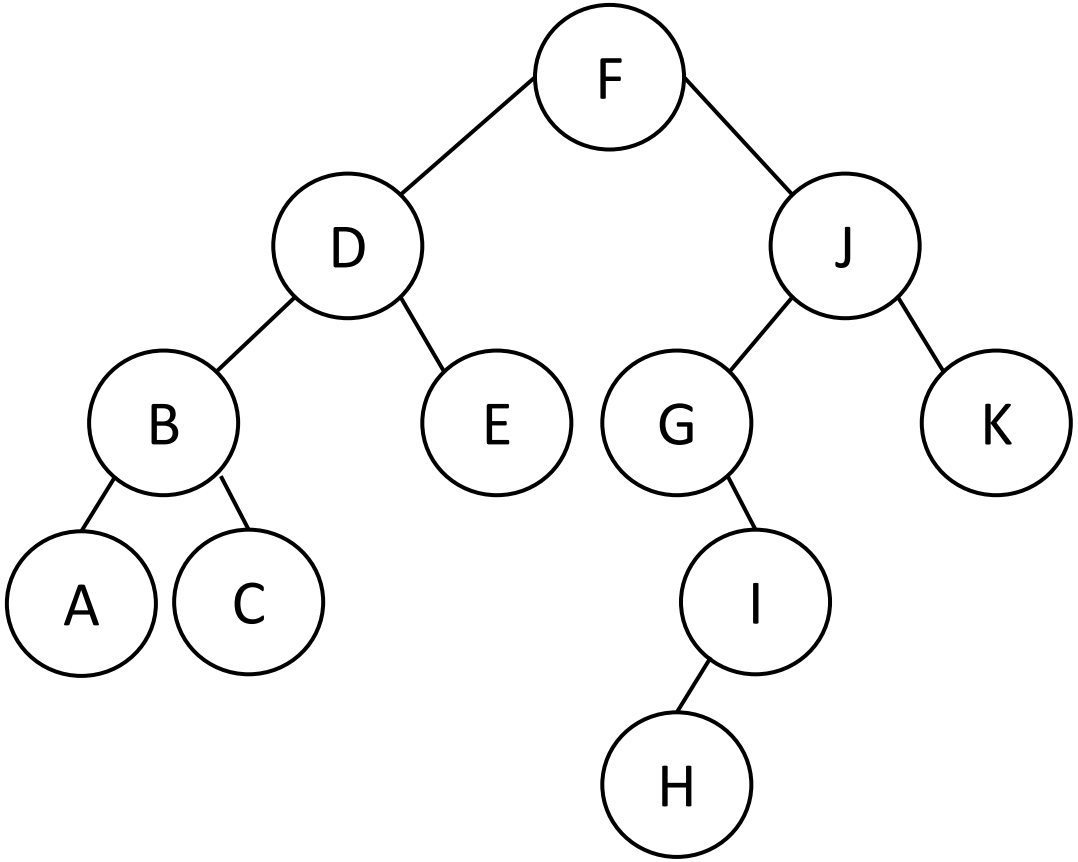
~~F(H)~~
~~F(I)~~
~~F(G)~~
F(J)
~~F(E)~~
~~F(C)~~
~~F(A)~~
~~F(B)~~
~~F(D)~~
F(F)



Preorder traversal

Order is : F D B A C E J G I H

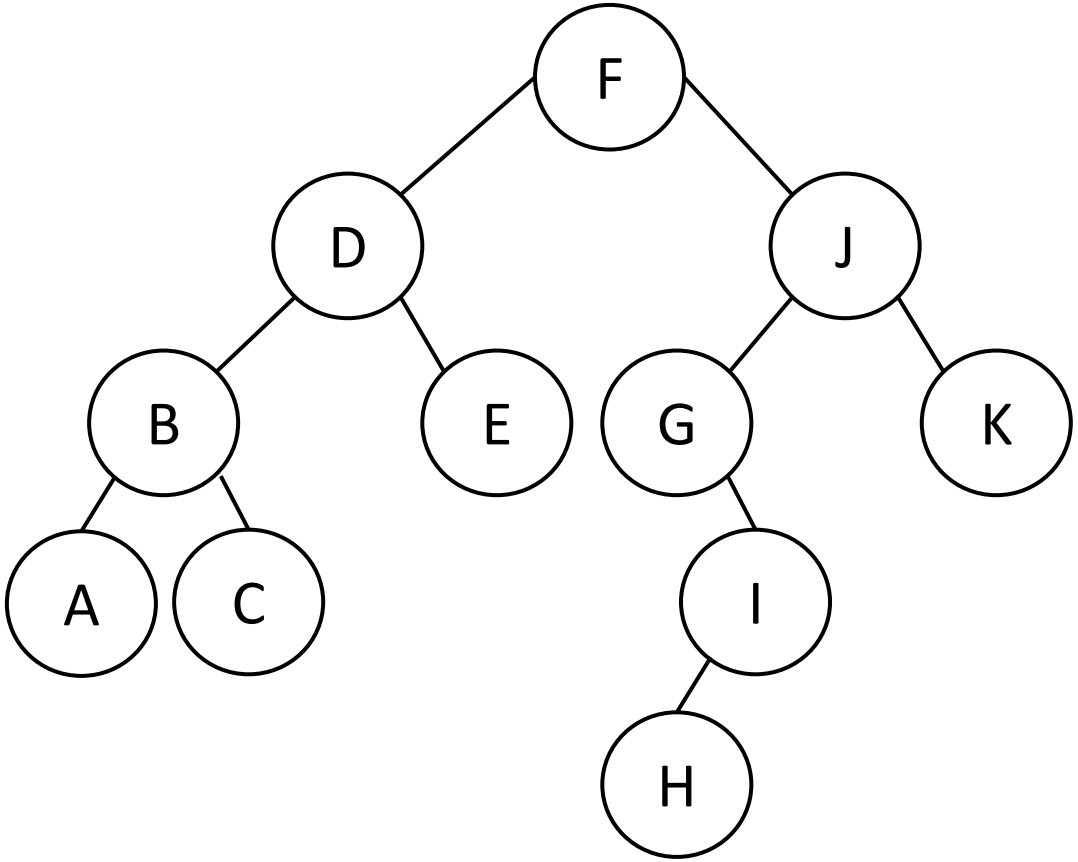
F(K)
~~F(H)~~
~~F(I)~~
~~F(G)~~
F(J)
~~F(E)~~
~~F(C)~~
~~F(A)~~
~~F(B)~~
~~F(D)~~
F(F)



Preorder traversal

Order is : F D B A C E J G I H K

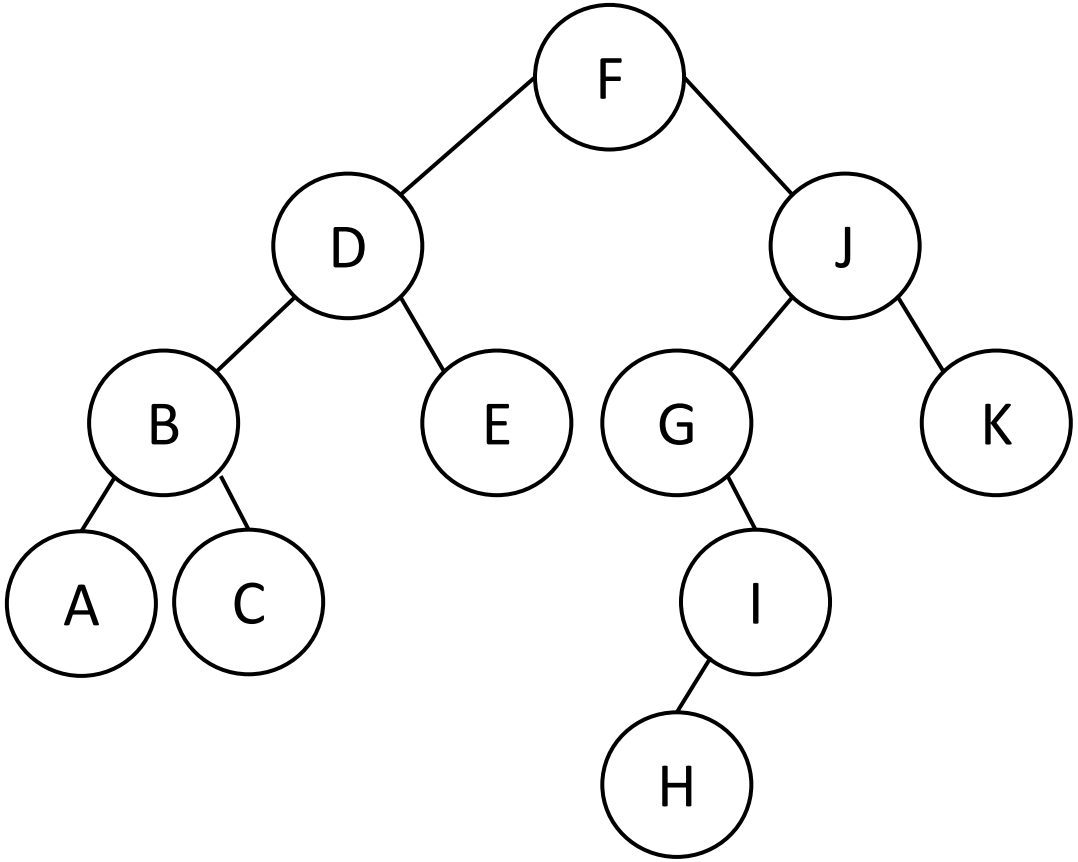
~~F(K)~~
~~F(H)~~
~~F(I)~~
~~F(G)~~
F(J)
~~F(E)~~
~~F(C)~~
~~F(A)~~
~~F(B)~~
~~F(D)~~
F(F)



Preorder traversal

Order is : F D B A C E J G I H K

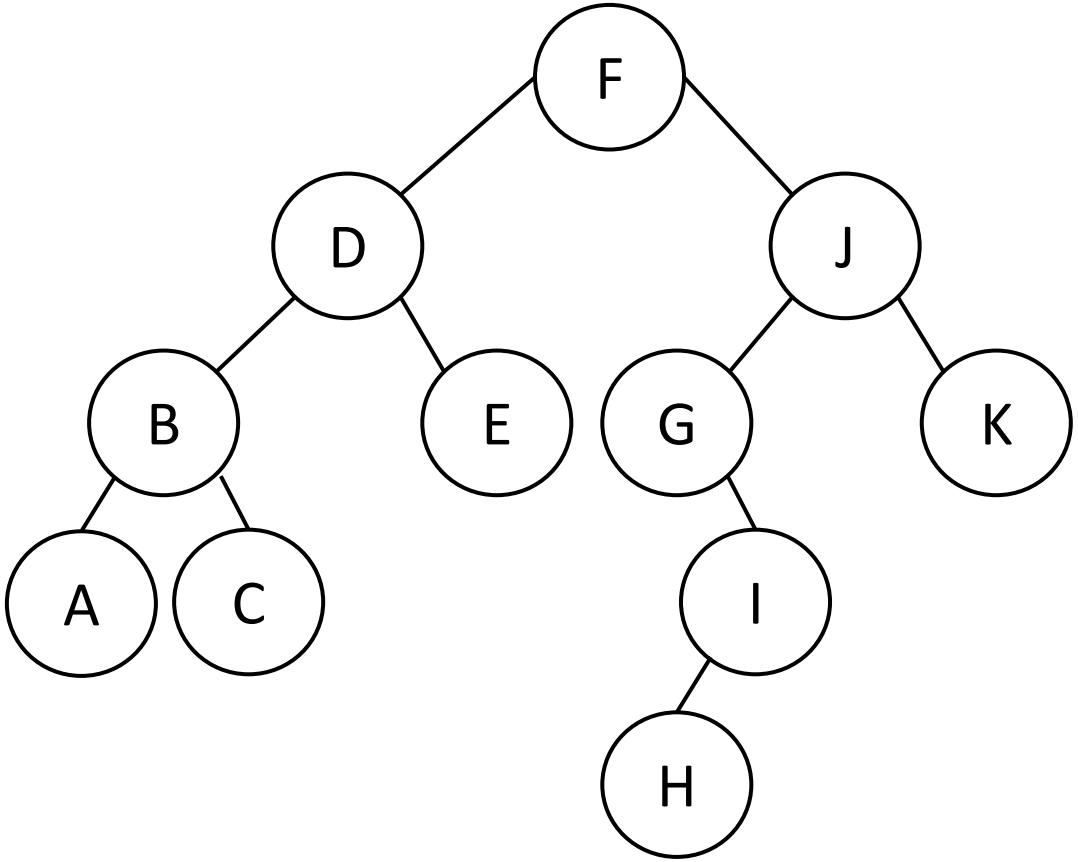
~~F(K)~~
~~F(H)~~
~~F(I)~~
~~F(G)~~
~~F(J)~~
~~F(E)~~
~~F(C)~~
~~F(A)~~
~~F(B)~~
~~F(D)~~
F(F)



Preorder traversal

Order is : F D B A C E J G I H K

~~F(K)~~
~~F(H)~~
~~F(I)~~
~~F(G)~~
~~F(J)~~
~~F(E)~~
~~F(C)~~
~~F(A)~~
~~F(B)~~
~~F(D)~~
~~F(F)~~



```
void preorder(Node *node){



}
```

```
void preorder(Node *node){


     print( node->data)
     preorder(node->left)
     preorder(node->right)
}
```

```
void preorder(Node *node){
     if (node == Null)
          return
     print( node->data)
     preorder(node->left)
     preorder(node->right)
}
```

```
void preorder(Node *node){
      if (node == Null)
            return
      print( node->data)
      preorder(node->left)
      preorder(node->right)
}
```

Time complexity: $O(N)$

```
void preorder(Node *node){
      if (node == Null)
            return
      print( node->data)
      preorder(node->left)
      preorder(node->right)
}
```

Time complexity: $O(N)$

Space complexity?

Need to find what was the max length of the recursion stack!!!

```
void preorder(Node *node){
      if (node == Null)
            return
      print( node->data)
      preorder(node->left)
      preorder(node->right)
}
```

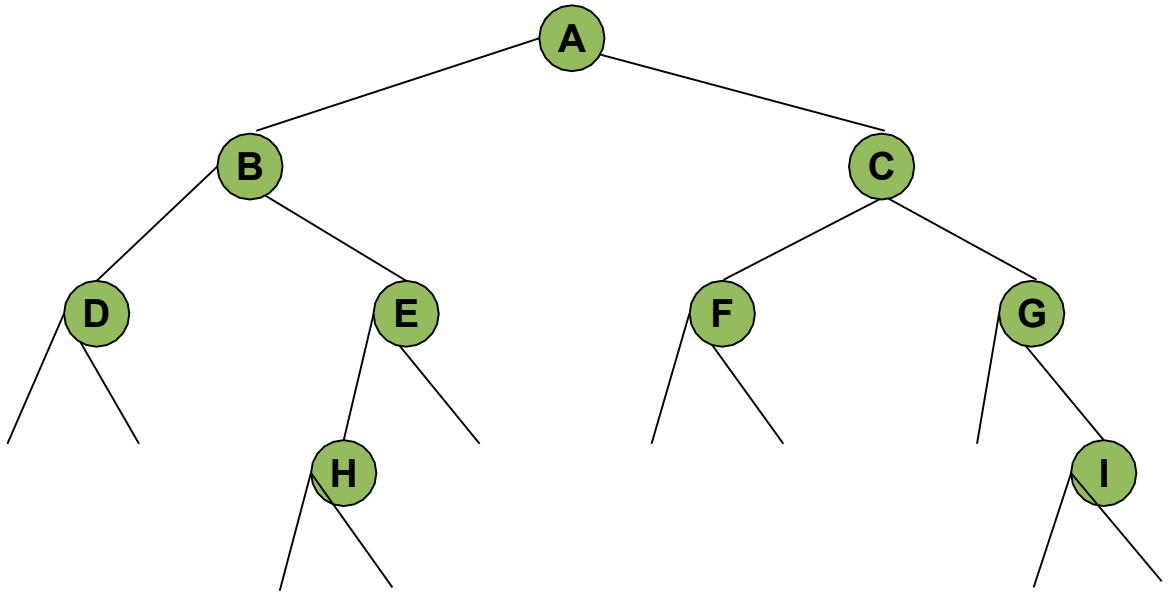
Time complexity: $O(N)$

Space complexity?

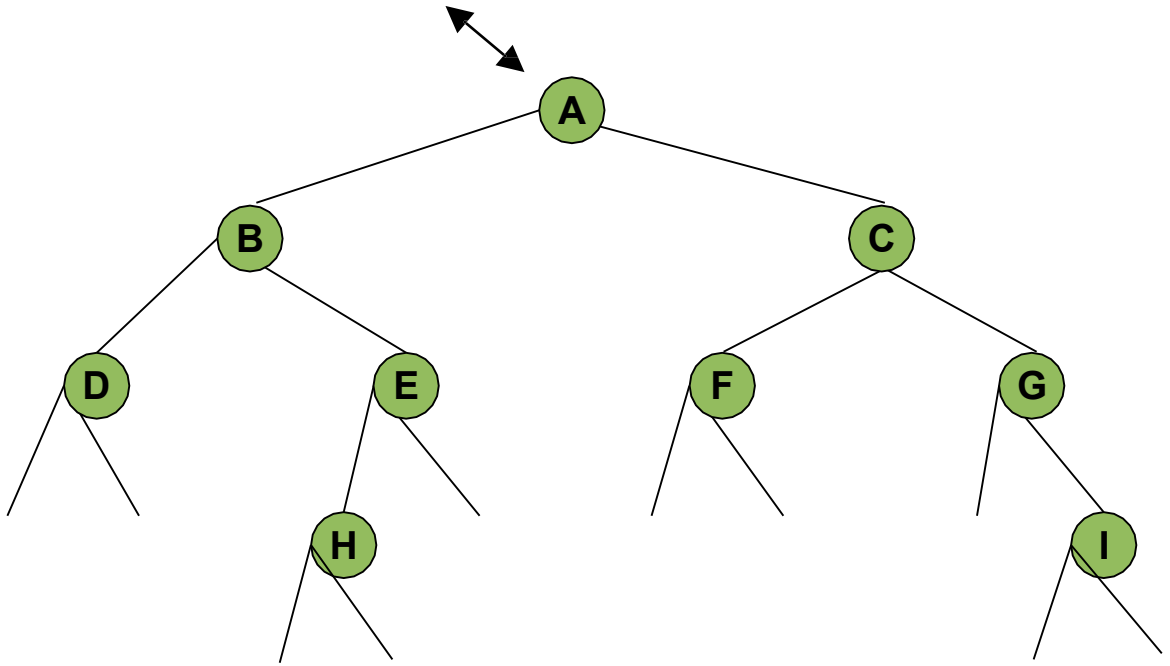
Need to find what was the max length of the recursion stack!!!

$$O(height)$$

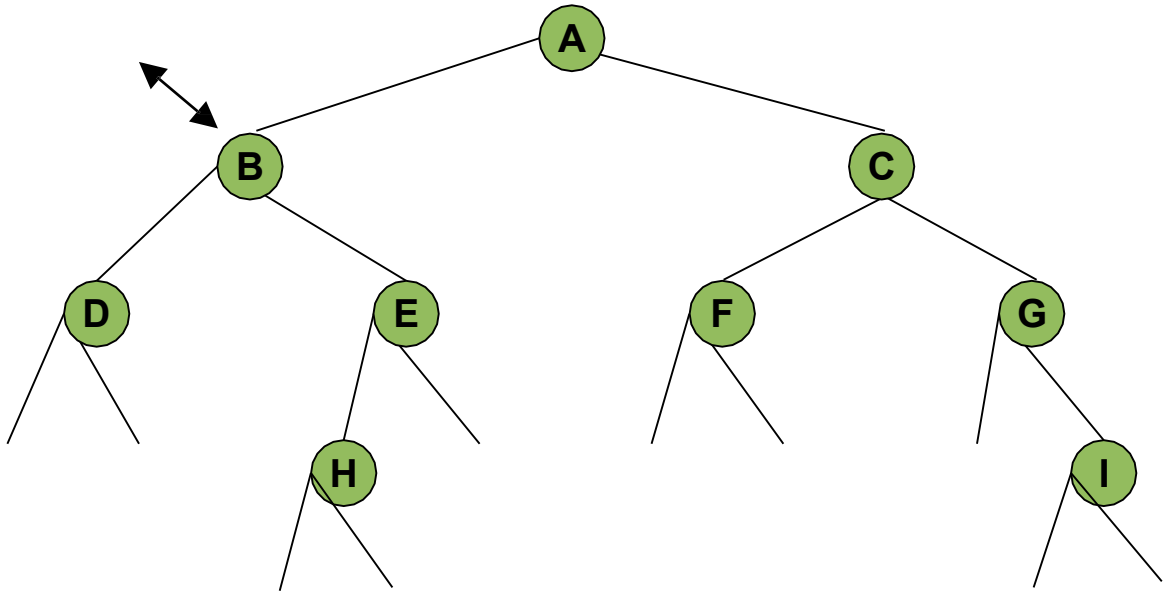
Inorder traversal



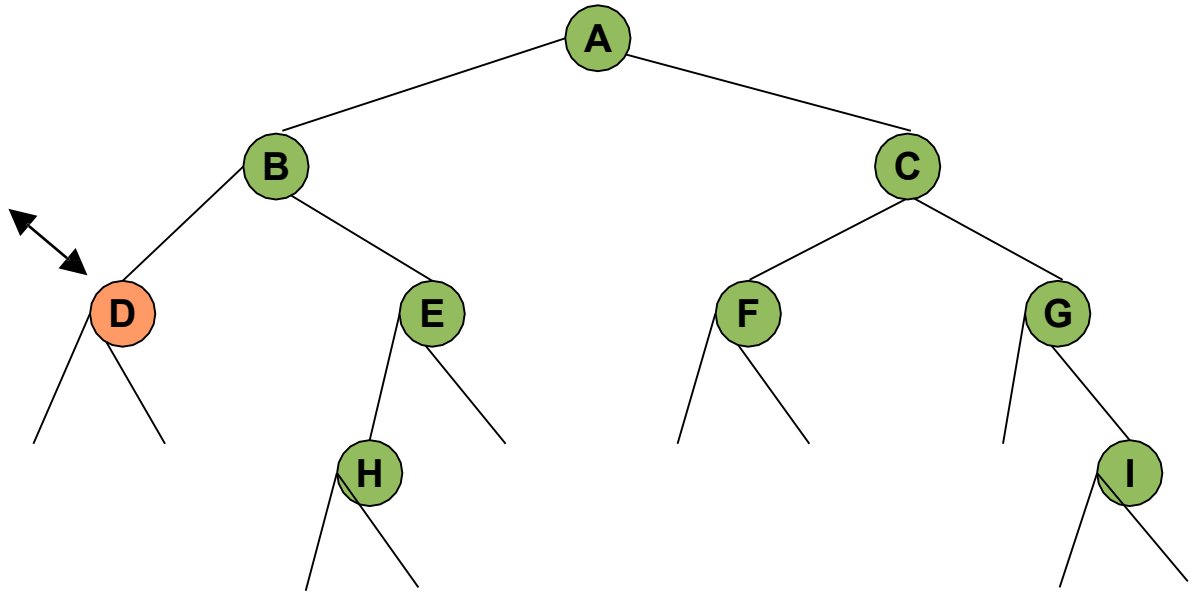
Inorder traversal



Inorder traversal

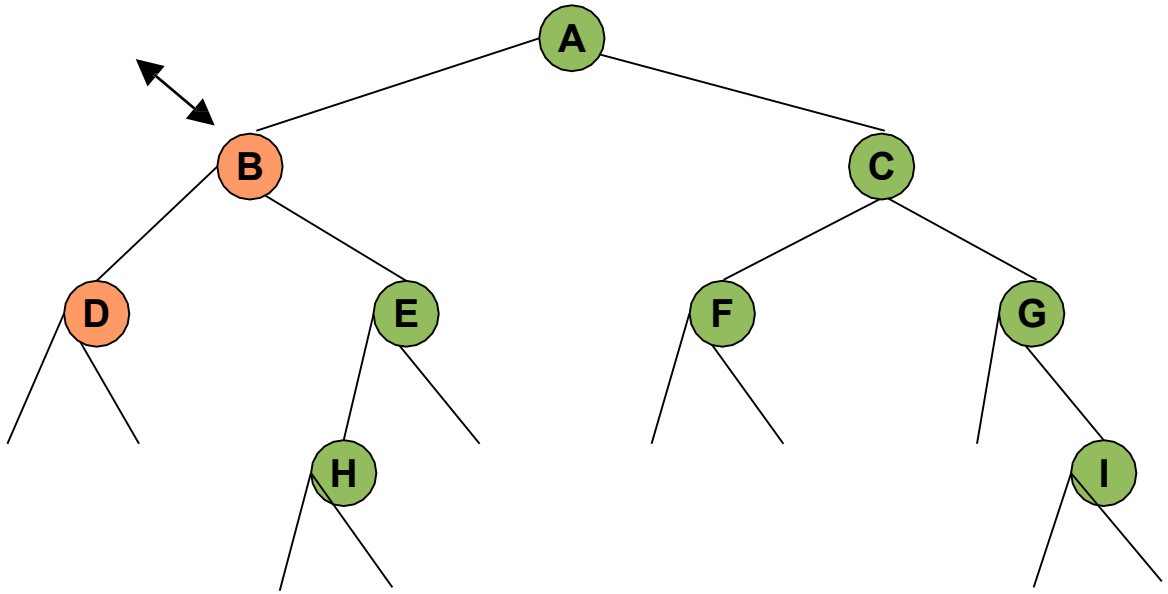


Inorder traversal



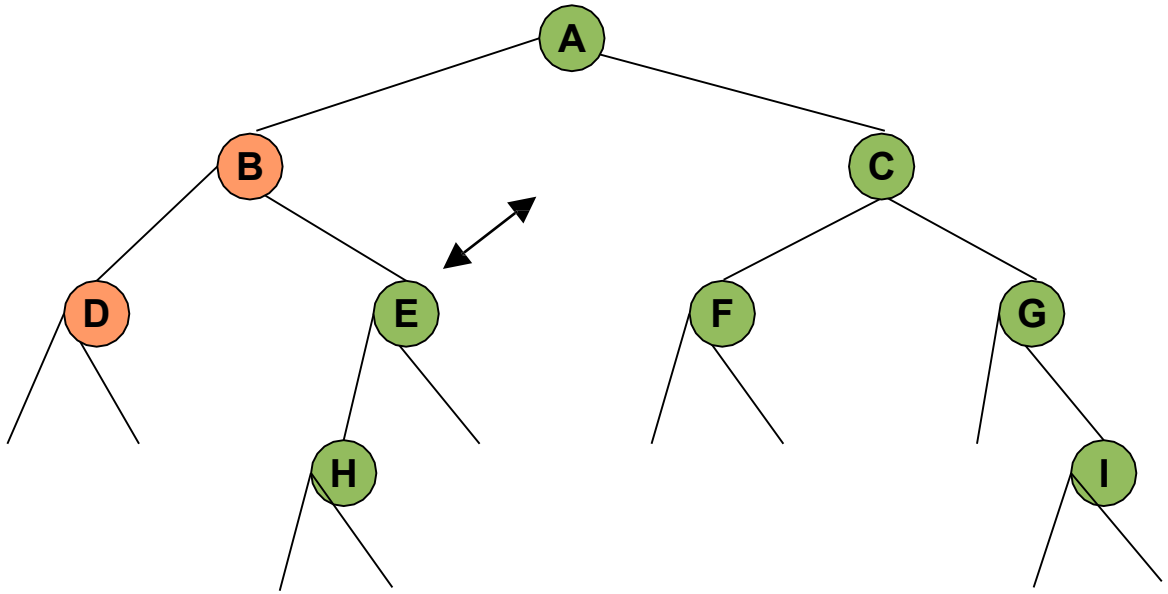
Result: D

Inorder traversal



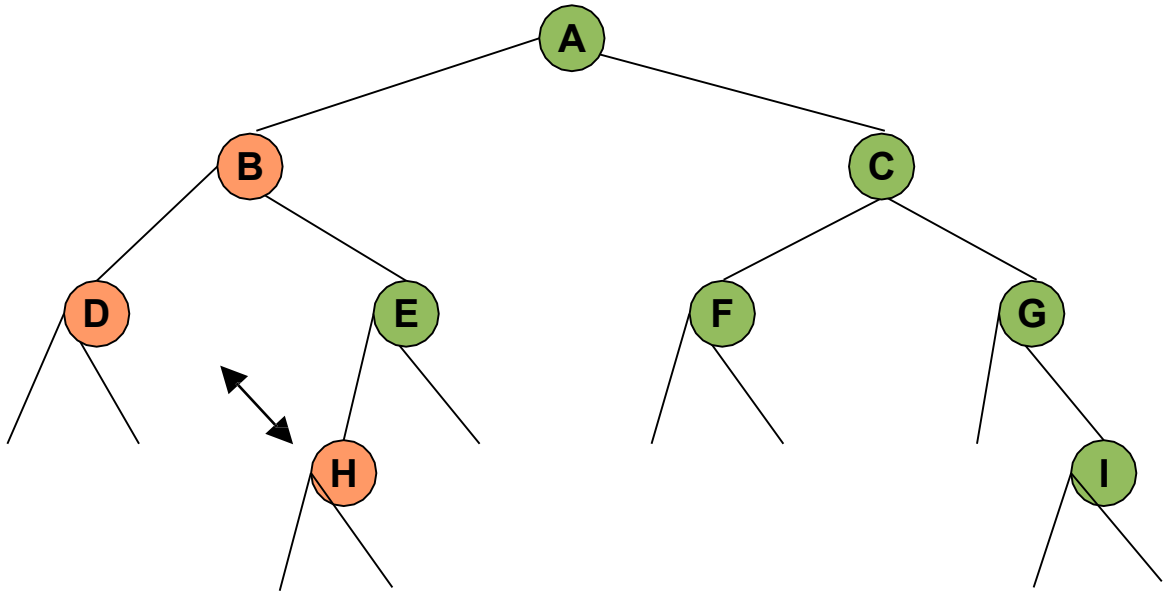
Result: DB

Inorder traversal



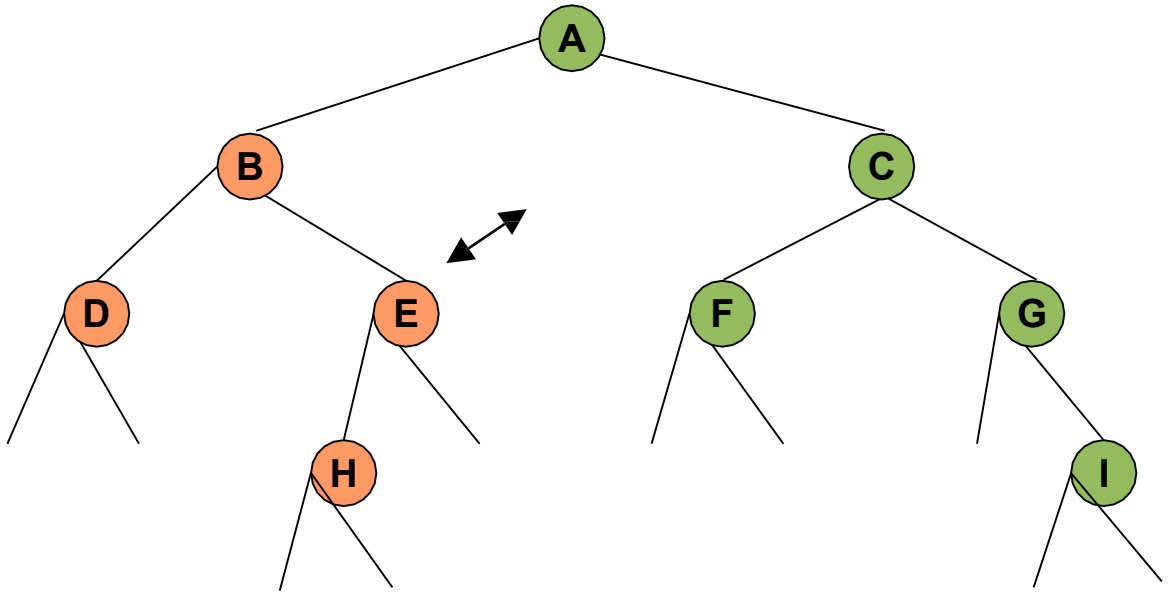
Result: DB

Inorder traversal



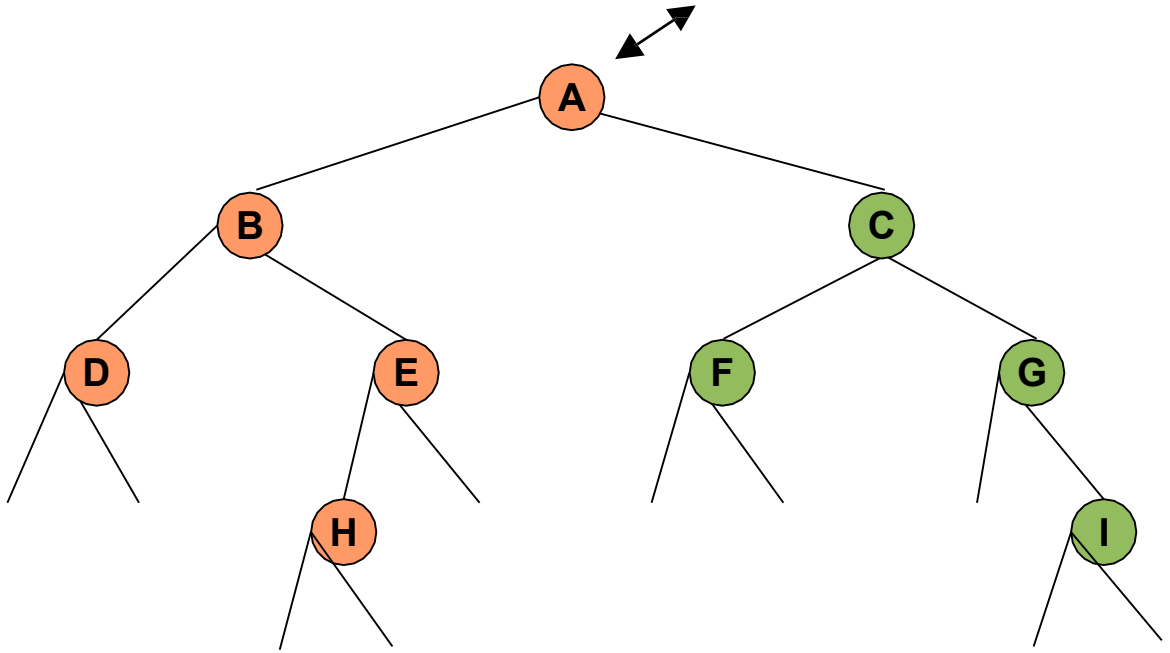
Result: DBH

Inorder traversal



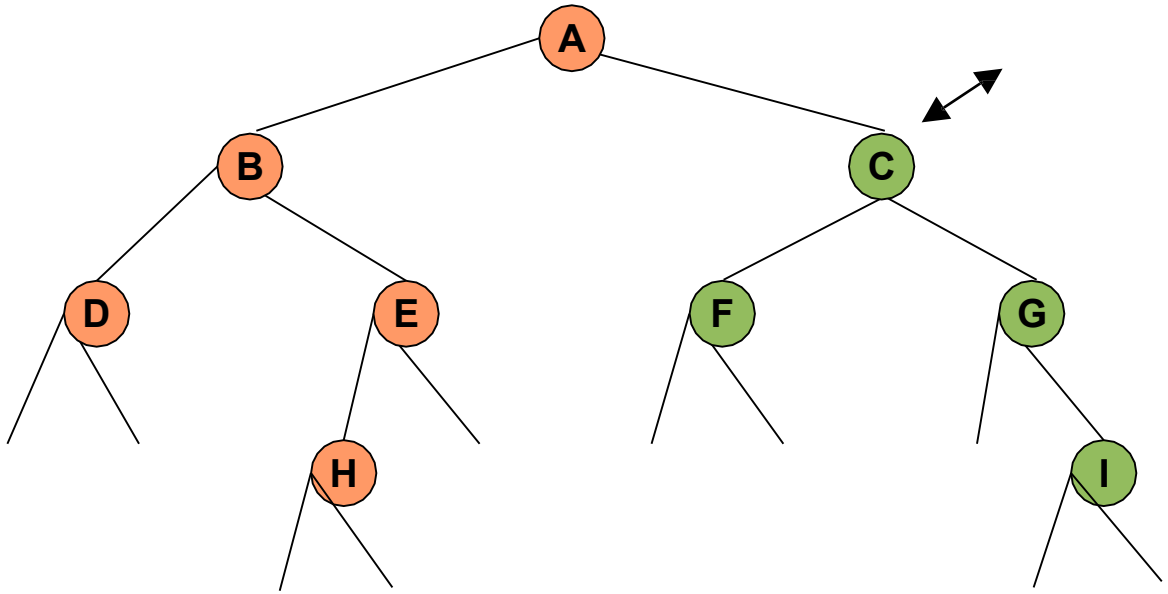
Result: DBHE

Inorder traversal



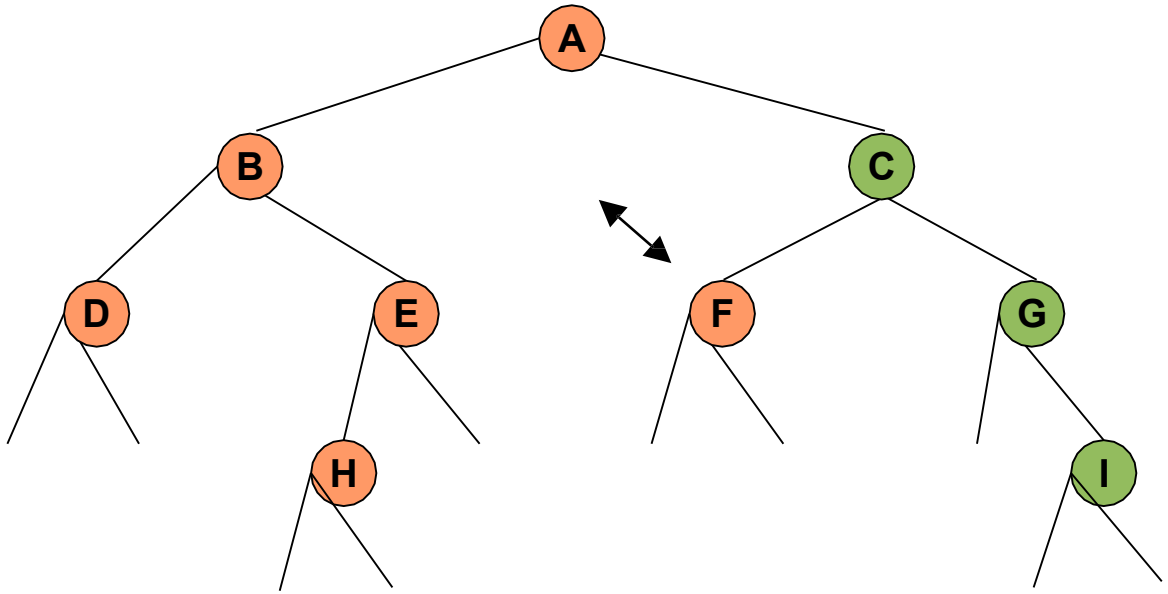
Result: DBHEA

Inorder traversal



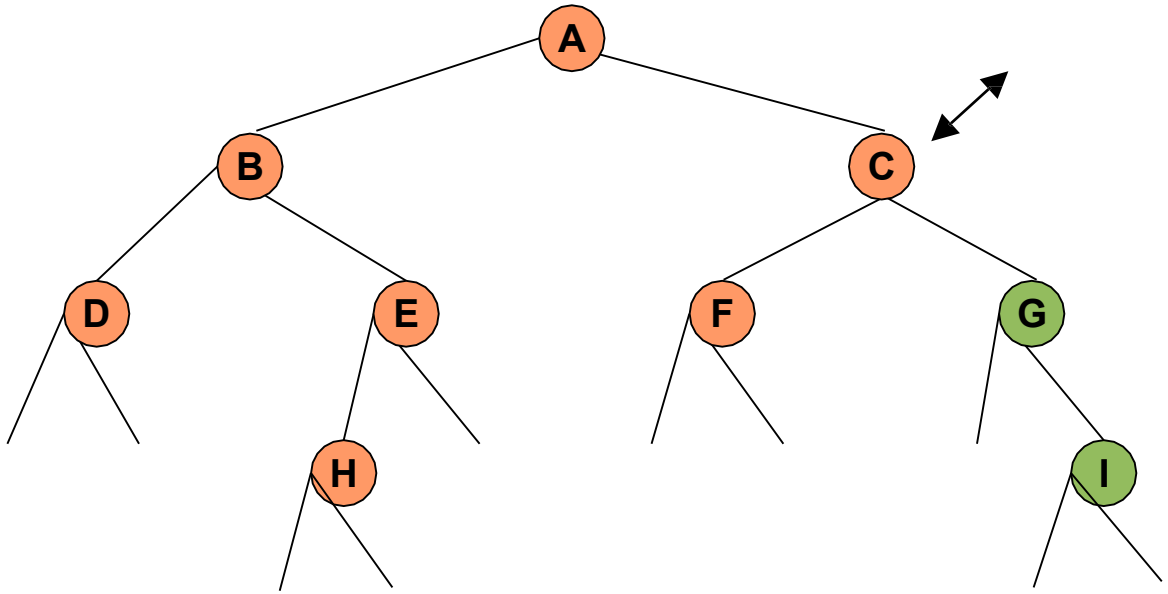
Result: DBHEA

Inorder traversal



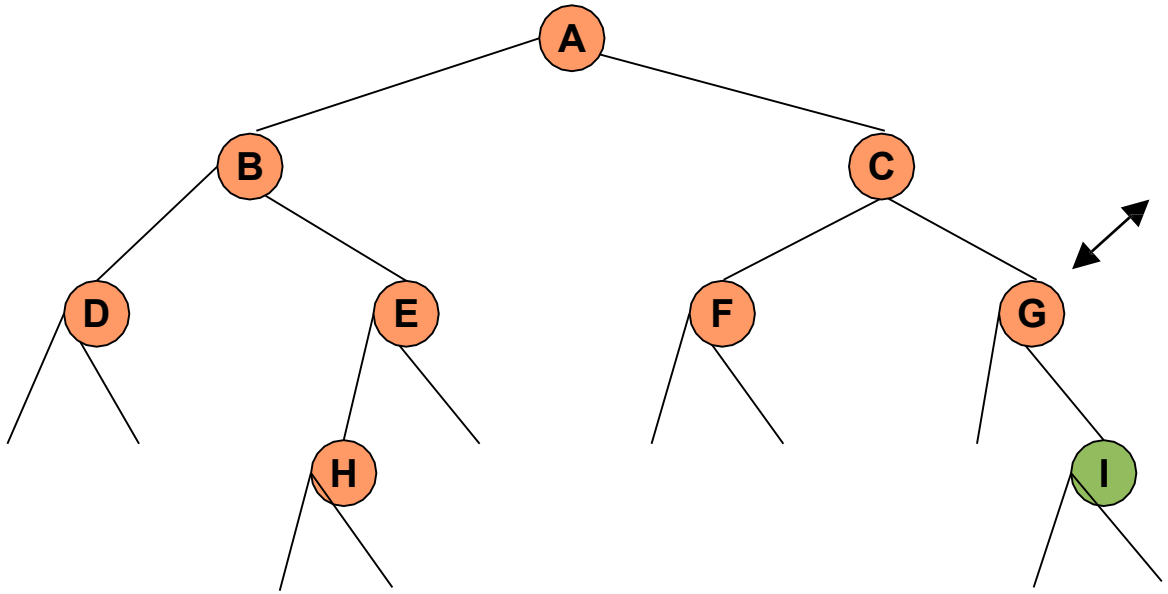
Result: DBHEAF

Inorder traversal



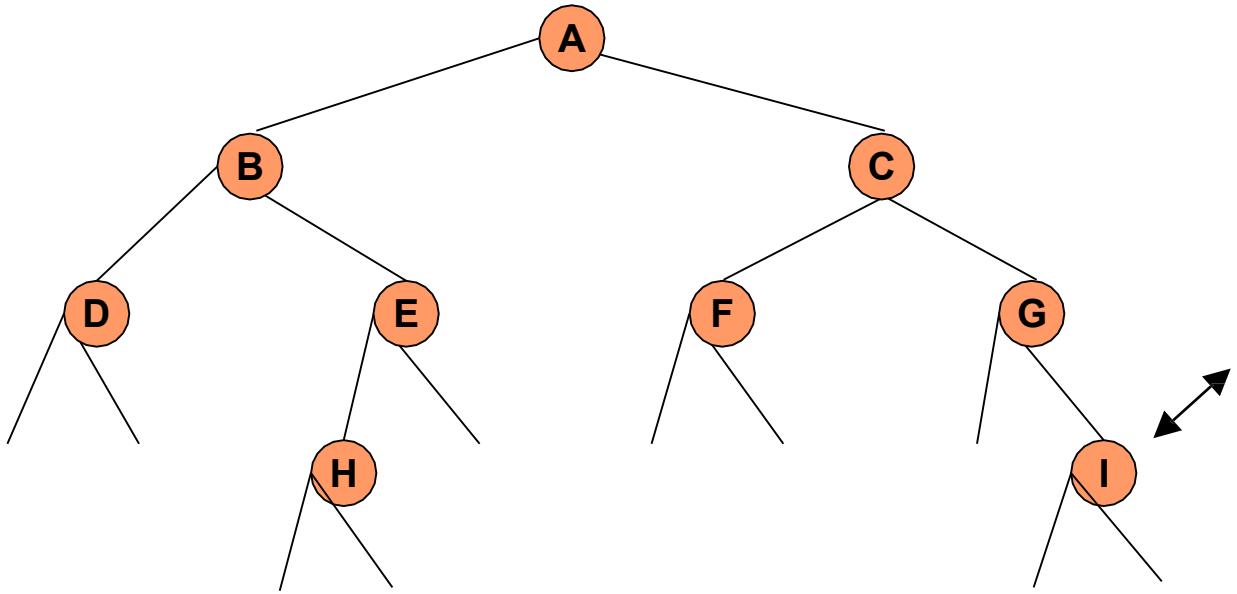
Result: DBHEAFC

Inorder traversal



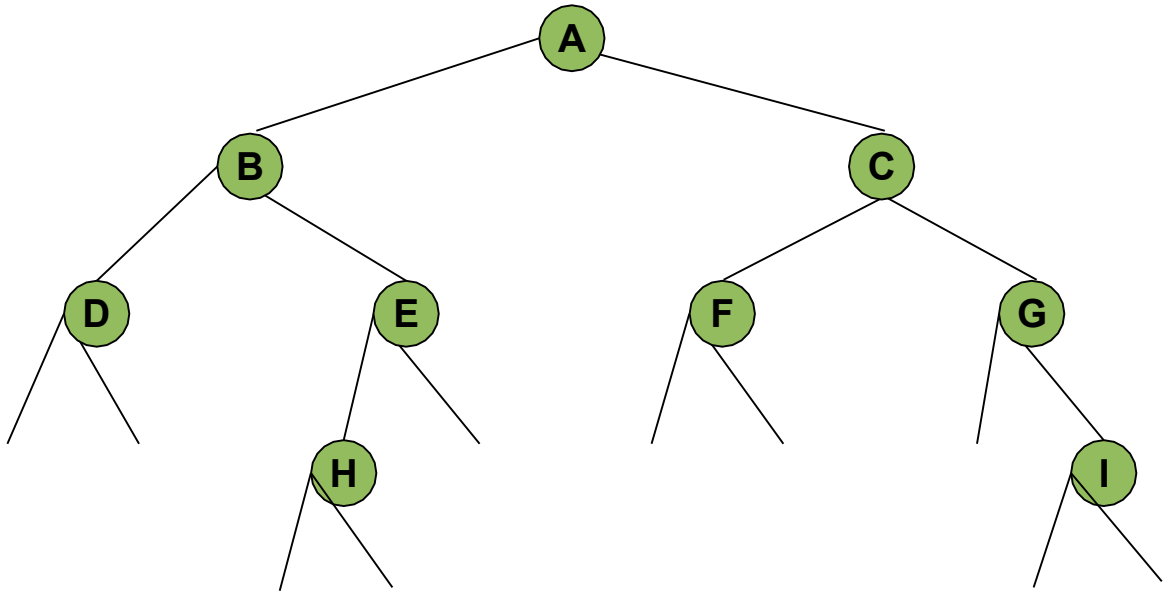
Result: DBHEAFCG

Inorder traversal

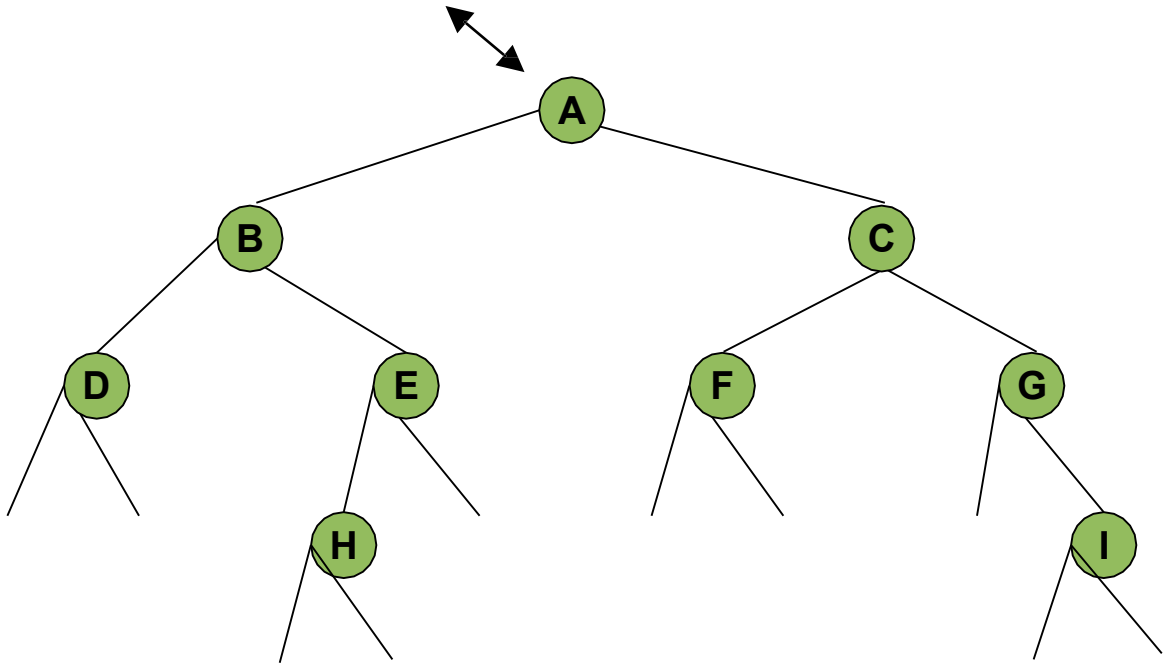


Result: DBHEAFCGI

Postorder traversal

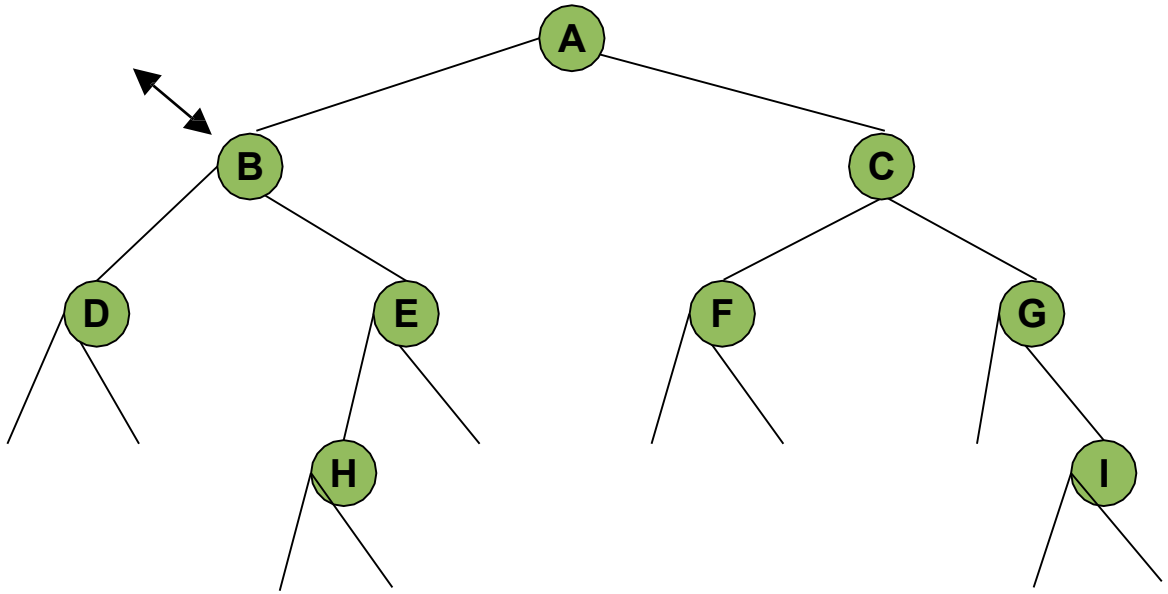


Postorder traversal



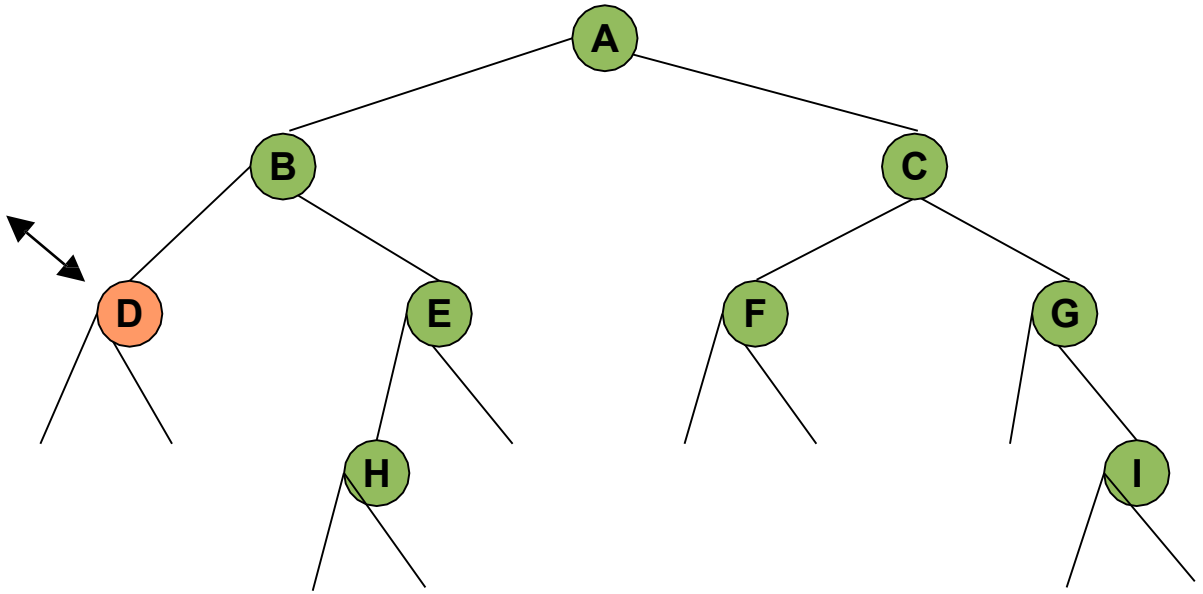
Result:

Postorder traversal



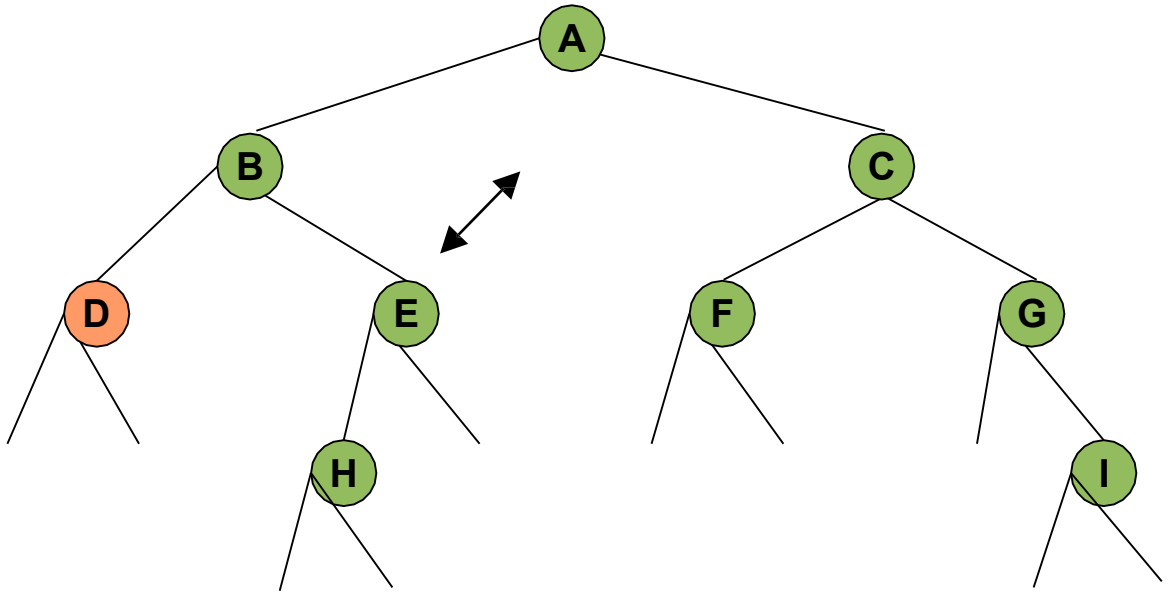
Result:

Postorder traversal



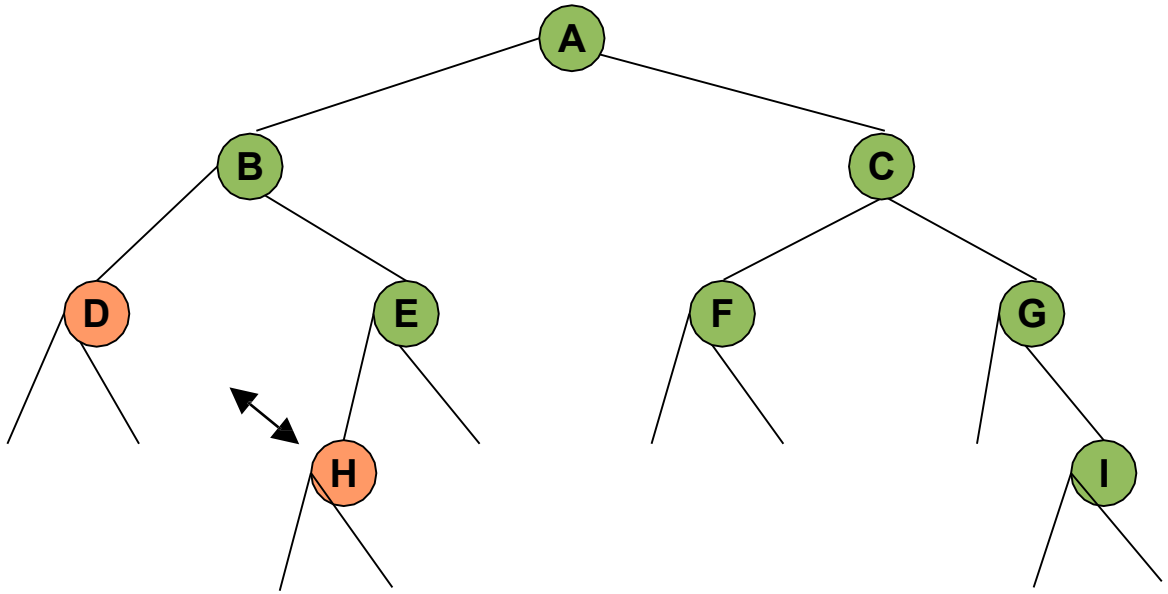
Result: D

Postorder traversal



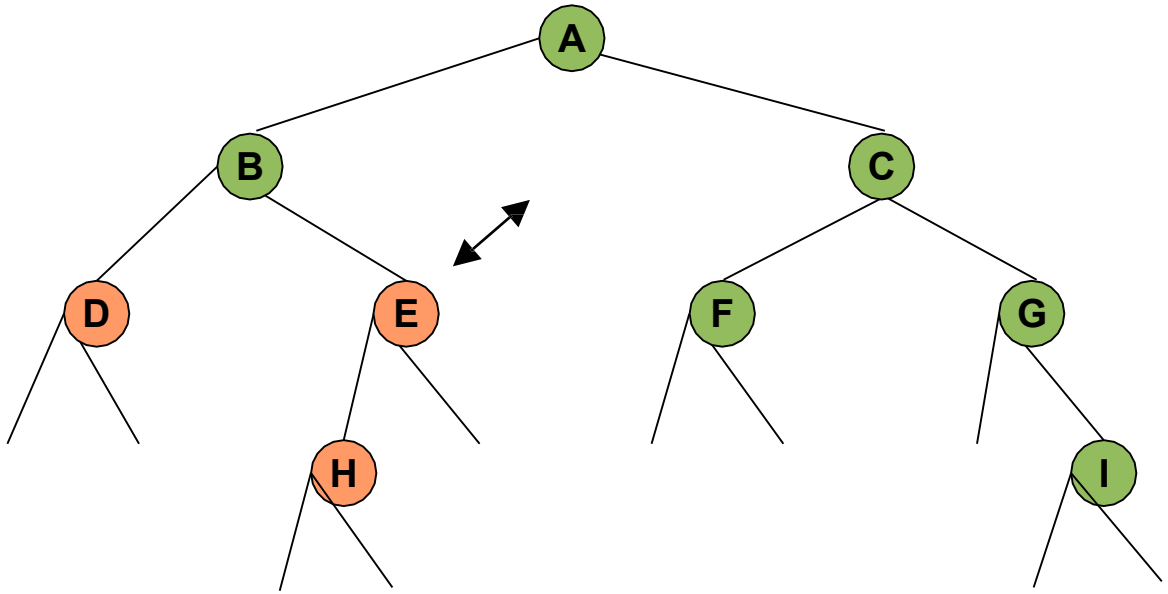
Result: D

Postorder traversal



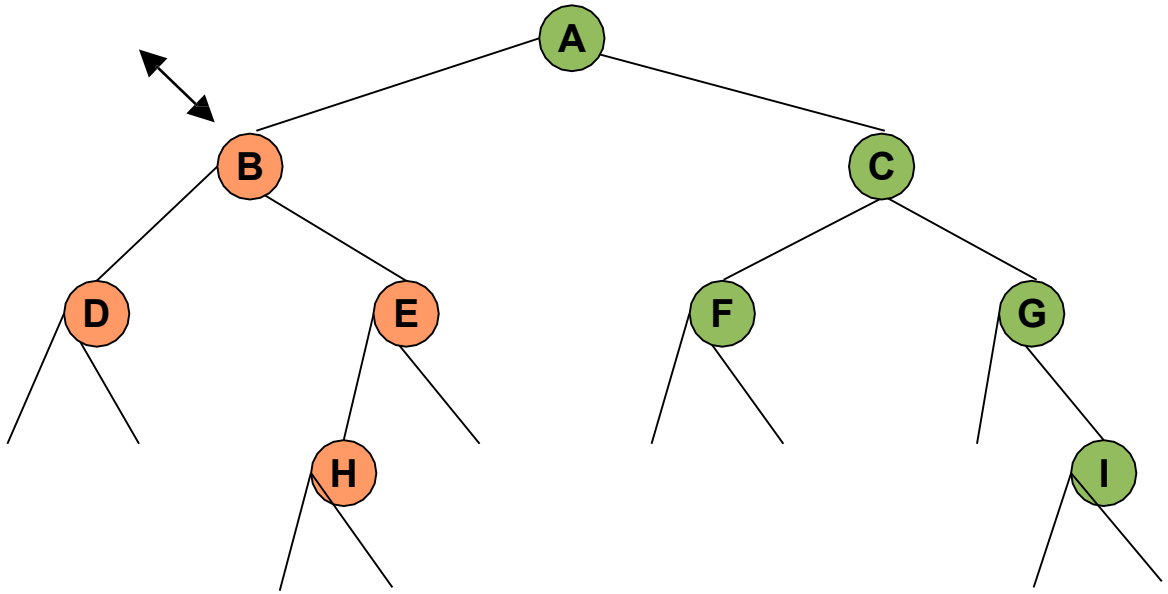
Result: DH

Postorder traversal



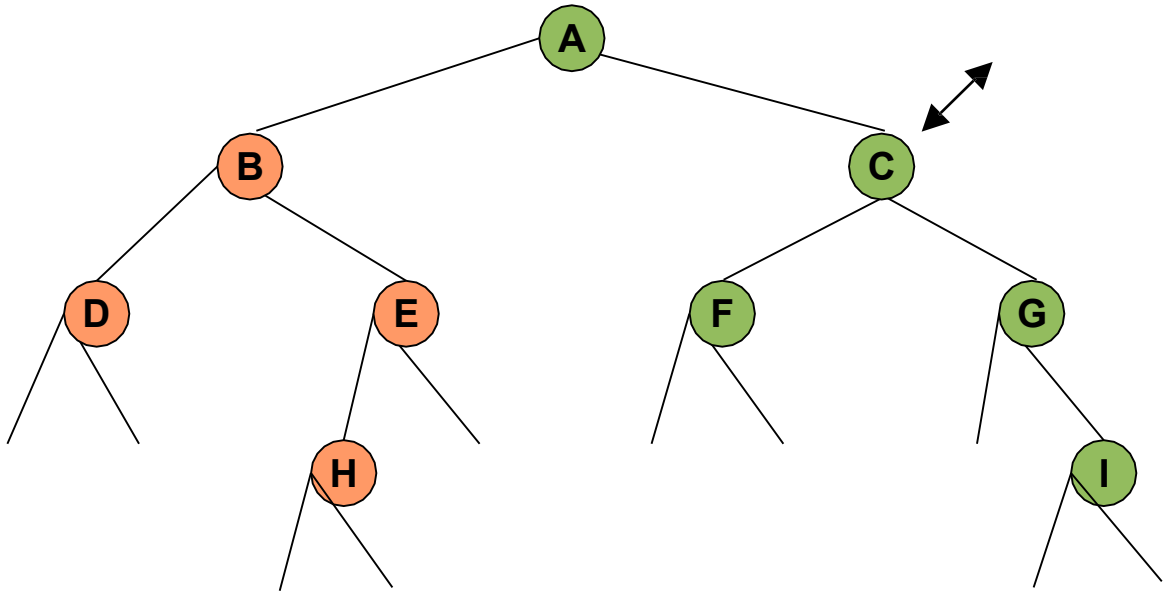
Result: DHE

Postorder traversal



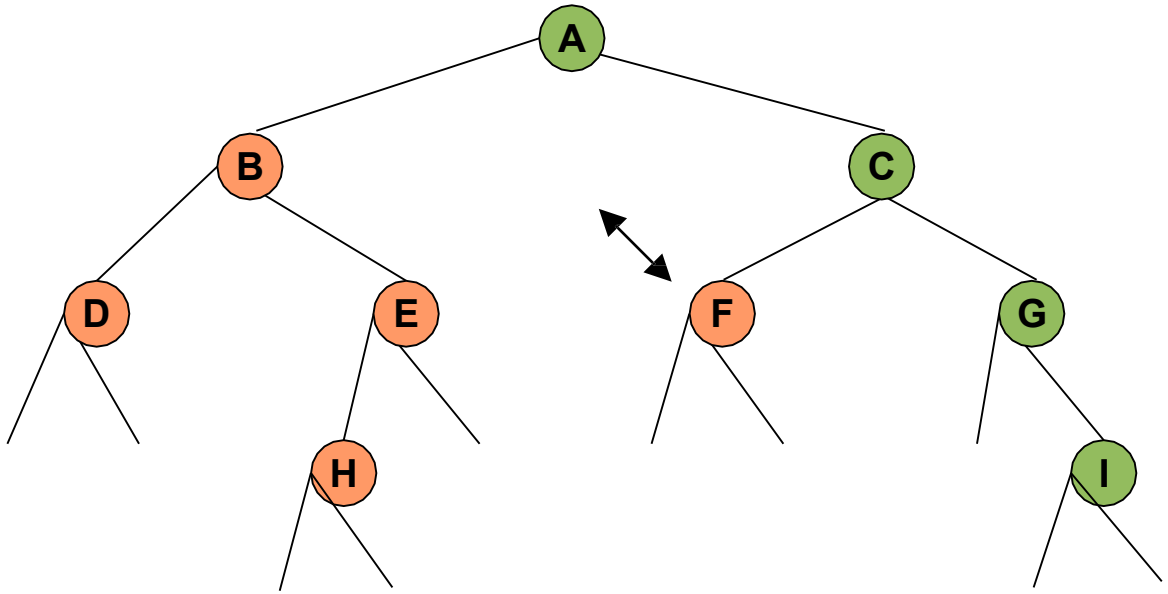
Result: DHEB

Postorder traversal



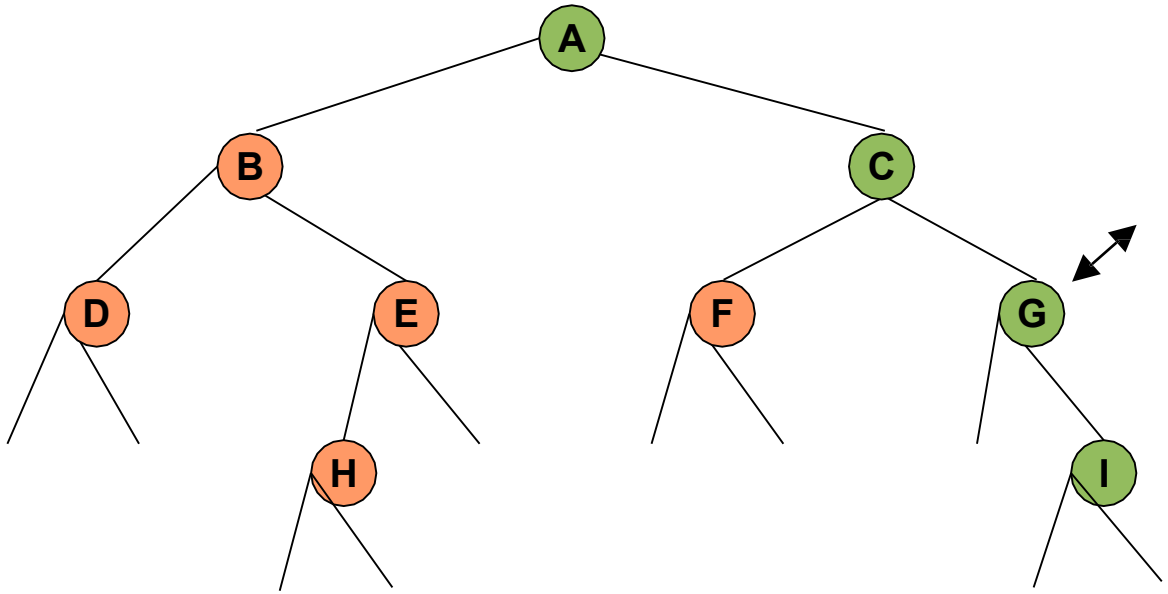
Result: DHEB

Postorder traversal



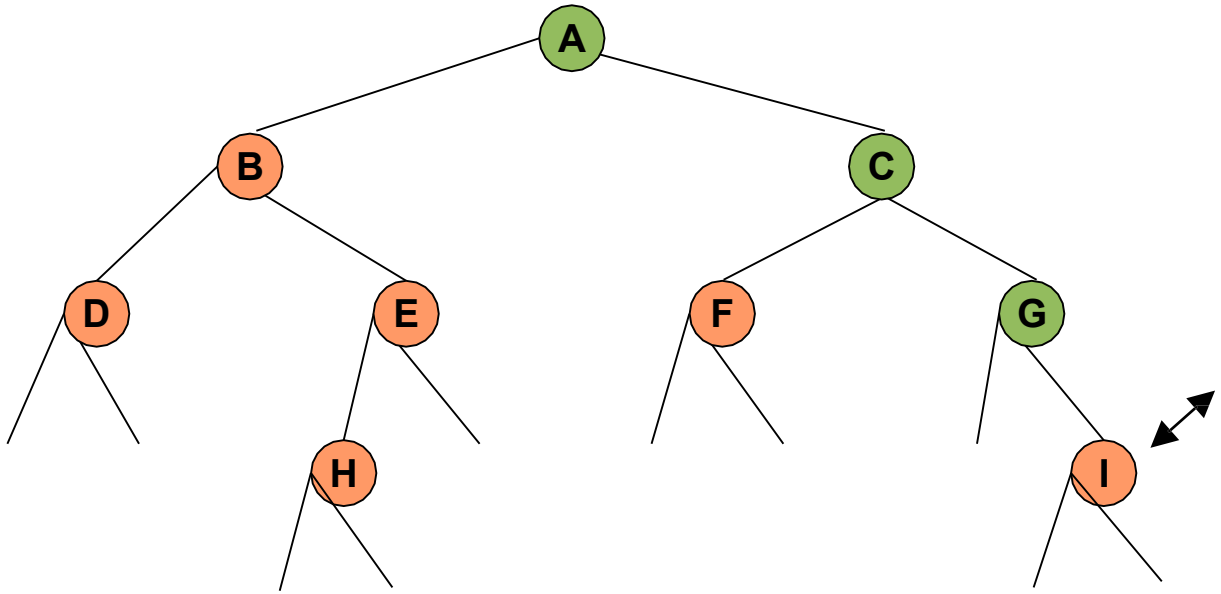
Result: DHEBF

Postorder traversal



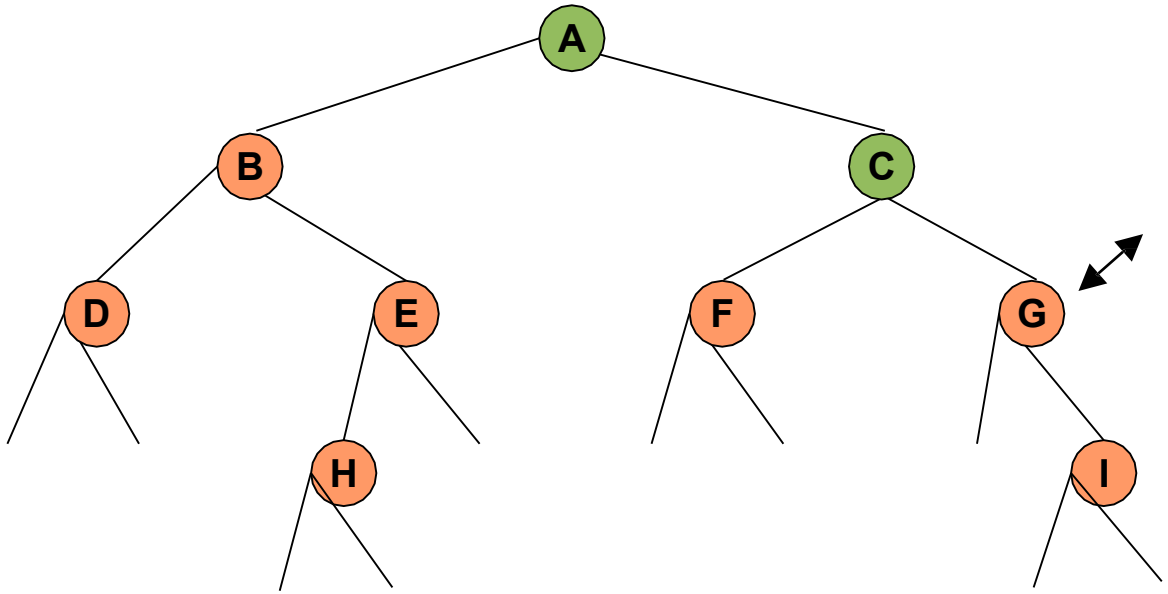
Result: DHEBF

Postorder traversal



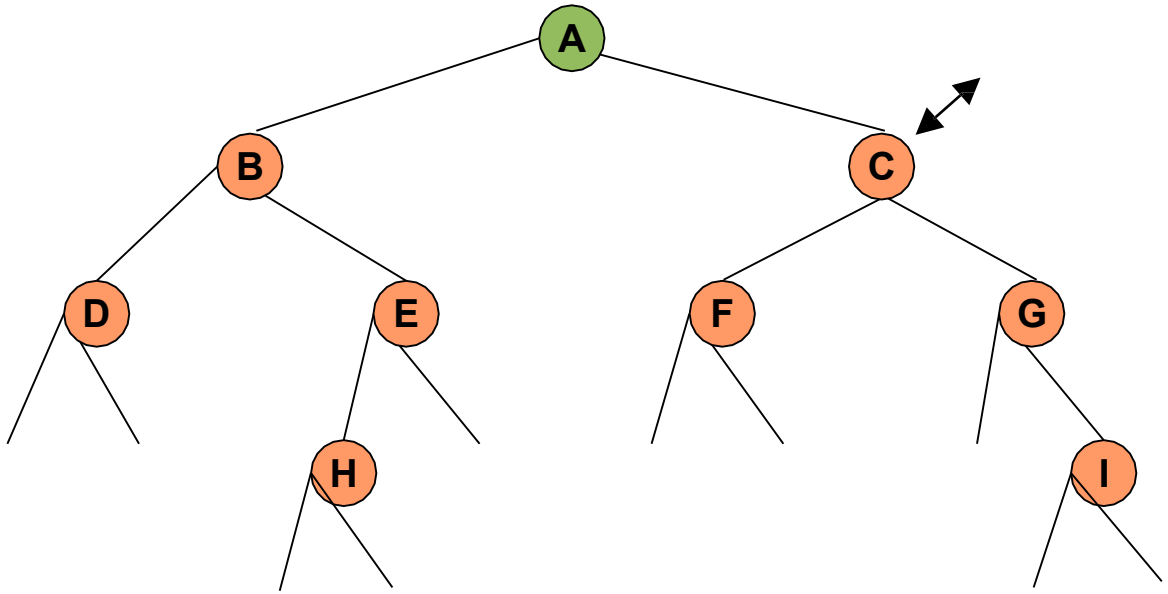
Result: DHEBFI

Postorder traversal



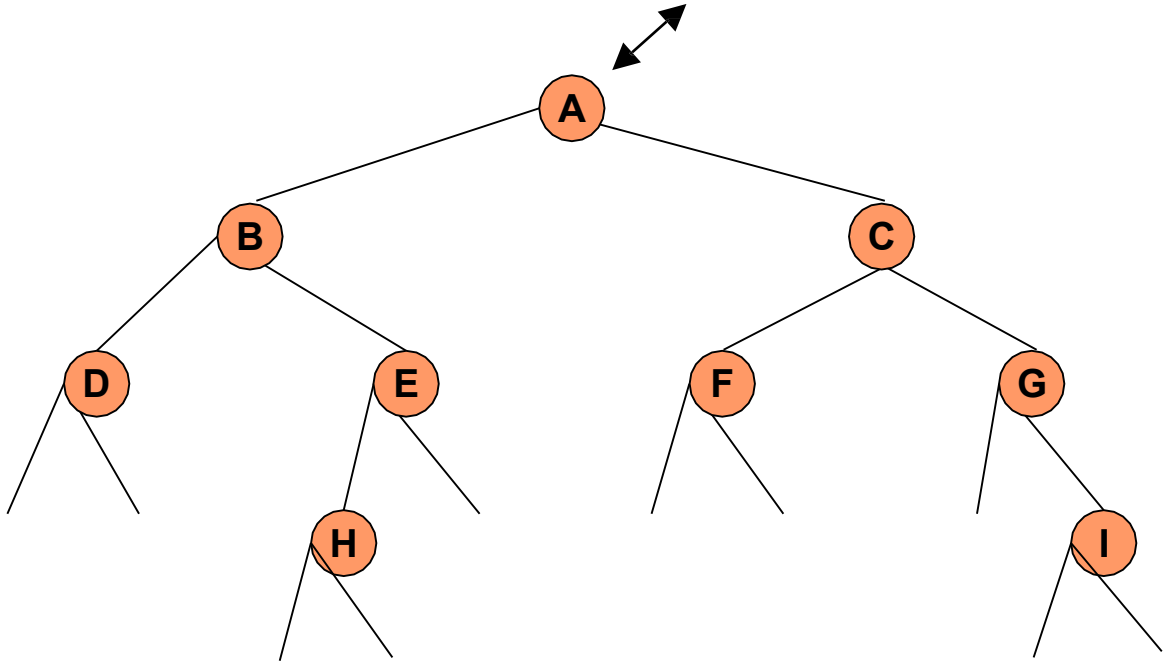
Result: DHEBFIG

Postorder traversal



Result: DHEBFIGC

Postorder traversal



Result: DHEBFIGCA

Asymptotic analysis

Stack

Queue

Linked List

Tree

Thank you